课 程 设 计 报 告

课程名称 计算机程序设计基础2

课题名称 车辆综合管理系统

班 级 无37

学 号 2023010659

姓 名 董皓彧

2024 年 9 月 1 日

目录

1 任务书 **3**

1.1 课程设计目的 . . . 3
1.2 课题题目 . . . 3
1.3 文档设计要求 . . . 3
1.4 程序设计的基本要求 . . . 3

2 系统需求分析 **4**

2.1 系统设计目的与意义 . . . 4
2.2 系统功能需求 . . . 4
2.3 系统流程图 . . . 5

3 系统设计 **5**

3.1 系统架构 . . . 5
3.2 服务端设计 . . . 5
3.3 客户端设计 . . . 6
3.4 通信协议 . . . 6

4 系统实现 **6**

4.1 服务端实现 . . . 6
4.1.1 主要类的简介 . . . 7
4.1.2 主要功能实现 . . . 9
4.1.2.1 车辆类：Car . . . 9
4.1.2.2 车辆池类：CarPool . . . 10
4.1.2.3 账户类：Account . . . 12
4.1.2.4 账户池类：AccountPool . . . 12
4.1.2.5 Base64编码类：Base64 . . . 15
4.1.2.6 哈希类：Hash . . . 15
4.1.2.7 HTTP请求处理类：ServerHttpHandler . . . 15
4.1.2.8 日志类：MyLogger . . . 20
4.1.2.9 json处理 . . . 21
4.1.3 工作流程 . . . 21
4.2 客户端实现 . . . 21
4.2.1 主要类的简介 . . . 21
4.2.2 主要功能实现 . . . 21
4.2.2.1 带有子窗口的界面类：QWidgetWithSubWindows . . . 22
4.2.2.2 各种窗口类：...Window . . . 22
4.2.2.3 HTTP请求发送类：ClientHttpHandler . . . 22
4.2.3 工作流程 . . . 24

5 系统调试与测试 **24**
5.1 系统调试 24
5.2 系统测试 24
5.2.1 单元测试 24
5.2.2 API接口测试 25
5.2.3 跨平台测试 25

6 使用方法 **25**
6.1 软件的获取 25
6.2 服务端使用方法 25
6.3 客户端使用方法 27

7 编程体会 **27**
7.1 面向对象的编程思想 27
7.2 善用前人的轮子 27

A 单元测试代码 **27**
A.1 AccountPool测试 27
A.2 Base64测试 35
A.3 CarPool测试 36
A.4 Hash测试 43

B 源代码 **45**

C 评分表 **246**

1 任务书

1.1 课程设计目的

面向对象程序设计课程设计是集中实践性环节之一，是学习完《面向对象程序设计》课程后进行的一次全面的综合练习。要求学生达到熟练掌握C++语言的基本知识和技能；基本掌握面向对象程序设计的思想和方法；能够利用所学的基本知识和技能，解决简单的面向对象程序设计问题，从而提高动手编程解决实际问题的能力。尤其重视创新思维培养。

1.2 课题题目

车辆综合管理系统

1.3 文档设计要求

1. 设计课题题目：每个同学都单独完成1道课题。后面有范题，仅供同学们参考，不列入本次课程设计的课题。
2. 对于程设题目，按照范题的格式。自行虚构软件需求。并按照第4点要求，编写设计文档。基本要求系统中设计的类的数目不少于4个，每个类中要有各自的属性（多于3个）和方法（多于3个）；需要定义一个抽象类，采用继承方式派生这些类。并设计一个多重继承的派生类。在程序设计中，引入虚函数的多态性、运算符重载等机制。

1.4 程序设计的基本要求

1. 要求利用面向对象的方法以及C++的编程思想来完成系统的设计；
2. 要求在设计的过程中，建立清晰的类层次；
3. 根据课题完成以下主要工作：
   (a) 完成系统需求分析：包括系统设计目的与意义；系统功能需求（系统流程图）；输入输出的要求。
   (b) 完成系统总体设计：包括系统功能分析；系统功能模块划分与设计（系统功能模块图）。
   (c) 完成系统详细设计：数据文件；类层次图；界面设计与各功能模块实现。
   (d) 系统调试：调试出现的主要问题，编译语法错误及修改，重点是运行逻辑问题修改和调整。
   (e) 使用说明书及编程体会：说明如何使用你编写的程序，详细列出每一步的操作步骤。
   (f) 关键源程序（带注释）
4. 车辆综合管理系统应至少包含以下功能：
   - 系统应区分两种用户类型：管理员和普通用户。管理员可以添加、删除、修改和查询车辆记录，而普通用户仅能查询车辆信息。
   - 系统根据用户类型提供不同的功能菜单，其中用户窗口可以实现：
     – 需先登录系统，根据提供的账号信息（如用户名和密码）系统识别用户类型。

  - 允许用户通过车牌号或车型等信息进行车辆查询。
- 管理员入口可以实现下述功能：
  - 添加车辆：输入车辆信息（例如车牌号、车型、颜色、年份等）并保存。
  - 删除车辆：通过车牌号删除车辆记录。
  - 修改车辆信息：允许修改指定车辆的详细信息。
  - 查询车辆：通过车牌号或车型查询车辆信息，并显示详细信息。
- 数据存储：
  - 使用文本文件处理（例如利用fstream库）来存储和读取车辆信息，以实现车辆信息的持久存储。
  - 每辆车的信息可以作为文件中的一行，属性之间可以用逗号分隔。
- 控制台界面：
  - 使用输入输出流（cin和cout）进行用户交互。
  - 在用户登录时检查其类别，并根据类别提供不同的功能菜单，确保普通用户无法访问管理功能。
- 可以自定义其他功能。

5. 自己设计测试数据，将将测试数据存在文件中，通过文件来进行数据读写来测试；
6. 按规定格式完成课程设计报告，并在网络学堂上按时提交；
7. 不得抄袭他人程序、课程设计报告，每个人应独立完成，在程序和设计报告中体现自己的个性设计。

2 系统需求分析

2.1 系统设计目的与意义

本系统是一个车辆综合管理系统，旨在为车辆管理者提供一个便捷的管理工具，帮助管理者更好地管理车辆信息，提高车辆管理效率。

2.2 系统功能需求

本系统的主要功能需求如下：

- **管理车辆**：添加、删除、修改、查询车辆信息。其中车辆信息包括：
  - 车牌号
  - 车辆类型
  - 车辆颜色
  - 车主
  - 车辆年份

- **管理账户**：添加、删除、修改、查询账户信息。其中账户信息包括：
  - 用户名
  - 密码
  - 账户类型（管理员/普通用户）
- **权限控制**：管理员可以对车辆信息和账户信息进行增删改查操作，普通用户只能查询车辆信息。

2.3 系统流程图

本系统分为服务端和客户端两部分，服务端负责处理存储相关数据，客户端负责与用户交互。系统流程图如图1所示：

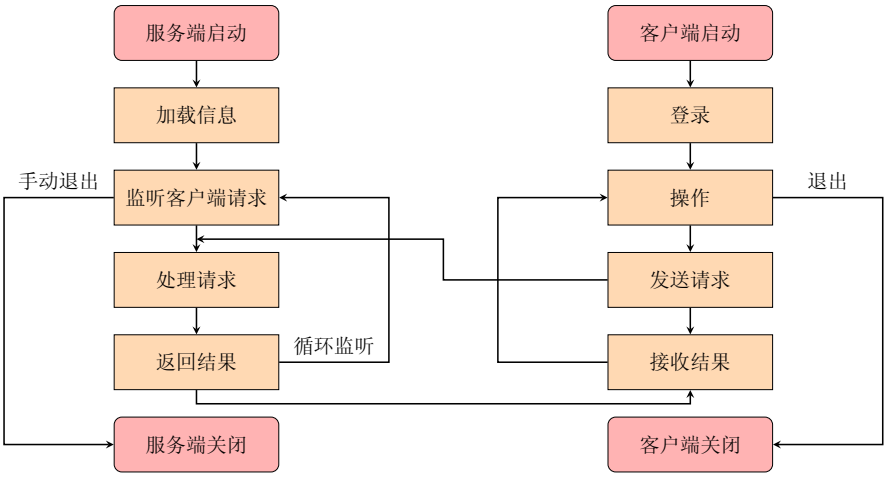


图 1: 系统流程图

3 系统设计

3.1 系统架构

本系统分为服务端和客户端两部分，服务端负责处理存储相关数据，客户端负责与用户交互。

3.2 服务端设计

服务端主要包括以下几个模块：

- **数据存储模块**：在数据结构层面实现信息的存储和处理，提供增删改查操作的接口，并提供数据持久化功能。
- **请求通信模块**：接收客户端请求，调用数据存储模块进行处理，并返回结果。

3.3 客户端设计

客户端主要包括以下几个模块：

- **图形界面**：提供用户操作界面，使用Qt6进行界面设计：
  - 身份认证相关：
    - 服务器连接界面
    - 登录界面
    - 修改密码界面
  - 选择操作界面：
    - 管理员操作界面
    - 普通用户操作界面
  - 车辆相关：
    - 添加车辆界面
    - 删除车辆界面
    - 修改车辆界面
    - 查询车辆界面
    - 查询车辆结果界面
  - 账户相关：
    - 选择操作界面
    - 添加账户界面
    - 删除账户界面
    - 修改账户界面
    - 查询账户界面
    - 查询账户结果界面
- **请求通信模块**：向服务端发送请求，并接收处理结果。

3.4 通信协议

服务端与客户端之间采用HTTP协议进行通信。服务端暴露若干个API接口，客户端通过HTTP请求访问这些接口进行相应操作。

4 系统实现

4.1 服务端实现

使用C++编写服务端程序。采用面向对象的设计思想。

4.1.1 主要类的简介

服务端程序主要包括以下几个类：

- **Car**：车辆类，包含车辆的基本信息。
- **Account**：账户类，包含账户的基本信息。
- **BasicPool**：池基类，为纯虚类，定义了一个池需要的基本接口。
- **CarPool**：车辆池类，继承自BasePool，实现了车辆信息的增删改查操作。
- **AccountPool**：账户池类，继承自BasePool，实现了账户信息的增删改查操作。
- **Base64**：Base64编码类，实现了Base64编码和解码。
- **Hash**：哈希类，实现了加盐的sha256哈希算法。
- **ServerHttpHandler**：HTTP请求处理类，处理客户端请求。
- **MyLogger**：日志类，实现了日志的记录。

各个类之间的关系图如图2所示：

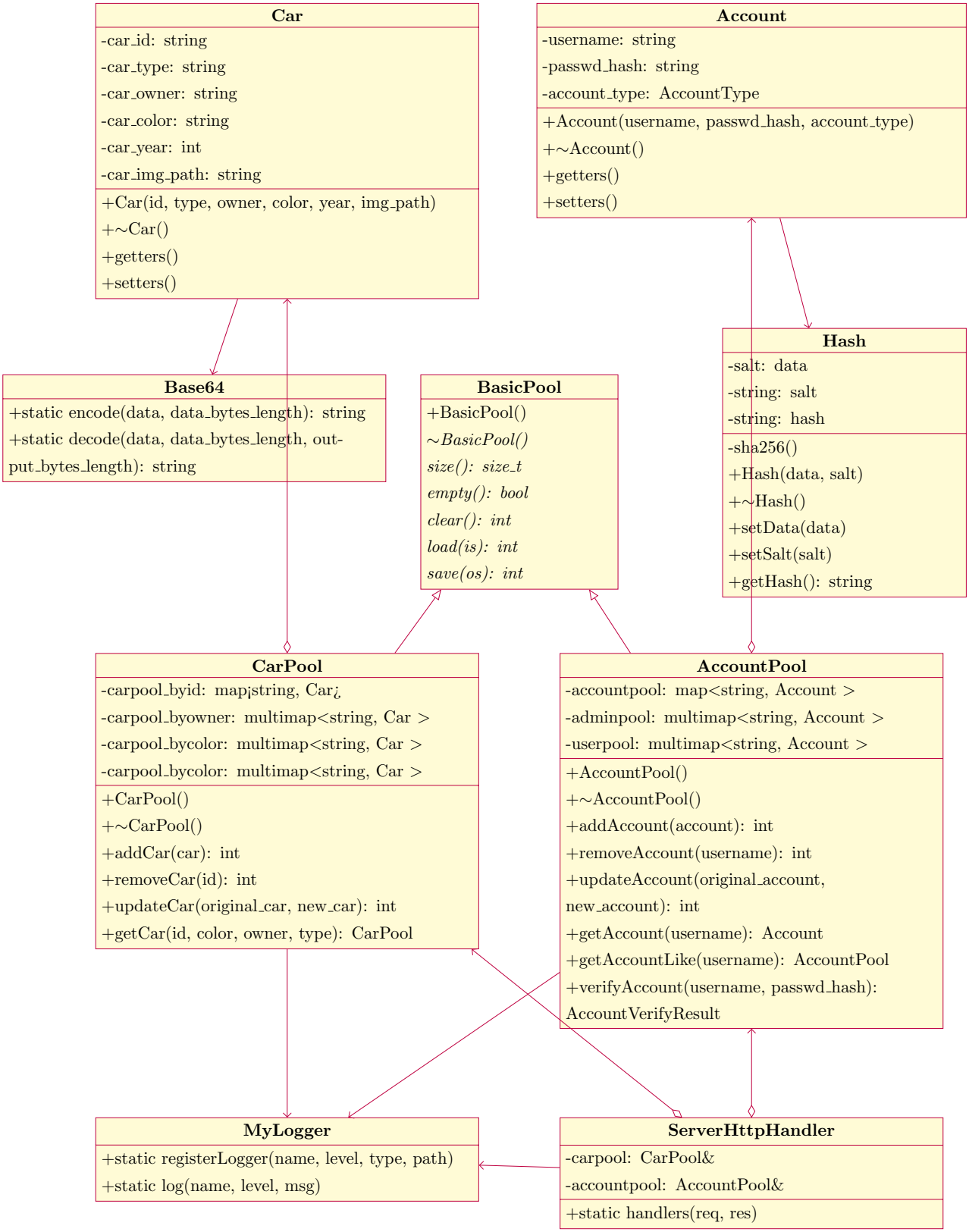

Car
-car_id: string
-car_type: string
-car_owner: string
-car_color: string
-car_year: int
-car_img_path: string
+Car(id, type, owner, color, year, img_path)
+~Car()
+getters()
+setters()
Account
-username: string
-passwd_hash: string
-account_type: AccountType
+Account(username, passwd_hash, account_type)
+~Account()
+getters()
+setters()
Base64
+static encode(data, data_bytes_length): string
+static decode(data, data_bytes_length, output_bytes_length): string
BasicPool
+BasicPool()
~BasicPool()
size(): size_t
empty(): bool
clear(): int
load(is): int
save(os): int
Hash
-salt: data
-string: salt
-string: hash
-sha256()
+Hash(data, salt)
+~Hash()
+setData(data)
+setSalt(salt)
+getHash(): string
CarPool
-carpool_byid: map¡string, Car¿
-carpool_byowner: multimap<string, Car >
-carpool_bycolor: multimap<string, Car >
-carpool_bycolor: multimap<string, Car >
+CarPool()
+~CarPool()
+addCar(car): int
+removeCar(id): int
+updateCar(original_car, new_car): int
+getCar(id, color, owner, type): CarPool
AccountPool
-accountpool: map<string, Account >
-adminpool: multimap<string, Account >
-userpool: multimap<string, Account >
+AccountPool()
+~AccountPool()
+addAccount(account): int
+removeAccount(username): int
+updateAccount(original_account, new_account): int
+getAccount(username): Account
+getAccountLike(username): AccountPool
+verifyAccount(username, passwd_hash): AccountVerifyResult
MyLogger
+static registerLogger(name, level, type, path)
+static log(name, level, msg)
ServerHttpHandler
-carpool: CarPool&
-accountpool: AccountPool&
+static handlers(req, res)

图 2: 服务端类关系图

4.1.2 主要功能实现

4.1.2.1 车辆类：Car

车辆类Car包含车辆的基本信息，包括车牌号、车辆类型、车主、车辆颜色、车辆年份、车辆图片路径等。

Car类包含以下成员变量：

- **car_id**：车牌号（私有），为std::string类型，用于唯一标识车辆。
- **car_type**：车辆类型（私有），为std::string类型。
- **car_owner**：车主（私有），为std::string类型。
- **car_color**：车辆颜色（私有），为std::string类型。
- **car_year**：车辆年份（私有），为int类型。
- **car_img_path**：车辆图片路径（私有），为std::string类型，根据此路径读取车辆图片。

Car类包含以下成员函数：

- **Car(std::string &id, std::string &type, std::string &owner, std::string &color, int year, std::string &img_path)**：构造函数，初始化车辆信息。
- **Car(const Car &c)**：拷贝构造函数，拷贝车辆信息。
- **~Car()**：析构函数，释放资源。
- **void setId(const std::string &id)**：设置车牌号。
- **void setType(const std::string &type)**：设置车辆类型。
- **void setOwner(const std::string &owner)**：设置车主。
- **void setColor(const std::string &color)**：设置车辆颜色。
- **void setYear(const int year)**：设置车辆年份。
- **void setImgPath(const std::string &img_path)**：设置车辆图片路径。
- **std::string getId() const**：获取车牌号。
- **std::string getType() const**：获取车辆类型。
- **std::string getOwner() const**：获取车主。
- **std::string getColor() const**：获取车辆颜色。
- **int getYear() const**：获取车辆年份。
- **std::string getImgPath() const**：获取车辆图片路径。

以及为对Car类型进行复制、比较等的操作符重载，如=、==、!=、<、>等。

4.1.2.2 车辆池类：CarPool

车辆池类CarPool用于管理车辆信息。CarPool类继承自BasicPool类，实现了车辆信息的增删改查操作。CarPool内部通过std::map和std::multimap存储车辆信息，以车牌号、车主、车辆颜色、车辆类型等为键，实现了时间复杂度为$O(\log n)$的增删改查操作。

CarPool类包含以下成员变量：

- **carpool_byid**：以车牌号为键的map（私有），存储车辆信息。
- **carpool_byowner**：以车主为键的multimap（私有），存储车辆信息。
- **carpool_bycolor**：以车辆颜色为键的multimap（私有），存储车辆信息。
- **carpool_bytype**：以车辆类型为键的multimap（私有），存储车辆信息。

CarPool类包含以下成员函数：

- **CarPool()**：构造函数，初始化空车辆池。
- **CarPool(Car *begin, Car *end)**：构造函数，通过指向车辆信息的指针初始化车辆池。
- **CarPool(const std::vector<Car>&cars)**：构造函数，通过车辆信息的vector初始化车辆池。
- **CarPool(const CarPool &cp)**：拷贝构造函数，拷贝车辆池。
- **~CarPool()**：析构函数，释放资源。
- **int addCar(const Car &car)**：添加车辆信息。返回值为一个int类型状态码，含义如下：
  - 0：添加成功。
  - 0x70：车辆已存在。
  - 0x7F：未知错误。
- **int removeCar(const Car &car)**：删除车辆信息。返回值为一个int类型状态码，含义如下：
  - 0：删除成功。
  - 0x80：车辆不存在。
  - 0x8F：未知错误。
- **int removeCar(const std::string &id)**：删除车辆信息。返回值同上。
- **int updateCar(const Car &original_car, const Car &new_car)**：更新车辆信息。返回值为一个int类型状态码，含义如下：
  - 0：更新成功。
  - 0x90：删除原车辆失败。
  - 0x91：删除原车辆成功，添加新车辆失败。
  - 0x9F：未知错误。
- **int updateCar(const std::string &id, const Car &new_car)**：更新车辆信息。返回值同上。

- **CarPool getCarById(const std::string &id)**：通过车牌号查询车辆信息，返回一个CarPool对象，其中包含查询到的车辆信息。由于维护了carpool_byid，时间复杂度为O($\log n$)。
- **CarPool getCarByColor(const std::string &color)**：通过车辆颜色查询车辆信息，返回一个CarPool对象，其中包含查询到的车辆信息。由于维护了carpool_bycolor，时间复杂度为O($\log n$)。
- **CarPool getCarByOwner(const std::string &owner)**：通过车主查询车辆信息，返回一个CarPool对象，其中包含查询到的车辆信息。由于维护了carpool_byowner，时间复杂度为O($\log n$)。
- **CarPool getCarByType(const std::string &type)**：通过车辆类型查询车辆信息，返回一个CarPool对象，其中包含查询到的车辆信息。由于维护了carpool_bytype，时间复杂度为O($\log n$)。
- **CarPool getCar(const std::string &id, const std::string &color, const std::string &owner, const std::string &type)**：通过车牌号、车主、车辆颜色、车辆类型查询车辆信息，返回一个CarPool对象，其中包含符合所有条件的车辆信息。时间复杂度为O($\log n$)。若某个条件为空，则不对该条件进行筛选。
- **size_t size() const**：获取车辆池大小。
- **bool empty() const**：判断车辆池是否为空。
- **int clear()**：清空车辆池。返回值为一个int类型状态码，含义如下。
  - 0：清空成功。
  - 0xAF：未知错误。
- **int load(std::istream &is)**：从输入流加载车辆信息。输入流应当是一个文本流，其内容为给定格式的json字符串。返回值为一个int类型状态码，含义如下：
  - 0：加载成功。
  - 0xB0：输入流不可读。
  - 0xB1：CarPool清空失败。
  - 0xB2：json对象解析失败。
  - 0xB3：json对象解析成功，但不包含所需字段。
  - 0xB4：json对象解析成功，包含所需字段，但所需字段类型错误。
  - 0xB5：添加车辆失败。
  - 0xBF：未知错误。
- **int save(std::ostream &os)**：将车辆信息保存到输出流。输出内容为给定格式的json字符串，可通过load函数加载。返回值为一个int类型状态码，含义如下：
  - 0：保存成功。
  - 0xC0：输出流不可写。
  - 0xCF：未知错误。
- **std::vector<Car>list() const**：将车辆信息转换为vector类型。

以及对CarPool进行赋值、比较的操作符重载：=、==、!=。

4.1.2.3 账户类：Account

账户类Account包含账户的基本信息，包括用户名、密码哈希、账户类型（管理员、普通用户）等。为了安全起见，不保存明文密码，而是保存加盐后的sha256哈希值。
Account类包含以下成员变量：

- **username**：用户名（私有），为std::string类型，用于唯一标识账户。
- **passwd_hash**：密码哈希（私有），为std::string类型，保存加盐后的sha256哈希值。
- **account_type**：账户类型（私有），为AccountType枚举类型，表示账户类型。其中，AccountType枚举类型包括：
  - NONETYPE = 0：未知类型。
  - ADMIN = 1：管理员。
  - USER = 2：普通用户。

Account类包含以下成员函数：

- **Account(const std::string &username, const std::string &passwd_hash, const AccountType account_type)**：构造函数，初始化账户信息。
- **Account(const Account &acc)**：拷贝构造函数，拷贝账户信息。
- **~Account()**：析构函数，释放资源。
- **void setUsername(const std::string &username)**：设置用户名。
- **void setPasswdHash(const std::string &passwd_hash)**：设置密码哈希。
- **void setAccountType(const AccountType account_type)**：设置账户类型。
- **std::string getUsername() const**：获取用户名。
- **std::string getPasswdHash() const**：获取密码哈希。
- **AccountType getAccountType() const**：获取账户类型。
- **bool isAdmin() const**：判断是否为管理员。
- **bool isUser() const**：判断是否为普通用户。

以及为对Account类型进行复制、比较等的操作符重载，如=、==、!=、<、>等。

4.1.2.4 账户池类：AccountPool

账户池类AccountPool用于管理账户信息。AccountPool类继承自BasicPool类，实现了账户信息的增删改查操作。AccountPool内部通过std::map和存储账户信息，以用户名为键，实现了时间复杂度为$O(\log n)$的增删改查操作。
AccountPool类包含以下成员变量：

- **accountpool**：以用户名为键的map（私有），存储账户信息。

- **adminpool**：以用户名为键的map（私有），存储管理员账户信息。
- **userpool**：以用户名为键的map（私有），存储普通用户账户信息。

AccountPool类包含以下成员函数：

- **AccountPool()**：构造函数，初始化空账户池。
- **AccountPool(Account *begin, Account *end)**：构造函数，通过指向账户信息的指针初始化账户池。
- **AccountPool(const std::vector<Account>&accounts)**：构造函数，通过账户信息的vector初始化账户池。
- **AccountPool(const AccountPool &ap)**：拷贝构造函数，拷贝账户池。
- **~AccountPool()**：析构函数，释放资源。
- **int addAccount(const Account &acc)**：添加账户信息。返回值为一个int类型状态码，含义如下：
  - 0：添加成功。
  - 0x10：账户已存在。
  - 0x1F：未知错误。
- **int removeAccount(const Account &acc)**：删除账户信息。返回值为一个int类型状态码，含义如下：
  - 0：删除成功。
  - 0x20：账户不存在。
  - 0x2F：未知错误。
- **int removeAccount(const std::string &username)**：删除账户信息。返回值同上。
- **int updateAccount(const Account &original_account, const Account &new_account)**：更新账户信息。返回值为一个int类型状态码，含义如下：
  - 0：更新成功。
  - 0x30：删除原账户失败。
  - 0x31：删除原账户成功，添加新账户失败。
  - 0x3F：未知错误。
- **int updateAccount(const std::string &username, const Account &new_account)**：更新账户信息。返回值同上。
- **Account getAccount(const std::string &username)**：通过用户名查询账户信息，返回一个Account对象，其为查询到的账户信息。时间复杂度为$O(\log n)$。

- **AccountPool getAccountLike(const std::string &username)**：通过用户名模糊查询账户信息，返回一个AccountPool对象，其中包含查询到的账户信息。时间复杂度为O(n)。
- **AccountVerifyResult verifyAccount(const std::string &username, const std::string &passwd_hash)** 验证账户信息。返回一个AccountVerifyResult对象，包含验证结果。时间复杂度为O($\log n$)。其中，AccountVerifyResult对象取值如下：
  - SUCCESS = 0：验证成功。
  - ACCOUNT_NOT_FOUND = 1：用户不存在。
  - WRONG_PASSWORD = 2：密码错误。
- **AccountType getAccountType(const std::string &username)**：获取账户类型。时间复杂度为O($\log n$)。
- **size_t size() const**：获取账户池大小。
- **bool empty() const**：判断账户池是否为空。
- **int clear()**：清空账户池。返回值为一个int类型状态码，含义如下：
  - 0：清空成功。
  - 0x4F：未知错误。
- **int load(std::istream &is)**：从输入流加载账户信息。输入流应当是一个文本流，其内容为给定格式的json字符串。返回值为一个int类型状态码，含义如下：
  - 0：加载成功。
  - 0x50：输入流不可读。
  - 0x51：AccountPool清空失败。
  - 0x52：json对象解析失败。
  - 0x53：json对象解析成功，但不包含所需字段。
  - 0x54：json对象解析成功，包含所需字段，但所需字段类型错误。
  - 0x55：添加账户失败。
  - 0x5F：未知错误。
- **int save(std::ostream &os)**：将账户信息保存到输出流。输出内容为给定格式的json字符串，可通过load函数加载。返回值为一个int类型状态码，含义如下：
  - 0：保存成功。
  - 0x60：输出流不可写。
  - 0x6F：未知错误。
- **std::vector<Account>list() const**：将账户信息转换为vector类型。

以及对AccountPool进行赋值、比较的操作符重载：=、==、!=。

4.1.2.5 Base64编码类：Base64

Base64编码类Base64实现了Base64编码和解码。该类用于存储车辆图片时对图片名进行编码，以防止特殊字符导致的文件名错误。

Base64类包含以下成员函数：

- **static std::string encode(const void *data, size_t data_bytes_length)**：对数据进行Base64编码。返回一个std::string类型的Base64编码字符串。
- **static std::string decode(const void *data, size_t data_bytes_length, size_t output_bytes_length)**：对Base64编码字符串进行解码。返回一个std::string类型的解码字符串。参数中传入的output_bytes_length为解码后的数据长度。

4.1.2.6 哈希类：Hash

哈希类Hash实现了加盐的sha256哈希算法。该类用于存储账户密码时对密码进行哈希，以保证密码安全。

Hash类包含以下成员变量：

- **salt**：盐（私有），为std::string类型，用于加盐。
- **data**：数据（私有），为std::string类型，用于哈希。
- **hash**：哈希值（私有），为std::string类型，保存哈希值。

Hash类包含以下成员函数：

- **Hash(const std::string &data, const std::string &salt)**：构造函数，初始化哈希类。
- **~Hash()**：析构函数，释放资源。
- **void sha256()**：进行sha256哈希（私有函数）。将data和salt拼接后进行sha256哈希，保存到hash中。
- **void setData(const std::string &data)**：设置数据。设置后会重新计算哈希值。
- **void setSalt(const std::string &salt)**：设置盐，设置后会重新计算哈希值。
- **std::string getHash() const**：获取哈希值。

4.1.2.7 HTTP请求处理类：ServerHttpHandler

HTTP请求处理类ServerHttpHandler用于处理客户端请求。

该类依赖于第三方开源库cpp-httplib，github地址：`https://github.com/yhirose/cpp-httplib`。ServerHttpHandler类包含以下成员变量：

- **carpool**：车辆池（私有），为CarPool类型的引用，用于处理车辆信息方面的请求。
- **accountpool**：账户池（私有），为AccountPool类型的引用，用于处理账户信息方面的请求。
- **imgDir**：图片目录（私有），为std::string类型，为车辆图片本地存储目录。

ServerHttpHandler类包含以下成员函数：

- **json parse_post_body(const httplib::Request &req) const**：解析POST请求体。返回一个json对象，包含POST请求体的内容。（私有）
- **void handler_test_connection(const httplib::Request &req, httplib::Response &res) const**：处理测试连接请求。HTTP请求方法为GET/POST，无请求参数。HTTP响应如下：
  - 连接正常：状态码：200；信息："Connection is OK"（text/plain类型）
- **void handler_login(const httplib::Request &req, httplib::Response &res) const**：处理登录请求。HTTP请求方法为POST，请求参数如下：
  - username：用户名
  - passwd_hash：密码哈希

  HTTP响应如下：
  - 登录成功：状态码：200；信息：包含登录账户信息的json格式字符串（application/json类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Account Not Found or Wrong Password"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）
- **void handler_change_password(const httplib::Request &req, httplib::Response &res)**：处理修改密码请求。HTTP请求方法为POST，请求参数如下：
  - username：用户名
  - old_passwd_hash：旧密码哈希
  - new_passwd_hash：新密码哈希

  HTTP响应如下：
  - 修改成功：状态码：200；信息："Password Changed"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 账户未找到或密码错误：状态码：403；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）
- **void handler_get_carinfo(const httplib::Request &req, httplib::Response &res) const**：处理获取车辆信息请求。HTTP请求方法为POST，请求参数如下：
  - username：用户名
  - passwd_hash：密码哈希
  - car_id：车牌号（可为空）
  - car_owner：车主（可为空）
  - car_color：车辆颜色（可为空）

– car_type：车辆类型（可为空）

HTTP响应如下：

– 获取成功：状态码：200；信息：包含车辆信息的json格式字符串（application/json类型）

– 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）

– 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）

– 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_get_carimg(const httplib::Request &req, httplib::Response &res) const**：处理获取车辆图片请求。HTTP请求方法为POST，请求参数如下：

  – username：用户名

  – passwd_hash：密码哈希

  – car_img_path：车辆图片路径

  HTTP响应如下：

  – 获取成功：状态码：200；信息：车辆图片（image/jpeg或image/png类型）

  – 未找到文件：状态码：404；信息："File Not Found"（text/plain类型）

  – 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）

  – 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）

  – 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_add_car(const httplib::Request &req, httplib::Response &res)**：处理添加车辆请求。HTTP请求方法为POST，请求参数如下：

  – username：用户名

  – passwd_hash：密码哈希

  – car_id：车牌号

  – car_type：车辆类型

  – car_owner：车主

  – car_color：车辆颜色

  – car_year：车辆年份

  – car_img_path：车辆图片路径

  – car_img：车辆图片二进制数据

  – car_img_type：车辆图片格式（jpg、png等）

  HTTP响应如下：

  – 添加成功：状态码：200；信息："Car Added"（text/plain类型）

  – 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）

  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_remove_car(const httplib::Request &req, httplib::Response &res)**：处理删除车辆请求。HTTP请求方法为POST，请求参数如下：
  - username：用户名
  - passwd_hash：密码哈希
  - car_id：车牌号

  HTTP响应如下：
  - 删除成功：状态码：200；信息："Car Removed"（text/plain类型）
  - 未找到车辆：状态码：404；信息："Car Not Found"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_update_car(const httplib::Request &req, httplib::Response &res)**：处理更新车辆请求。HTTP请求方法为POST，请求参数如下：
  - username：用户名
  - passwd_hash：密码哈希
  - original_car_id：原车牌号
  - new_car_id：新车牌号
  - new_car_owner：新车主
  - new_car_color：新车辆颜色
  - new_car_year：新车辆年份
  - new_car_img：新车辆图片二进制数据
  - new_car_img_type：新车辆图片格式（jpg、png等）

  HTTP响应如下：
  - 更新成功：状态码：200；信息："Car Updated"（text/plain类型）
  - 未找到车辆：状态码：404；信息："Car Not Found"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_get_accountinfo(const httplib::Request &req, httplib::Response &res) const**：处理获取账户信息请求。HTTP请求方法为POST，请求参数如下：

  - username：用户名
  - passwd_hash：密码哈希
  - target_username：目标用户名

  HTTP响应如下：

  - 获取成功：状态码：200；信息：包含账户信息的json格式字符串（application/json类型）
  - 请求查询的账户不存在：状态码：404；信息："Account Not Found"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **handler_get_all_account(const httplib::Request &req, httplib::Response &res) const**：处理获取所有账户信息请求。HTTP请求方法为POST，请求参数如下：

  - username：用户名
  - passwd_hash：密码哈希

  HTTP响应如下：

  - 获取成功：状态码：200；信息：包含所有账户信息的json格式字符串（application/json类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_add_account(const httplib::Request &req, httplib::Response &res)**：处理添加账户请求。HTTP请求方法为POST，请求参数如下：

  - username：用户名
  - passwd_hash：密码哈希
  - target_username：目标用户名
  - target_passwd_hash：目标密码哈希
  - target_account_type：目标账户类型

  HTTP响应如下：

  - 添加成功：状态码：200；信息："Account Added"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_remove_account(const httplib::Request &req, httplib::Response &res)**：处理删除账户请求。HTTP请求方法为POST，请求参数如下：

  - username：用户名
  - passwd_hash：密码哈希
  - target_username：目标用户名

  HTTP响应如下：

  - 删除成功：状态码：200；信息："Account Removed"（text/plain类型）
  - 未找到账户：状态码：404；信息："Account Not Found"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

- **void handler_update_account(const httplib::Request &req, httplib::Response &res)**：处理更新账户请求。HTTP请求方法为POST，请求参数如下：

  - username：用户名
  - passwd_hash：密码哈希
  - target_username：目标用户名
  - new_username：新用户名
  - new_passwd_hash：新密码哈希
  - new_account_type：新账户类型

  HTTP响应如下：

  - 更新成功：状态码：200；信息："Account Updated"（text/plain类型）
  - 未找到账户：状态码：404；信息："Account Not Found"（text/plain类型）
  - 不合法请求：状态码：400；信息："Bad Request"（text/plain类型）
  - 未找到账户或密码错误：状态码：401；信息："Forbidden"（text/plain类型）
  - 服务器内部错误：状态码：500；信息："Internal Server Error"（text/plain类型）

4.1.2.8 日志类：MyLogger

日志类MyLogger用于记录系统运行日志。在CarPool、AccountPool中的查询、更改、加载、保存等操作中和ServerHttpHandler的处理函数中都有日志记录。
该类依赖于第三方开源库spdlog，github地址：`https://github.com/gabime/spdlog`
MyLogger类为纯静态类，包含以下成员函数：

- **static void registerLogger(const std::string &name, const uint8_t level, const uint8_t type, const std::string &path)**：对spdlog提供的api进行包装，注册日志记录器。注册一个日志记录器，指定日志级别、日志类型和日志文件路径（若将日志保存到文件中）。
- **static void log(const std::string &name, const uint8_t level, const std::string &msg)**：对spdlog提供的api进行包装，记录日志。记录一条日志，指定日志记录器、日志级别和日志内容。

4.1.2.9 json处理

使用第三方库nlohmann/json处理json数据，github地址：`https://github.com/nlohmann/json`。

4.1.3 工作流程

1. 注册日志记录器。
2. 读取配置文件，初始化车辆池和账户池，整理冗余文件。
3. 构建依赖于上一步车辆池和账户池的HTTP服务器，配置API接口URL。
4. 启动HTTP服务器，监听端口，处理客户端请求。

4.2 客户端实现

使用C++编写客户端程序，实现对服务器的连接、数据的发送与接收。使用Qt6进行界面设计，实现了数据的实时显示与控制。

4.2.1 主要类的简介

客户端程序主要包含以下几个类：

- **Car**：车辆类，包含车辆的基本信息。
- **Account**：账户类，包含账户的基本信息。
- **BasicPool**：池基类，为纯虚类，定义了一个池需要的基本接口。
- **CarPool**：车辆池类，继承自BasePool，实现了车辆信息的增删改查操作。
- **AccountPool**：账户池类，继承自BasePool，实现了账户信息的增删改查操作。
- **Hash**：哈希类，实现了加盐的sha256哈希算法。
- **ClientHttpHandler**：HTTP请求发送类，将请求发送到服务器并接收响应。
- **QWidgetWithSubWindows**：带有子窗口的QWidget类，继承于QWidget，维护一个子窗口的集合，实现了子窗口在父窗口关闭时自动关闭。
- **...Window**：各种窗口类，继承自QWidgetWithSubWindows或QWidget，实现了各种界面的设计与逻辑。
- **MyLogger**：日志类，实现了日志的记录。

4.2.2 主要功能实现

车辆类Car、车辆池类CarPool、账户类Account、账户池类AccountPool、哈希类Hash、日志类MyLogger与服务端实现相同。详见4.1.2节。

4.2.2.1 带有子窗口的界面类：QWidgetWithSubWindows

QWidgetWithSubWindows类继承自QWidget，维护了一个set<*QWidgetWithSubWindows >，用来存储子窗口的指针，并实现了添加、移除子窗口的方法。同时重写了closeEvent方法，使得在父窗口关闭时，子窗口也会自动关闭。

QWidgetWithSubWindows在QWidget之外，新增了如下成员变量：

- **subWindows**：存储子窗口的指针的set（保护成员）。
- **parent**：父窗口的指针（保护成员）。该“父窗口”与Qt中的父窗口概念不同。
- **closeFromParent**：是否由父窗口关闭的标志（保护成员）。一般地，当窗口关闭时，需要从父窗口的subWindows中移除自己。该动作在重写的closeEvent方法中实现。但是，在某种情况下，移除操作会引起段错误。为避免这种情况，在closeEvent中，如果closeFromParent为真，则不执行移除操作。

QWidgetWithSubWindows新增了如下成员函数：

- **QWidgetWithSubWindows(QWidget *parent, QWidgetWithSubWindows *parentWindow = nullptr, ...)**：构造函数。
- **~QWidgetWithSubWindows()**：析构函数。
- **void addSubWindow(QWidgetWithSubWindows *window)**：添加子窗口。
- **void removeSubWindow(QWidgetWithSubWindows *window)**：移除子窗口。
- **void (QWidgetWithSubWindows *window)**：关闭子窗口。
- **void closeAllSubWindows()**：关闭所有子窗口。
- **void removeFromParent()**：从父窗口的subWindows中移除自己。
- **void setCloseFromParent(const bool closeFromParent)**：设置closeFromParent的值。
- **bool getCloseFromParent()**：获取closeFromParent的值。
- **virtual void closeEvent(QCloseEvent *event) override**：重写closeEvent方法，使得在父窗口关闭时，子窗口也会自动关闭。

4.2.2.2 各种窗口类：...Window

每个类对应一个窗口，继承自QWidgetWithSubWindows或QWidget，使用QLabel、QLineEdit、QPushButton等控件实现了界面的设计。按钮点击函数基本上与HTTP请求对应，这些类在此不再赘述。

4.2.2.3 HTTP请求发送类：ClientHttpHandler

ClientHttpHandler类实现了HTTP请求的发送与响应的接收。返回一般为一个HttpResult对象，包含一个int类型状态码、一个string类型状态信息和一个json类型数据（若需要）。

该类依赖于第三方开源库cpp-httplib，github地址：`https://github.com/yhirose/cpp-httplib`。ServerHttpHandler类包含以下成员函数：

- **static bool handler_verify_server(const std::string &ip, int port)**：验证服务器是否可连接。
- **static AccountVerifyResult handler_login(const std::string &ip, int port, Account &acc)**：进行登录操作，返回登录结果。其中AccountVerifyResult是一个枚举类型，有如下取值：
  - SUCCESS = 0：登录成功。
  - ACCOUNT_NOT_FOUND_OR_WRONG_PASSWORD = 1：账户不存在或密码错误。
  - NETWORK_ERROR = 2：网络错误。
  - INVALID_RESPONSE = 3：无效的响应。
  - BAD_REQUEST = 4：请求错误。
- **static HttpResult handler_get_carinfo(const std::string &ip, int port, const Account &acc, const std::string &carid, const std::string &carowner, const std::string &carcolor, const std::string &cartype)**：获取车辆信息。其中参数含义如下（下同）：
  - ip：服务器IP。
  - port：服务器端口。
  - acc：账户信息，用于验证是否有操作权限。
  - carid：车辆ID（可选）。
  - carowner：车主（可选）。
  - carcolor：车辆颜色（可选）。
  - cartype：车辆类型（可选）。
- **static HttpResult handler_get_carimg(const std::string &ip, int port, const Account &acc, std::string car_img_path)**：获取车辆图片。
- **static HttpResult handler_add_car(const std::string &ip, const int port, const Account &acc, const std::string &car_id, const std::string &car_owner, const std::string&car_type, const std::string &car_color, const int car_year, const std::string &car_img, const std::string &car_img_type)**：添加车辆。
- **static HttpResult handler_remove_car(const std::string &ip, const int port, const Account &acc, const std::string &car_id)**：删除车辆。
- **static HttpResult handler_update_car(const std::string &ip, const int port, const Account &acc, const std::string &original_car_id, const std::string &new_car_id, const std::string &new_car_owner, const std::string &new_car_type, const std::string &new_car_color, const int new_car_year, const std::string &new_car_img, const std::string &new_car_img_type)**：修改车辆信息。
- **static HttpResult handler_search_account(const std::string &ip, const int port, const Account &acc, const std::string &search_username)**：查找账户。

- **static HttpResult handler_get_all_accounts(const std::string &ip, const int port, const Account &acc)**：获取所有账户。
- **static HttpResult handler_add_account(const std::string &ip, const int port, const Account &acc, const std::string &new_username, const std::string &new_passwd_hash, const Account::AccountType new_account_type)**：添加账户。
- **static HttpResult handler_remove_account(const std::string &ip, const int port, const Account &acc, const std::string &target_username)**：删除账户。
- **static HttpResult handler_update_account(const std::string &ip, const int port, const Account &acc, const std::string &target_username, const std::string &new_username, const std::string &new_passwd_hash, const Account::AccountType new_account_type)**：修改账户信息。
- **static HttpResult handler_change_password(const std::string &ip, const int port, const Account &acc, const std::string &new_passwd_hash)**：修改密码。

4.2.3 工作流程

1. 启动客户端程序，输入服务器IP和端口号，连接服务器。
2. 输入用户名和密码，登录。
3. 进入主界面，根据用户权限显示车辆相关和账户相关的功能按钮。
4. 点击按钮，填写信息，发送HTTP请求。
5. 接收服务器响应，显示结果。

5 系统调试与测试

5.1 系统调试

- 在编写程序的过程中，遵循“编写一个，测试一个”的原则，在每个模块编写完成后对其进行测试，尽早地发现编写过程中出现的错误或遗漏之处（如一开始在carpool中以car_owner为键创建了std::map，忘记了car_owner可能重复）。
- 在编写类方法的过程中广泛使用了状态码和日志系统，方便在运行结果不符合预期时快速定位问题所在。
- 使用第三方库时仔细阅读文档，并事先编写简单的使用样例，确保正确使用。

5.2 系统测试

5.2.1 单元测试

使用GoogleTest框架为AccountPool、Base64、CarPool、Hash等后端类编写了单元测试。测试点基本覆盖了这些类的主要函数。在每次commit前，均会进行单元测试，确保代码的正确性。详细的测试代码见附录A。

5.2.2 API接口测试

使用Postman对服务端的API接口进行了测试，测试点包括了ServerHttpHandler类中提供的所有接口。测试结果显示，所有接口均能正常工作，符合预期。

5.2.3 跨平台测试

将代码在安装有Qt6.7.1的Windows 11和Ubuntu 20.04 LTS上编译运行，均能正常工作。Windows下测试了MSVC 2022与MinGW 13.2.0，Ubuntu下测试了GCC 11.4.0，编译产物均可正常运行。

6 使用方法

6.1 软件的获取

项目代码已经上传至github，地址为`https://github.com/FHYQ-Dong/Carinfo-Manager`。可以通过以下两种方式获取软件：

- **使用编译完成的二进制文件：**在项目的release页面下载与系统对应的压缩包，解压后即可使用。（目前仅提供Windows和Linux平台的二进制文件）
- **从源码构建：**
  1. 环境要求：Qt6.7.1及以上版本，CMake3.6及以上版本，gcc/g++或者MSVC编译器。
  2. 克隆项目：使用git克隆项目代码到本地。
  3. 编辑配置文件：编辑项目根目录下CMakeLists.txt文件，将第12行中的路径改为Qt的安装路径。
  4. 生成：使用CMake构建项目。

6.2 服务端使用方法

服务端为纯命令行程序，启动方法如下：

```
Carinfo-Manager-Server [path_to_config_file]
```

其中`[path_to_config_file]`为配置文件的路径。配置文件为json格式，包含以下字段：

- **ip：**服务器IP地址。
- **port：**服务器端口。
- **dataDir：**数据存储目录。

一个配置文件示例如下：

```
{
    "ip": "127.0.0.1",
    "port": 12345,
    "dataDir": "data/"
}
```

注意：

- “dataDir”字段需要以“/”结尾。
- 数据存储目录结构如下，需要确保结构正确：

```
dataDir/
|-- account.json
|-- car.json
`-- img/
```

- 其中，account.json和car.json为json格式文件，分别存储账户信息和车辆信息。img目录存储车辆图片。
- accounts.json的格式如下，其中记录了一个用户名为admin1，密码为123456的管理员账户和一个用户名为user1，密码为123456的普通用户账户：

```
{
    "admin1": {
        "account_type": 1,
        "passwd_hash": "4b7897eed3bebaa5cfe986afb4075ba5
                        d9f42959bc9b89cf6af6c3af403ef14e",
        "username": "admin1"
    },
    "user1": {
        "account_type": 2,
        "passwd_hash": "4b7897eed3bebaa5cfe986afb4075ba5
                        d9f42959bc9b89cf6af6c3af403ef14e",
        "username": "user1"
    }
}
```

- car.json的格式如下，其中记录了一个车辆信息：

```
{
    "京A12345": {
        "color": "red",
        "id": "京A12345",
        "img_path": "data/img/5LqsQTEyMzQ1.jpg",
        "owner": "爱丽丝",
        "type": "特斯拉",
        "year": 2024
    }
}
```

- 车辆图片存储在img目录下，文件名为车辆ID的Base64编码，每次启动后会根据car.json文件中记录的信息删除冗余的文件。
- 以上所有示例均在github仓库中的对应目录下提供。

服务端程序启动后，会将debug及以上级别的日志保存在log/carinfo-manager-server.log下，同时将info及以上级别的日志输出到标准输出中。一般地，看到如下输出即代表服务端启动成功：

```
[2024-08-31 19:56:16.347] [carinfo-manager-logger] [info] Using config:
- dataDir: data/
- ip: 127.0.0.1
- port: 12345
[2024-08-31 19:56:16.349] [carinfo-manager-logger] [info] Server
   started
```

6.3 客户端使用方法

客户端为GUI程序，双击可执行文件即可使用。一般地，需要经过如下几个步骤：

1. **选择服务器**：输入服务端监听的IP地址及端口，如图3(a)。
2. **登录**：输入用户名及密码，如图3(b)。
3. **操作**：点击页面显示的按钮进行相关操作，如图3(c)（管理员界面）和3(d)（用户界面）。

7 编程体会

7.1 面向对象的编程思想

在项目中运用面向对象的编程思想能够极大程度地解耦各个模块。与上学期做黄永锋老师的附加作业时的感受完全不同，由于C语言更多时面向过程的编程思想，导致多人协作的分工、代码功能的回忆等具有一定难度。但在建构起清晰的类层次、明确各模块的具体功能后，项目的编写就变得极为方便快捷。一个明显的感受便是一段时间后继续编写未完成的功能时能够快速地明确思路。

7.2 善用前人的轮子

本项目在编写过程中使用了cpp-httplib等第三方开源库。在不了解或不需要了解具体技术细节（如HTTP协议的具体内容等）的情况下，从头编写库文件在效率上显然不如使用前人已经开发完善的开源库。但需注意开源库的许可协议限制。



(a) 选择服务器



(b) 登录



(c) 管理员界面



(d) 用户界面

图 3: 客户端使用方法

A 单元测试代码

A.1 AccountPool测试

```
/**
 * @file test/test_AccountPool.cc
 * @brief Test cases for class AccountPool in carinfo-manager project, a car information management system, which is a part of the course project for data structure and algorithm analysis in Tsinghua University, spring 2021. The project is maintained by donghy23@mails.tsinghua.edu.cn
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/hash.hpp"
#include "gtest/gtest.h"
#include <fstream>

TEST(Test_AccountPool, test_AccountPool_default_constructor) {
    AccountPool accountpool;
    EXPECT_EQ(accountpool.size(), 0);
    EXPECT_TRUE(accountpool.empty());
}

TEST(Test_AccountPool, test_AccountPool_parameterized_constructor) {
    Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
       hash3("password3", "salt3");
    Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
       );
    Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
    Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
    Account accounts[] = {acc1, acc2, acc3};
    AccountPool accountpool(accounts, accounts + 3);
    EXPECT_EQ(accountpool.size(), 3);
    EXPECT_FALSE(accountpool.empty());
}

TEST(Test_AccountPool,
   test_AccountPool_parameterized_constructor_vector) {
```

```
31      Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
32      Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
33      Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
34      Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
35      std::vector<Account> accounts = {acc1, acc2, acc3};
36      AccountPool accountpool(accounts);
37      EXPECT_EQ(accountpool.size(), 3);
38      EXPECT_FALSE(accountpool.empty());
39  }
40
41  TEST(Test_AccountPool, test_AccountPool_copy_constructor) {
42      Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
43      Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
44      Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
45      Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
46      Account accounts[] = {acc1, acc2, acc3};
47      AccountPool accountpool1(accounts, accounts + 3);
48      AccountPool accountpool2(accountpool1);
49      EXPECT_EQ(accountpool2.size(), 3);
50      EXPECT_FALSE(accountpool2.empty());
51  }
52
53  TEST(Test_AccountPool, test_AccountPool_addAccount) {
54      AccountPool accountpool;
55      Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
56      Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
57      Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
58      Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
59      EXPECT_EQ(accountpool.addAccount(acc1), 0);
60      EXPECT_EQ(accountpool.addAccount(acc2), 0);
61      EXPECT_EQ(accountpool.addAccount(acc3), 0);
62      EXPECT_EQ(accountpool.size(), 3);
63      EXPECT_FALSE(accountpool.empty());
64      EXPECT_EQ(accountpool.addAccount(acc1), 0x10);
65      EXPECT_EQ(accountpool.addAccount(acc2), 0x10);
```

```
66      EXPECT_EQ(accountpool.addAccount(acc3), 0x10);
67  }
68
69  TEST(Test_AccountPool, test_AccountPool_removeAccount) {
70      AccountPool accountpool;
71      Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
72      Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
73      Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
74      Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
75      EXPECT_EQ(accountpool.addAccount(acc1), 0);
76      EXPECT_EQ(accountpool.addAccount(acc2), 0);
77      EXPECT_EQ(accountpool.addAccount(acc3), 0);
78      EXPECT_EQ(accountpool.size(), 3);
79      EXPECT_FALSE(accountpool.empty());
80      EXPECT_EQ(accountpool.removeAccount(acc1), 0);
81      EXPECT_EQ(accountpool.removeAccount(acc2), 0);
82      EXPECT_EQ(accountpool.removeAccount(acc3), 0);
83      EXPECT_EQ(accountpool.size(), 0);
84      EXPECT_TRUE(accountpool.empty());
85      EXPECT_EQ(accountpool.removeAccount(acc1), 0x20);
86      EXPECT_EQ(accountpool.removeAccount(acc2), 0x20);
87      EXPECT_EQ(accountpool.removeAccount(acc3), 0x20);
88  }
89
90  TEST(Test_AccountPool, test_AccountPool_updateAccount) {
91      AccountPool accountpool;
92      Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
93      Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
94      Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
95      Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
96      EXPECT_EQ(accountpool.addAccount(acc1), 0);
97      EXPECT_EQ(accountpool.addAccount(acc2), 0);
98      EXPECT_EQ(accountpool.addAccount(acc3), 0);
99      EXPECT_EQ(accountpool.size(), 3);
100     EXPECT_FALSE(accountpool.empty());
101     Hash hash4("password4", "salt4"), hash5("password5", "salt5"),
            hash6("password6", "salt6");
```

```
102     Account acc4("admin2", hash4.getHash(), Account::AccountType::ADMIN
            );
103     Account acc5("user3", hash5.getHash(), Account::AccountType::USER);
104     Account acc6("user4", hash6.getHash(), Account::AccountType::USER);
105     EXPECT_EQ(accountpool.updateAccount(acc1, acc4), 0);
106     EXPECT_EQ(accountpool.updateAccount(acc2, acc5), 0);
107     EXPECT_EQ(accountpool.updateAccount(acc3, acc6), 0);
108     EXPECT_EQ(accountpool.size(), 3);
109     EXPECT_FALSE(accountpool.empty());
110     EXPECT_EQ(accountpool.updateAccount(acc1, acc4), 0x30);
111     EXPECT_EQ(accountpool.updateAccount(acc2, acc5), 0x30);
112     EXPECT_EQ(accountpool.updateAccount(acc3, acc6), 0x30);
113 }
114
115 TEST(Test_AccountPool, test_AccountPool_getAccount) {
116     AccountPool accountpool;
117     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
118     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
119     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
120     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
121     EXPECT_EQ(accountpool.addAccount(acc1), 0);
122     EXPECT_EQ(accountpool.addAccount(acc2), 0);
123     EXPECT_EQ(accountpool.addAccount(acc3), 0);
124     EXPECT_EQ(accountpool.size(), 3);
125     EXPECT_FALSE(accountpool.empty());
126     EXPECT_EQ(accountpool.getAccount("admin1"), acc1);
127     EXPECT_EQ(accountpool.getAccount("user1"), acc2);
128     EXPECT_EQ(accountpool.getAccount("user2"), acc3);
129     EXPECT_EQ(accountpool.getAccount("admin2"), Account::NULL_ACCOUNT);
130     EXPECT_EQ(accountpool.getAccount("user3"), Account::NULL_ACCOUNT);
131     EXPECT_EQ(accountpool.getAccount("user4"), Account::NULL_ACCOUNT);
132 }
133
134 TEST(Test_AccountPool, test_AccountPool_verifyAccount) {
135     AccountPool accountpool;
136     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
137     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
```

```
138     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
139     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
140     EXPECT_EQ(accountpool.addAccount(acc1), 0);
141     EXPECT_EQ(accountpool.addAccount(acc2), 0);
142     EXPECT_EQ(accountpool.addAccount(acc3), 0);
143     EXPECT_EQ(accountpool.size(), 3);
144     EXPECT_FALSE(accountpool.empty());
145     EXPECT_EQ(accountpool.verifyAccount("admin1", hash1.getHash()),
            AccountPool::AccountVerifyResult::SUCCESS);
146     EXPECT_EQ(accountpool.verifyAccount("user1", hash2.getHash()),
            AccountPool::AccountVerifyResult::SUCCESS);
147     EXPECT_EQ(accountpool.verifyAccount("user2", hash3.getHash()),
            AccountPool::AccountVerifyResult::SUCCESS);
148     EXPECT_EQ(accountpool.verifyAccount("admin1", hash2.getHash()),
            AccountPool::AccountVerifyResult::WRONG_PASSWORD);
149     EXPECT_EQ(accountpool.verifyAccount("user1", hash3.getHash()),
            AccountPool::AccountVerifyResult::WRONG_PASSWORD);
150     EXPECT_EQ(accountpool.verifyAccount("user2", hash1.getHash()),
            AccountPool::AccountVerifyResult::WRONG_PASSWORD);
151     EXPECT_EQ(accountpool.verifyAccount("admin2", hash1.getHash()),
            AccountPool::AccountVerifyResult::ACCOUNT_NOT_FOUND);
152     EXPECT_EQ(accountpool.verifyAccount("user3", hash2.getHash()),
            AccountPool::AccountVerifyResult::ACCOUNT_NOT_FOUND);
153     EXPECT_EQ(accountpool.verifyAccount("user4", hash3.getHash()),
            AccountPool::AccountVerifyResult::ACCOUNT_NOT_FOUND);
154 }
155
156 TEST(Test_AccountPool, test_AccountPool_save) {
157     AccountPool accountpool;
158     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
159     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
160     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
161     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
162     EXPECT_EQ(accountpool.addAccount(acc1), 0);
163     EXPECT_EQ(accountpool.addAccount(acc2), 0);
164     EXPECT_EQ(accountpool.addAccount(acc3), 0);
165     EXPECT_EQ(accountpool.size(), 3);
166     EXPECT_FALSE(accountpool.empty());
167     std::ofstream ofs("test_AccountPool_save.dat", std::ios::binary);
```

```
168     ASSERT_EQ(accountpool.save(ofs), 0);
169     ofs.close();
170 }
171 
172 TEST(Test_AccountPool, test_AccountPool_load) {
173     AccountPool accountpool;
174     std::ifstream ifs("test_AccountPool_save.dat", std::ios::binary);
175     ASSERT_EQ(accountpool.load(ifs), 0);
176     ifs.close();
177     EXPECT_EQ(accountpool.size(), 3);
178     EXPECT_FALSE(accountpool.empty());
179     EXPECT_EQ(accountpool.getAccount("admin1").getUsername(), "admin1")
        ;
180     EXPECT_EQ(accountpool.getAccount("user1").getUsername(), "user1");
181     EXPECT_EQ(accountpool.getAccount("user2").getUsername(), "user2");
182 }
183 
184 TEST(Test_AccountPool, test_AccountPool_operator_eq) {
185     AccountPool accountpool1;
186     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
        hash3("password3", "salt3");
187     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
        );
188     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
189     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
190     EXPECT_EQ(accountpool1.addAccount(acc1), 0);
191     EXPECT_EQ(accountpool1.addAccount(acc2), 0);
192     EXPECT_EQ(accountpool1.addAccount(acc3), 0);
193     EXPECT_EQ(accountpool1.size(), 3);
194     EXPECT_FALSE(accountpool1.empty());
195     AccountPool accountpool2;
196     accountpool2 = accountpool1;
197     EXPECT_EQ(accountpool2.size(), 3);
198     EXPECT_FALSE(accountpool2.empty());
199 }
200 
201 TEST(Test_AccountPool, test_AccountPool_operator_ne) {
202     AccountPool accountpool1;
203     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
        hash3("password3", "salt3");
```

```
204     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
205     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
206     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
207     EXPECT_EQ(accountpool1.addAccount(acc1), 0);
208     EXPECT_EQ(accountpool1.addAccount(acc2), 0);
209     EXPECT_EQ(accountpool1.addAccount(acc3), 0);
210     EXPECT_EQ(accountpool1.size(), 3);
211     EXPECT_FALSE(accountpool1.empty());
212     AccountPool accountpool2;
213     EXPECT_NE(accountpool1, accountpool2);
214     accountpool2 = accountpool1;
215     EXPECT_EQ(accountpool1, accountpool2);
216     accountpool2.removeAccount("admin1");
217     EXPECT_NE(accountpool1, accountpool2);
218     accountpool2.addAccount(acc1);
219     EXPECT_EQ(accountpool1, accountpool2);
220 }
221
222 TEST(Test_AccountPool, test_AccountPool_operator_assign) {
223     AccountPool accountpool1;
224     Hash hash1("password1", "salt1"), hash2("password2", "salt2"),
            hash3("password3", "salt3");
225     Account acc1("admin1", hash1.getHash(), Account::AccountType::ADMIN
            );
226     Account acc2("user1", hash2.getHash(), Account::AccountType::USER);
227     Account acc3("user2", hash3.getHash(), Account::AccountType::USER);
228     EXPECT_EQ(accountpool1.addAccount(acc1), 0);
229     EXPECT_EQ(accountpool1.addAccount(acc2), 0);
230     EXPECT_EQ(accountpool1.addAccount(acc3), 0);
231     EXPECT_EQ(accountpool1.size(), 3);
232     EXPECT_FALSE(accountpool1.empty());
233     AccountPool accountpool2;
234     accountpool2 = accountpool1;
235     EXPECT_EQ(accountpool2.size(), 3);
236     EXPECT_FALSE(accountpool2.empty());
237     accountpool2.removeAccount("admin1");
238     EXPECT_EQ(accountpool2.size(), 2);
239     EXPECT_FALSE(accountpool2.empty());
240     accountpool2 = accountpool1;
241     EXPECT_EQ(accountpool2.size(), 3);
```

```
242     EXPECT_FALSE(accountpool2.empty());
243 }
```

A.2 Base64测试

```
1  /**
2   * @file test/test_Base64.cc
3   * @brief Test for Base64 class in carinfo-manager project, including
        encode and decode functions, using Google Test framework.
4   * @author donghy23@mails.tsinghua.edu.cn
5   * @version 1.0
6   */
7
8  #include "gtest/gtest.h"
9  #include "carinfo-manager/base64.hpp"
10 #include <iostream>
11
12 TEST(Test_Base64, TEST_Base64_encode) {
13     const char* tmp = "ABCXABC";
14     unsigned char* src = new unsigned char[8];
15     for (size_t i = 0; i < 8; i++) src[i] = (unsigned char)(tmp[i]);
16     src[3] = '\0'; src[7] = '\0';
17     std::string res = Base64::encode(src, 7);
18     std::string ans = "QUJDAEFCQw==";
19     for (size_t i = 0; i < ans.length(); i++)
20         EXPECT_TRUE(res[i] == ans[i]);
21     EXPECT_EQ(ans.length(), res.length());
22 }
23
24 TEST(Test_Base64, TEST_Base64_decode) {
25     const char* src = "QUJDAEFCQw==";
26     size_t res_len = 0;
27     std::string res = Base64::decode(src, 12, res_len);
28     std::string ans = "ABC";
29     ans += '\0';
30     ans += "ABC";
31     for (size_t i = 0; i < ans.length(); i++)
32         EXPECT_TRUE(ans[i] == res[i]);
33     EXPECT_EQ(ans.length(), res_len);
34 }
```

A.3 CarPool测试

```cpp
/**
 * @file test/test_CarPool.cc
 * @brief Test for CarPool class in carinfo-manager project, including
    constructors, addCar, removeCar, updateCar, getCarbyId,
    getCarbyColor, getCarbyType, save, load, operator==, operator!=,
    operator=, using Google Test framework.
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "gtest/gtest.h"
#include "carinfo-manager/carpool.hpp"
#include <fstream>
#include <vector>

TEST(Test_CarPool, test_CarPool_default_constructor) {
    CarPool carpool;
    EXPECT_EQ(carpool.size(), 0);
    EXPECT_TRUE(carpool.empty());
}

TEST(Test_CarPool, test_CarPool_parameterized_constructor) {
    Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
    Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
    Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
       jpg");
    Car cars[] = {car1, car2, car3};
    CarPool carpool(cars, cars + 3);
    EXPECT_EQ(carpool.size(), 3);
    EXPECT_FALSE(carpool.empty());
}

TEST(Test_CarPool, test_CarPool_parameterized_constructor_vector) {
    Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
    Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
    Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
       jpg");
    std::vector<Car> cars = {car1, car2, car3};
    CarPool carpool(cars);
```

```
35      EXPECT_EQ(carpool.size(), 3);
36      EXPECT_FALSE(carpool.empty());
37  }
38
39  TEST(Test_CarPool, test_CarPool_copy_constructor) {
40      Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
41      Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
42      Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
            jpg");
43      Car cars[] = {car1, car2, car3};
44      CarPool carpool1(cars, cars + 3);
45      CarPool carpool2(carpool1);
46      EXPECT_EQ(carpool2.size(), 3);
47      EXPECT_FALSE(carpool2.empty());
48  }
49
50  TEST(Test_CarPool, test_CarPool_addCar) {
51      CarPool carpool;
52      Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
53      Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
54      Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
            jpg");
55      EXPECT_EQ(carpool.addCar(car1), 0);
56      EXPECT_EQ(carpool.addCar(car2), 0);
57      EXPECT_EQ(carpool.addCar(car3), 0);
58      EXPECT_EQ(carpool.size(), 3);
59      EXPECT_FALSE(carpool.empty());
60      EXPECT_EQ(carpool.addCar(car1), 0x70);
61      EXPECT_EQ(carpool.addCar(car2), 0x70);
62      EXPECT_EQ(carpool.addCar(car3), 0x70);
63  }
64
65  TEST(Test_CarPool, test_CarPool_removeCar) {
66      CarPool carpool;
67      Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
68      Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
69      Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
            jpg");
70      EXPECT_EQ(carpool.addCar(car1), 0);
71      EXPECT_EQ(carpool.addCar(car2), 0);
72      EXPECT_EQ(carpool.addCar(car3), 0);
```

```
    EXPECT_EQ(carpool.size(), 3);
    EXPECT_FALSE(carpool.empty());
    EXPECT_EQ(carpool.removeCar(car1), 0);
    EXPECT_EQ(carpool.removeCar(car2), 0);
    EXPECT_EQ(carpool.removeCar(car3), 0);
    EXPECT_EQ(carpool.size(), 0);
    EXPECT_TRUE(carpool.empty());
    EXPECT_EQ(carpool.removeCar(car1), 0x80);
    EXPECT_EQ(carpool.removeCar(car2), 0x80);
    EXPECT_EQ(carpool.removeCar(car3), 0x80);
}

TEST(Test_CarPool, test_CarPool_updateCar) {
    CarPool carpool;
    Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
    Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
    Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
    EXPECT_EQ(carpool.addCar(car1), 0);
    EXPECT_EQ(carpool.addCar(car2), 0);
    EXPECT_EQ(carpool.addCar(car3), 0);
    EXPECT_EQ(carpool.size(), 3);
    EXPECT_FALSE(carpool.empty());

    Car car4("京D13579", "SUV", "Dick", "green", 2021, "img4.jpg");
    Car car5("京E24680", "Sedan", "Eva", "orange", 2020, "img5.jpg");
    Car car6("京F35791", "Hatchback", "Frank", "purple", 2019, "img6.jpg
        ");
    EXPECT_EQ(carpool.updateCar(car1, car4), 0);
    EXPECT_EQ(carpool.updateCar(car2, car5), 0);
    EXPECT_EQ(carpool.updateCar(car3, car6), 0);
    EXPECT_EQ(carpool.size(), 3);
    EXPECT_FALSE(carpool.empty());
    EXPECT_EQ(carpool.updateCar(car1, car4), 0x90);
    EXPECT_EQ(carpool.updateCar(car2, car5), 0x90);
    EXPECT_EQ(carpool.updateCar(car3, car6), 0x90);

    EXPECT_EQ(carpool.updateCar("京D13579", car1), 0);
    EXPECT_EQ(carpool.updateCar("京E24680", car2), 0);
    EXPECT_EQ(carpool.updateCar("京F35791", car3), 0);
    EXPECT_EQ(carpool.size(), 3);
```

```
112     EXPECT_FALSE(carpool.empty());
113     EXPECT_EQ(carpool.updateCar("京D13579", car1), 0x90);
114     EXPECT_EQ(carpool.updateCar("京E24680", car2), 0x90);
115     EXPECT_EQ(carpool.updateCar("京F35791", car3), 0x90);
116 }
117
118 TEST(Test_CarPool, test_CarPool_getCarbyId) {
119     CarPool carpool;
120     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
121     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
122     Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
123     EXPECT_EQ(carpool.addCar(car1), 0);
124     EXPECT_EQ(carpool.addCar(car2), 0);
125     EXPECT_EQ(carpool.addCar(car3), 0);
126     EXPECT_TRUE(carpool.getCarbyId("京A12345") == CarPool((Car*)&car1, (
        Car*)&car1 + 1));
127     EXPECT_TRUE(carpool.getCarbyId("京B67890") == CarPool((Car*)&car2, (
        Car*)&car2 + 1));
128     EXPECT_TRUE(carpool.getCarbyId("京C24680") == CarPool((Car*)&car3, (
        Car*)&car3 + 1));
129 }
130
131 TEST(Test_CarPool, test_CarPool_getCarbyColor) {
132     CarPool carpool;
133     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
134     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
135     Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
136     EXPECT_EQ(carpool.addCar(car1), 0);
137     EXPECT_EQ(carpool.addCar(car2), 0);
138     EXPECT_EQ(carpool.addCar(car3), 0);
139     CarPool redCars = carpool.getCarbyColor("red"), redCarsExpected((
        Car*)&car1, (Car*)&car1 + 1);
140     CarPool greenCars = carpool.getCarbyColor("yellow"),
        greenCarsExpected((Car*)&car2, (Car*)&car2 + 1);
141     CarPool blueCars = carpool.getCarbyColor("blue"), blueCarsExpected
        ((Car*)&car3, (Car*)&car3 + 1);
142     EXPECT_TRUE(redCars == redCarsExpected);
143     EXPECT_TRUE(greenCars == greenCarsExpected);
144     EXPECT_TRUE(blueCars == blueCarsExpected);
```

```
}

TEST(Test_CarPool, test_CarPool_getCarbyType) {
    CarPool carpool;
    Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
    Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
    Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
    EXPECT_EQ(carpool.addCar(car1), 0);
    EXPECT_EQ(carpool.addCar(car2), 0);
    EXPECT_EQ(carpool.addCar(car3), 0);
    CarPool suvs = carpool.getCarbyType("SUV"), suvsExpected((Car*)&
        car1, (Car*)&car1 + 1);
    CarPool sedans = carpool.getCarbyType("Sedan"), sedansExpected((Car
        *)&car2, (Car*)&car2 + 1);
    CarPool hatchbacks = carpool.getCarbyType("Hatchback"),
        hatchbacksExpected((Car*)&car3, (Car*)&car3 + 1);
    EXPECT_TRUE(suvs == suvsExpected);
    EXPECT_TRUE(sedans == sedansExpected);
    EXPECT_TRUE(hatchbacks == hatchbacksExpected);
}

TEST(Test_CarPool, test_CarPool_getCarbyOwner) {
    CarPool carpool;
    Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
    Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
    Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
    EXPECT_EQ(carpool.addCar(car1), 0);
    EXPECT_EQ(carpool.addCar(car2), 0);
    EXPECT_EQ(carpool.addCar(car3), 0);
    CarPool aliceCars = carpool.getCarbyOwner("Alice"),
        aliceCarsExpected((Car*)&car1, (Car*)&car1 + 1);
    CarPool bobCars = carpool.getCarbyOwner("Bob"), bobCarsExpected((
        Car*)&car2, (Car*)&car2 + 1);
    CarPool catherineCars = carpool.getCarbyOwner("Catherine"),
        catherineCarsExpected((Car*)&car3, (Car*)&car3 + 1);
    EXPECT_TRUE(aliceCars == aliceCarsExpected);
    EXPECT_TRUE(bobCars == bobCarsExpected);
    EXPECT_TRUE(catherineCars == catherineCarsExpected);
}
```

```
178
179 TEST(Test_CarPool, test_CarPool_getCar) {
180     CarPool carpool;
181     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
182     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
183     Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
184     EXPECT_EQ(carpool.addCar(car1), 0);
185     EXPECT_EQ(carpool.addCar(car2), 0);
186     EXPECT_EQ(carpool.addCar(car3), 0);
187     CarPool redSUVs = carpool.getCar("", "red", "", "SUV"),
        redSUVsExpected((Car*)&car1, (Car*)&car1 + 1);
188     CarPool yellowSUVs = carpool.getCar("", "yellow", "", "SUV"),
        yellowSUVsExpected;
189     CarPool blueSUVs = carpool.getCar("", "blue", "", "SUV"),
        blueSUVsExpected;
190     EXPECT_TRUE(redSUVs == redSUVsExpected);
191     EXPECT_TRUE(yellowSUVs == yellowSUVsExpected);
192     EXPECT_TRUE(blueSUVs == blueSUVsExpected);
193 }
194
195 TEST(Test_CarPool, test_CarPool_save) {
196     CarPool carpool;
197     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
198     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
199     Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
        jpg");
200
201     EXPECT_EQ(carpool.addCar(car1), 0);
202     EXPECT_EQ(carpool.addCar(car2), 0);
203     EXPECT_EQ(carpool.addCar(car3), 0);
204     std::ofstream ofs("test_CarPool_save.json");
205     std::stringstream ss;
206     ASSERT_EQ(carpool.save(ofs), 0);
207 }
208
209 TEST(Test_CarPool, test_CarPool_load) {
210     CarPool carpool;
211     std::ifstream ifs("test_CarPool_save.json");
212     ASSERT_EQ(carpool.load(ifs), 0);
213     ifs.close();
```

```
214     EXPECT_EQ(carpool.size(), 3);
215     EXPECT_FALSE(carpool.empty());
216     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
217     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
218     Car car3("京C24680", "Hatchback", "Catherine", "blue", 2019, "img3.
            jpg");
219     EXPECT_TRUE(carpool.getCarbyId("京A12345") == CarPool((Car*)&car1, (
            Car*)&car1 + 1));
220     EXPECT_TRUE(carpool.getCarbyId("京B67890") == CarPool((Car*)&car2, (
            Car*)&car2 + 1));
221     EXPECT_TRUE(carpool.getCarbyId("京C24680") == CarPool((Car*)&car3, (
            Car*)&car3 + 1));
222 }
223
224 TEST(Test_CarPool, test_CarPool_operator_eq) {
225     CarPool carpool1, carpool2;
226     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
227     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
228     carpool1.addCar(car1); carpool1.addCar(car2);
229     EXPECT_TRUE(carpool1 == carpool1);
230     EXPECT_FALSE(carpool1 == carpool2);
231 }
232
233 TEST(Test_CarPool, test_CarPool_operator_ne) {
234     CarPool carpool1, carpool2;
235     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
236     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
237     carpool1.addCar(car1); carpool1.addCar(car2);
238     EXPECT_FALSE(carpool1 != carpool1);
239     EXPECT_TRUE(carpool1 != carpool2);
240 }
241
242 TEST(Test_CarPool, test_CarPool_operator_assign) {
243     CarPool carpool1, carpool2;
244     Car car1("京A12345", "SUV", "Alice", "red", 2021, "img1.jpg");
245     Car car2("京B67890", "Sedan", "Bob", "yellow", 2020, "img2.jpg");
246     carpool1.addCar(car1); carpool1.addCar(car2);
247     carpool2 = carpool1;
248     EXPECT_TRUE(carpool1 == carpool2);
249 }
```

A.4 Hash测试

```cpp
/**
 * @file test/test_Hash.cc
 * @brief Test for Hash class in carinfo-manager project, including getHash function, using Google Test framework.
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/hash.hpp"
#include "gtest/gtest.h"

TEST(Test_Hash, test_Hash_original) {
    Hash hash("test1", "");
    EXPECT_TRUE(hash.getHash() == "1b4f0e9851971998e732078544c96b36c3d01cedf7caa332359d6f1d83567014");
    hash.setData("test2");
    EXPECT_TRUE(hash.getHash() == "60303ae22b998861bce3b28f33eec1be758a213c86c93c076dbe9f558c11c752");
    hash.setData("test3");
    EXPECT_TRUE(hash.getHash() == "fd61a03af4f77d870fc21e05e7e80678095c92d808cfb3b5c279ee04c74aca13");
}

TEST(Test_Hash, test_Hash_salt) {
    Hash hash("test1", "salt1");
    EXPECT_TRUE(hash.getHash() == "a1cbb225c3c0cd1cdf283c22ed489639acfeff291f73072a1cdbab11697edcd8");
    hash.setData("test2"); hash.setSalt("salt2");
    EXPECT_TRUE(hash.getHash() == "cdec2598121529d63c0aafced05a7a86e099f794b8889c9528df0cd171e2fbb5");
    hash.setData("test3"); hash.setSalt("salt3");
    EXPECT_TRUE(hash.getHash() == "c7d9e34aba1b7aadd65410ac6f5e4048547476be69953ccf31060eb88bea1b3f");
}
```

B 源代码

本项目已经上传至**github**，链接：https://github.com/FHYQ-Dong/Carinfo-Manager。为获得最好的阅读体验，推荐在**github**上查看或从**github**上克隆项目到本地。

项目的目录结构如下：

```
Carinfo-Manager/
|-- config/
|   `-- config.json
|-- data/
|   |-- img/
|   |-- account.json
|   `-- car.json
|-- include/carinfo-manager/
|   |-- accountpool.hpp
|   |-- base64.hpp
|   |-- basicpool.hpp
|   |-- carpool.hpp
|   |-- hash.hpp
|   |-- httphandler-client.hpp
|   |-- httphandler-server.hpp
|   `-- log.hpp
|-- qt-include/carinfo-manager/
|   |-- add-account-window.hpp
|   |-- add-car-window.hpp
|   |-- change-password-window.hpp
|   |-- client-window.hpp
|   |-- display-accountinfo-window.hpp
|   |-- display-carinfo-window.hpp
|   |-- login-window.hpp
|   |-- manage-account-window.hpp
|   |-- manage-car-window.hpp
|   |-- manage-window.hpp
|   |-- qwidget-with-subwindows.hpp
|   |-- remove-account-window.hpp
|   |-- remove-car-window.hpp
|   |-- search-account-window.hpp
|   |-- search-car-window.hpp
|   |-- select-server-window.hpp
|   |-- update-account-window.hpp
|   `-- update-car-window.hpp
```

```
|-- qt-src/
|   |-- add-account-window.cpp
|   |-- add-car-window.cpp
|   |-- change-password-window.cpp
|   |-- client-window.cpp
|   |-- display-accountinfo-window.cpp
|   |-- display-carinfo-window.cpp
|   |-- login-window.cpp
|   |-- manage-account-window.cpp
|   |-- manage-car-window.cpp
|   |-- manage-window.cpp
|   |-- qwidget-with-subwindows.cpp
|   |-- remove-account-window.cpp
|   |-- remove-car-window.cpp
|   |-- search-account-window.cpp
|   |-- search-car-window.cpp
|   |-- select-server-window.cpp
|   |-- update-account-window.cpp
|   `-- update-car-window.cpp
|-- site-packages/
|   |-- cpp-httplib-0.16.0/
|   |-- nlohmann-json-3.11.3/
|   `-- spdlog-1.14.1/
|-- src/
|   |-- accountpool.cpp
|   |-- base64.cpp
|   |-- basicpool.cpp
|   |-- carpool.cpp
|   |-- hash.cpp
|   |-- httphandler-client.cpp
|   |-- httphandler-server.cpp
|   `-- log.cpp
`-- CMakeLists.txt
```

对应的文件内容如下：

config/config.json

```
{
  "ip": "127.0.0.1",
  "port": 12345,
  "dataDir": "data/"
}
```

data/account.json

```
{
  "admin1": {
    "account_type": 1,
    "passwd_hash": "4
        b7897eed3bebaa5cfe986afb4075ba5d9f42959bc9b89cf6af6c3af403ef14e"
        ,
    "username": "admin1"
  },
  "user1": {
    "account_type": 2,
    "passwd_hash": "4
        b7897eed3bebaa5cfe986afb4075ba5d9f42959bc9b89cf6af6c3af403ef14e"
        ,
    "username": "user1"
  }
}
```

data/car.json

```
{
  "京A12345": {
    "color": "红",
    "id": "京A12345",
    "img_path": "data/img/5LqsQTEyMzQ1.jpg",
    "owner": "爱丽丝",
    "type": "特斯拉",
    "year": 2024
  }
}
```

include/carinfo-manager/accountpool.hpp

```
/**
 * @file include/carinfo-manager/accountpool.hpp
 * @brief Declaration of class Account and class AccountPool
 *
 * @details
 * This file contains the declaration of the Account class and the
    AccountPool class.
 * The Account class represents a user account with a username,
    password hash, and account type.
 * The AccountPool class manages a collection of user accounts and
```

```
     provides operations to add, remove, update, and verify accounts.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include <cstdint>
#include <map>
#include <string>
#include <vector>
#include "carinfo-manager/basicpool.hpp"

class Account {
  public:
  enum class AccountType { NONETYPE = 0, ADMIN = 1, USER = 2 };

  private:
  std::string username;
  std::string passwd_hash;
  AccountType account_type;

  public:
  Account();
  Account(const std::string &username,
      const std::string &passwd_hash,
      const AccountType &account_type);
  Account(const Account &acc);
  ~Account();
  std::string getUsername() const;
  std::string getPasswdHash() const;
  AccountType getAccountType() const;
  bool isAdmin() const;
  bool isUser() const;
  void setUsername(const std::string &username);
  void setPasswdHash(const std::string &passwd_hash);
  void setAccountType(const AccountType &account_type);

  bool operator==(const Account &acc) const;
  bool operator!=(const Account &acc) const;
```

```
49    bool operator<(const Account &acc) const;
50    bool operator>(const Account &acc) const;
51    bool operator<=(const Account &acc) const;
52    bool operator>=(const Account &acc) const;
53    Account &operator=(const Account &acc);
54    const static Account NULL_ACCOUNT;
55  };
56
57  class AccountPool : public BasicPool {
58    public:
59    enum class AccountVerifyResult { SUCCESS = 0, ACCOUNT_NOT_FOUND = 1,
          WRONG_PASSWORD = 2 };
60
61    private:
62    std::map<std::string, Account> accountpool;
63    std::map<std::string, Account> adminpool;
64    std::map<std::string, Account> userpool;
65
66    public:
67    AccountPool();
68    AccountPool(Account *begin, Account *end);
69    AccountPool(const std::vector<Account> &accounts);
70    AccountPool(const AccountPool &ap);
71    ~AccountPool();
72    int addAccount(const Account &acc);
73    int removeAccount(const std::string &username);
74    int removeAccount(const Account &acc);
75    int updateAccount(const Account &original_acc, const Account &new_acc
          );
76    Account getAccount(const std::string &username) const;
77    // 模糊查找
78    AccountPool getAccountLike(const std::string &username) const;
79    AccountVerifyResult verifyAccount(const std::string &username,
80                        const std::string &passwd_hash) const;
81    Account::AccountType getAccountType(const std::string &username)
          const;
82    size_t size() const;
83    bool empty() const;
84    int clear();
85    int load(std::istream &is);
86    int save(std::ostream &os) const;
```

```
87    std::vector<Account> list() const;
88
89    bool operator==(const AccountPool &ap) const;
90    bool operator!=(const AccountPool &ap) const;
91    AccountPool &operator=(const AccountPool &ap);
92  };
```

include/carinfo-manager/base64.hpp

```
1   /**
2    * @file include/carinfo-manager/base64.hpp
3    * @brief Declaration of class Base64
4    *
5    * @details
6    * This file contains the declaration of the Base64 class. The Base64
       class provides static methods to encode and decode data using the
       Base64 encoding scheme.
7    * The Base64 encoding scheme is commonly used to encode binary data as
       ASCII text, which can be easily transmitted over text-based
       protocols such as email or HTTP.
8    * The class provides two static methods:
9    *   - encode: Encodes a given byte array into a Base64-encoded string.
10   *   - decode: Decodes a Base64-encoded string into its original byte
       array.
11   * The class also defines two private static member variables:
12   *   - encodeTable: A lookup table used for encoding.
13   *   - decodeTable: A lookup table used for decoding.
14   * This class is designed to be used as a utility class and does not
       require any instantiation.
15   *
16   * @author donghy23@mails.tsinghua.edu.cn
17   * @version 1.0
18   */
19
20  #pragma once
21  #pragma execution_character_set("utf-8")
22  #include <string>
23
24  class Base64 {
25    private:
26    static const char encodeTable[];
27    static const char decodeTable[];
```

```
28
29    public:
30    static std::string encode(const unsigned char *data, size_t
          data_bytes_length);
31    static std::string decode(const char *data,
32                              size_t data_bytes_length,
33                              size_t &output_bytes_length);
34  };
```

include/carinfo-manager/basicpool.hpp

```
1   /**
2    * @file include/carinfo-manager/basicpool.hpp
3    * @brief Declaration of class BasicPool
4    *
5    * @details
6    * This file contains the declaration of the BasicPool class.
7    * The BasicPool class is an abstract class that provides a basic
         interface for managing a collection of objects.
8    *
9    * @author donghy23@mails.tsinghua.edu.cn
10   * @version 1.0
11   */
12
13  #pragma once
14  #pragma execution_character_set("utf-8")
15  #include <cstddef>
16  #include <cstdint>
17  #include <iostream>
18  #include <string>
19
20  class BasicPool {
21    protected:
22    size_t sz;
23
24    public:
25    BasicPool() {}
26
27    virtual ~BasicPool() {}
28
29    virtual size_t size() const = 0;
30    virtual bool empty() const = 0;
```

```
  virtual int clear() = 0;
  virtual int load(std::istream &is) = 0;
  virtual int save(std::ostream &os) const = 0;
};
```

include/carinfo-manager/carpool.hpp

```
/**
 * @file include/carinfo-manager/carpool.hpp
 * @brief Declaration of class Color, class Car, and class CarPool.
 *
 * @details
 * This file contains the declarations of the Color, Car, and CarPool
    classes.
 * The Color class represents a color with red, green, and blue
    components.
 * The Car class represents a car with an ID, type, color, year, and
    image path.
 * The CarPool class represents a collection of cars and provides
    various operations on them.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include <cstdint>
#include <map>
#include <set>
#include <string>
#include <vector>
#include "carinfo-manager/basicpool.hpp"

class Car {
  private:
  std::string car_id;
  std::string car_type;
  std::string car_owner;
  std::string car_color;
  int car_year;
  std::string car_img_path;
```

```
32
33    public:
34    Car();
35    Car(const std::string &id,
36      const std::string &type,
37      const std::string &owner,
38      const std::string &color,
39      const int year,
40      const std::string &img_path);
41    Car(const Car &c);
42    ~Car();
43    void setId(const std::string &id);
44    void setType(const std::string &type);
45    void setOwner(const std::string &owner);
46    void setColor(const std::string &color);
47    void setYear(const int year);
48    void setImagePath(const std::string &img_path);
49    std::string getId() const;
50    std::string getType() const;
51    std::string getOwner() const;
52    std::string getColor() const;
53    int getYear() const;
54    std::string getImagePath() const;
55
56    bool operator==(const Car &c) const;
57    bool operator!=(const Car &c) const;
58    bool operator<(const Car &c) const;
59    bool operator>(const Car &c) const;
60    bool operator<=(const Car &c) const;
61    bool operator>=(const Car &c) const;
62    Car &operator=(const Car &c);
63    const static Car NULL_CAR;
64  };
65
66  class CarPool : public BasicPool {
67    private:
68    std::map<std::string, Car> carpool_byid;
69    std::multimap<std::string, Car> carpool_byowner;
70    std::multimap<std::string, Car> carpool_bycolor;
71    std::multimap<std::string, Car> carpool_bytype;
72
```

```
73    public:
74    CarPool();
75    CarPool(Car *begin, Car *end);
76    CarPool(const std::vector<Car> &cars);
77    CarPool(const CarPool &cp);
78    ~CarPool();
79    int addCar(const Car &car);
80    int removeCar(const Car &car);
81    int removeCar(const std::string &id);
82    int updateCar(const Car &original_car, const Car &new_car);
83    int updateCar(const std::string &id, const Car &new_car);
84    CarPool getCarbyId(const std::string &id) const;
85    CarPool getCarbyColor(const std::string &color) const;
86    CarPool getCarbyOwner(const std::string &owner) const;
87    CarPool getCarbyType(const std::string &type) const;
88    CarPool getCar(const std::string &id = "",
89              const std::string &color = "",
90              const std::string &owner = "",
91              const std::string &type = "") const;
92    size_t size() const;
93    bool empty() const;
94    int clear();
95    int load(std::istream &is);
96    int save(std::ostream &os) const;
97    std::vector<Car> list() const;
98
99    // operator std::vector<Car>() const;
100   bool operator==(const CarPool &cp) const;
101   bool operator!=(const CarPool &cp) const;
102   CarPool &operator=(const CarPool &cp);
103 };
```

include/carinfo-manager/hash.hpp

```
1  /**
2   * @file include/carinfo-manager/hash.hpp
3   * @brief Declaration of class Hash, which is used to hash a string
        with salt using SHA-256 algorithm
4   *
5   * @details
6   * This file contains the declaration of the Hash class. The Hash class
        provides functionality to hash a string with a salt using the SHA
```

```
    -256 algorithm.
7   * The class includes private member variables for storing the data,
      salt, and hash. It also includes private member functions for
      performing various operations related to the SHA-256 algorithm.
8   * Public member functions are provided to set the data and salt,
      retrieve the hash, and perform other operations related to hashing.
9   * The Hash class is designed to be used in the Carinfo-Manager project
      .
10  *
11  * @author donghy23@mails.tsinghua.edu.cn & kibonga@github.com
12  * @version 1.0
13  */
14
15 #pragma once
16 #pragma execution_character_set("utf-8")
17 #include <cstdint>
18 #include <cstring>
19 #include <string>
20
21 class Hash {
22   private:
23   std::string data, salt, hash;
24   void sha256();
25
26   private:
27   const static uint32_t K[64];
28   const static int a, b, c, d, e, f, g, h;
29   uint32_t right_rotate(uint32_t x, uint32_t n);
30   uint32_t little_sigma_0(uint32_t x);
31   uint32_t little_sigma_1(uint32_t x);
32   uint32_t big_sigma_0(uint32_t x);
33   uint32_t big_sigma_1(uint32_t x);
34   void message_schedule(uint32_t (&W)[64], uint8_t (&block)[64]);
35   uint32_t choice(uint32_t x, uint32_t y, uint32_t z);
36   uint32_t majority(uint32_t x, uint32_t y, uint32_t z);
37   void round(uint32_t (&H)[8], uint32_t round_constant, uint32_t
       schedule_word);
38   void compress_block(uint32_t (&H)[8], uint8_t (&block)[64]);
39
40   public:
41   Hash();
```

```
42    Hash(std::string data, std::string salt);
43    Hash(const Hash &hashObj);
44    ~Hash();
45    void setData(std::string data);
46    void setSalt(std::string salt);
47    std::string getHash();
48  };
```

include/carinfo-manager/httphandler-client.hpp

```
1   /**
2    * @file include/carinfo-manager/httphandler-client.hpp
3    * @brief Declaration of ClientHttpHandler class, AccountVerifyResult
       enum, and HttpResult class
4    *
5    * @details
6    * This file contains the declaration of the ClientHttpHandler class,
       AccountVerifyResult enum, and HttpResult class.
7    * The ClientHttpHandler class provides static methods to handle HTTP
       requests from the client side.
8    * The AccountVerifyResult enum represents the result of account
       verification.
9    * The HttpResult class represents the result of an HTTP request.
10   *
11   * @author donghy23@mails.tsinghua.edu.cn
12   * @version 1.0
13   */
14
15  #pragma once
16  #pragma execution_character_set("utf-8")
17  #include <string>
18  #include "carinfo-manager/accountpool.hpp"
19  #include "carinfo-manager/carpool.hpp"
20  #include "cpp-httplib/httplib.h"
21  #include "json/json.hpp"
22
23  class ClientHttpHandler {
24    public:
25    enum class AccountVerifyResult {
26      SUCCESS = 0,
27      ACCOUNT_NOT_FOUND_OR_WRONG_PASSWORD = 1,
28      NETWORK_ERROR = 2,
```

```
29      INVALID_RESPONSE = 3,
30      BAD_REQUEST = 4
31    };
32
33    class HttpResult {
34      public:
35      int status;
36      std::string message;
37      nlohmann::json json_obj;
38
39      HttpResult(int status = 0,
40              const std::string &message = "",
41              const nlohmann::json &json_obj = nlohmann::json())
42        : status(status), message(message), json_obj(json_obj) {}
43
44      ~HttpResult() {}
45
46      operator bool() const { return status == 200; }
47    };
48
49    private:
50    static nlohmann::json parse_resp_content(const httplib::Response &
        resp);
51
52    public:
53    ClientHttpHandler();
54
55    static bool handler_verify_server(const std::string &ip, int port);
56    static AccountVerifyResult handler_login(const std::string &ip, int
        port, Account &acc);
57    static HttpResult handler_get_carinfo(const std::string &ip,
58                      int port,
59                      const Account &acc,
60                      const std::string &carid,
61                      const std::string &carowner,
62                      const std::string &carcolor,
63                      const std::string &cartype);
64    static HttpResult handler_get_carimg(const std::string &ip,
65                       int port,
66                       const Account &acc,
67                       std::string car_img_path);
```

```
68  static HttpResult handler_add_car(const std::string &ip,
69                      const int port,
70                      const Account &acc,
71                      const std::string &car_id,
72                      const std::string &car_owner,
73                      const std::string &car_type,
74                      const std::string &car_color,
75                      const int car_year,
76                      const std::string &car_img,
77                      const std::string &car_img_type);
78  static HttpResult handler_remove_car(const std::string &ip,
79                       const int port,
80                       const Account &acc,
81                       const std::string &car_id);
82  static HttpResult handler_update_car(const std::string &ip,
83                       const int port,
84                       const Account &acc,
85                       const std::string &original_car_id,
86                       const std::string &new_car_id,
87                       const std::string &new_car_owner,
88                       const std::string &new_car_type,
89                       const std::string &new_car_color,
90                       const int new_car_year,
91                       const std::string &new_car_img,
92                       const std::string &new_car_img_type);
93  static HttpResult handler_search_account(const std::string &ip,
94                         const int port,
95                         const Account &acc,
96                         const std::string &search_username);
97  static HttpResult handler_get_all_accounts(const std::string &ip,
98                           const int port,
99                           const Account &acc);
100 static HttpResult handler_add_account(const std::string &ip,
101                       const int port,
102                       const Account &acc,
103                       const std::string &new_username,
104                       const std::string &new_passwd_hash,
105                       const Account::AccountType new_account_type);
106 static HttpResult handler_remove_account(const std::string &ip,
107                         const int port,
108                         const Account &acc,
```

```
                        const std::string &target_username);
  static HttpResult handler_update_account(const std::string &ip,
                        const int port,
                        const Account &acc,
                        const std::string &target_username,
                        const std::string &new_username,
                        const std::string &new_passwd_hash,
                        const Account::AccountType new_account_type);
  static HttpResult handler_change_password(const std::string &ip,
                        const int port,
                        const Account &acc,
                        const std::string &new_passwd_hash);
};
```

include/carinfo-manager/httphandler-server.hpp

```
/**
 * @file include/carinfo-manager/httphandler-server.hpp
 * @brief Declaration of class ServerHttpHandler
 *
 * @details
 * This file contains the declaration of the ServerHttpHandler class,
    which handles HTTP requests for car information management.
 * The ServerHttpHandler class provides methods for handling various
    types of requests, such as testing the connection, login or
    password change, car management, and account management.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/carpool.hpp"
#include "cpp-httplib/httplib.h"
#include "json/json.hpp"

class ServerHttpHandler {
  private:
  AccountPool &accountpool;
  CarPool &carpool;
```

```
24    std::string imgDir;  // end with '/'
25
26    private:
27    nlohmann::json parse_post_body(const httplib::Request &req) const;
28
29    public:
30    ServerHttpHandler(AccountPool &accountpool, CarPool &carpool, std::
          string imgDir);
31    // test connection
32    void handler_test_connection(const httplib::Request &req, httplib::
          Response &res) const;
33    // login or change password
34    void handler_login(const httplib::Request &req, httplib::Response &
          res) const;
35    void handler_change_password(const httplib::Request &req, httplib::
          Response &res);
36    // car management
37    void handler_get_carinfo(const httplib::Request &req, httplib::
          Response &res) const;
38    void handler_get_carimg(const httplib::Request &req, httplib::
          Response &res) const;
39    void handler_add_car(const httplib::Request &req, httplib::Response &
          res);
40    void handler_remove_car(const httplib::Request &req, httplib::
          Response &res);
41    void handler_update_car(const httplib::Request &req, httplib::
          Response &res);
42    // account management
43    void handler_get_accountinfo(const httplib::Request &req, httplib::
          Response &res) const;
44    void handler_get_all_account(const httplib::Request &req, httplib::
          Response &res) const;
45    void handler_add_account(const httplib::Request &req, httplib::
          Response &res);
46    void handler_remove_account(const httplib::Request &req, httplib::
          Response &res);
47    void handler_update_account(const httplib::Request &req, httplib::
          Response &res);
48 };
```

include/carinfo-manager/log.hpp

```cpp
/**
 * @file include/carinfo-manager/log.hpp
 * @brief Declaration of class MyLogger, LOG_LEVEL, and LOG_TYPE
 *
 * @details
 * This file contains the declaration of the MyLogger class, LOG_LEVEL class, and LOG_TYPE class.
 * The MyLogger class provides a static method to register loggers and log messages.
 *   - registerLogger registers a logger with a logger name, log level, log type, and log file path.
 *   - log logs a message with a logger name, log level, and message.
 * The LOG_LEVEL class provides constants for different log levels.
 * The LOG_TYPE class provides constants for different log types.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#include <cstdint>
#include <string>

#ifdef ERROR
#undef ERROR
#endif

class MyLogger {
  public:
  class LOG_LEVEL {
    public:
    static constexpr uint8_t DEBUG = 1;
    static constexpr uint8_t INFO = 2;
    static constexpr uint8_t WARN = 4;
    static constexpr uint8_t ERROR = 8;
    static constexpr uint8_t CRITICAL = 16;
  };

  class LOG_TYPE {
    public:
    static constexpr uint8_t CONSOLE = 1;
```

```
39      static constexpr uint8_t FILE = 2;
40    };
41
42    public:
43    static void registerLogger(const std::string &name,
44                    const uint8_t level,
45                    const uint8_t type,
46                    const std::string &path = "");
47    static void log(const std::string &name, const uint8_t level, const
          std::string &msg);
48  };
```

qt-include/carinfo-manager/add-account-window.hpp

```
1   /**
2    * @file qt-include/carinfo-manager/add-account-window.hpp
3    * @brief Declaration of class AddAccountWindow
4    *
5    * @details
6    * This file contains the declaration of the AddAccountWindow class.
7    * The AddAccountWindow class provides a window to add a new account.
8    *
9    * @author donghy23@mails.tsinghua.edu.cn
10   * @version 1.0
11   */
12
13  #pragma once
14  #include <QPushButton>
15  #include <QWidget>
16  #include <QLabel>
17  #include <QLineEdit>
18  #include <QComboBox>
19  #include <string>
20  #include "carinfo-manager/accountpool.hpp"
21  #include "carinfo-manager/qwidget-with-subwindows.hpp"
22  #include "carinfo-manager/httphandler-client.hpp"
23
24  class AddAccountWindow : public QWidgetWithSubWindows {
25    Q_OBJECT
26    public:
27      AddAccountWindow(QWidget *parent,
28                const std::string &ip,
```

```
29                const int port,
30                const Account &acc,
31                QWidgetWithSubWindows *parentWindow = nullptr);
32      ~AddAccountWindow();
33
34    private:
35      std::string ip;
36      int port;
37      Account acc;
38      QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
39      QLabel *newUsernameLabel, *newPasswordLabel, *confirmPasswordLabel,
            *newAccounttypeLabel;
40      QLineEdit *newUsernameEdit, *newPasswordEdit, *confirmPasswordEdit;
41      QComboBox *newAccounttypeComboBox;
42      QPushButton *addAccountButton;
43
44    private slots:
45      void addAccount();
46  };
```

qt-include/carinfo-manager/add-car-window.hpp

```
1   /**
2    * @file qt-include/carinfo-manager/add-car-window.hpp
3    * @brief Declaration of class AddCarWindow
4    *
5    * @details
6    * This file contains the declaration of the AddCarWindow class.
7    * The AddCarWindow class provides a window to add a new car.
8    *
9    * @author donghy23@mails.tsinghua.edu.cn
10   * @version 1.0
11   */
12
13  #pragma once
14  #pragma execution_character_set("utf-8")
15  #include <QFileDialog>
16  #include <QLabel>
17  #include <QLineEdit>
18  #include <QPushButton>
19  #include <QWidget>
20  #include <string>
```

```
21 #include "carinfo-manager/accountpool.hpp"
22 #include "carinfo-manager/httphandler-client.hpp"
23 #include "carinfo-manager/qwidget-with-subwindows.hpp"
24
25 class AddCarWindow : public QWidgetWithSubWindows {
26   Q_OBJECT
27
28   public:
29   AddCarWindow(QWidget *parent = nullptr,
30           const std::string &ip = "",
31           const int port = 0,
32           const Account &acc = Account(),
33           QWidgetWithSubWindows *parentWindow = nullptr);
34   ~AddCarWindow();
35
36   private:
37   std::string ip;
38   int port;
39   Account acc;
40   QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
41   QLabel *carIdLabel, *carOwnerLabel, *carTypeLabel, *carColorLabel, *
       carYearLabel,
42     *carImgPathLabel;
43   QLineEdit *carIdLineEdit, *carOwnerLineEdit, *carTypeLineEdit, *
       carColorLineEdit,
44     *carYearLineEdit, *carImgPathLineEdit;
45   QPushButton *carImgSelectButton, *addCarButton;
46
47   private slots:
48   void getCarImagePath();
49   void addCar();
50 };
```

qt-include/carinfo-manager/change-password-window.hpp

```
1 /**
2  * @file qt-include/carinfo-manager/change-password-window.hpp
3  * @brief Declaration of class ChangePasswordWindow
4  *
5  * @details
6  * This file contains the declaration of the ChangePasswordWindow class
       .
```

```
 * The ChangePasswordWindow class provides a window to change the
    password of an account.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include <QWidget>
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/qwidget-with-subwindows.hpp"
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>

class ChangePasswordWindow : public QWidgetWithSubWindows {
  Q_OBJECT
  public:
  ChangePasswordWindow(QWidget *parent,
            const std::string &ip,
            const int port,
            const Account &acc,
            QWidgetWithSubWindows *parentWindow = nullptr);
  ~ChangePasswordWindow();

  private:
  std::string ip;
  int port;
  Account acc;
  QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
  QLabel *oldPasswordLabel, *newPasswordLabel, *confirmPasswordLabel;
  QLineEdit *oldPasswordEdit, *newPasswordEdit, *confirmPasswordEdit;
  QPushButton *changePasswordButton;

  private slots:
  void changePassword();
};
```

qt-include/carinfo-manager/client-window.hpp

```
/**
 * @file qt-include/carinfo-manager/client-window.hpp
 * @brief Declaration of class ClientMain
```

```
 *
 * @details
 * This file contains the declaration of the ClientMain class.
 * The ClientMain class provides the main entry of the client program.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include <QApplication>

class ClientMain : public QApplication {
  public:
  ClientMain(int argc, char *argv[]);
  ~ClientMain();
  int run();
};
```

qt-include/carinfo-manager/display-accountinfo-window.hpp

```
/**
 * @file qt-include/carinfo-manager/display-accountinfo-window.hpp
 * @brief Declaration of class DisplayAccountinfoWindow
 *
 * @details
 * This file contains the declaration of the DisplayAccountinfoWindow
    class.
 * The DisplayAccountinfoWindow class provides a window to display the
    information of an account.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include <QBoxLayout>
#include <QLabel>
#include <QScrollArea>
#include <QWidget>
```

```
#include <string>
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/qwidget-with-subwindows.hpp"

class DisplayAccountinfoWindow : public QWidgetWithSubWindows {
  Q_OBJECT

  public:
  DisplayAccountinfoWindow(QWidget *parent = nullptr,
                const AccountPool &accountpool = AccountPool(),
                const std::string searchCarId = "",
                const std::string &ip = "",
                const int port = 0,
                const Account &acc = Account(),
                QWidgetWithSubWindows *parentWindow = nullptr);
  ~DisplayAccountinfoWindow();

  private:
  std::string ip;
  int port;
  Account acc;
  AccountPool accountpool;
  std::string searchCarId;
  QVBoxLayout *outerLayout, *innerLayout;
  QWidget *accountinfoWidget;
  QScrollArea *accountinfoScrollArea;
  QLabel *accountQueryLabel;
};
```

qt-include/carinfo-manager/display-carinfo-window.hpp

```
/**
 * @file display-carinfo-window.hpp
 * @brief Declaration of class DisplayCarInfoWindow
 *
 * @details
 * This file contains the declaration of the DisplayCarInfoWindow class
    .
 * The DisplayCarInfoWindow class provides a window to display the
    information of a car.
 *
 * @author donghy23@mails.tsinghua.edu.cn
```

```
 * @version 1.0
 */

#pragma once
#pragma execution_character_set("utf-8")
#include <QBoxLayout>
#include <QLabel>
#include <QScrollArea>
#include <QWidget>
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/carpool.hpp"
#include "carinfo-manager/qwidget-with-subwindows.hpp"

class SearchCarQuery {
  public:
  SearchCarQuery(const std::string &searchCarId = "",
           const std::string &searchCarOwner = "",
           const std::string &searchCarType = "",
           const std::string &searchCarColor = "");
  SearchCarQuery(const SearchCarQuery &query);
  ~SearchCarQuery();

  public:
  std::string searchCarId, searchCarOwner, searchCarType,
     searchCarColor;
};

class DisplayCarInfoWindow : public QWidgetWithSubWindows, private
   SearchCarQuery {
  Q_OBJECT

  public:
  DisplayCarInfoWindow(QWidget *parent = nullptr,
             const CarPool &carpool = CarPool(),
             const SearchCarQuery &query = SearchCarQuery(),
             const std::string &ip = "",
             const int port = 0,
             const Account &acc = Account(),
             QWidgetWithSubWindows *parentWindow = nullptr);
  ~DisplayCarInfoWindow();

```

```
49    private:
50    std::string ip;
51    int port;
52    Account acc;
53    CarPool carpool;
54    QVBoxLayout *outerLayout, *innerLayout;
55    QWidget *carinfoWidget;
56    QScrollArea *carinfoScrollArea;
57    QLabel *carQueryTitleLabel, *carQueryIdLabel, *carQueryOwnerLabel, *
          carQueryTypeLabel,
58      *carQueryColorLabel;
59  };
```

qt-include/carinfo-manager/login-window.hpp

```
1   /**
2    * @file qt-include/carinfo-manager/login-window.hpp
3    * @brief Declaration of class LoginWindow
4    *
5    * @details
6    * This file contains the declaration of the LoginWindow class.
7    * The LoginWindow class provides a window to login.
8    *
9    * @author donghy23@mails.tsinghua.edu.cn
10   * @version 1.0
11   */
12
13  #pragma once
14  #pragma execution_character_set("utf-8")
15  #include <QLabel>
16  #include <QLineEdit>
17  #include <QMessageBox>
18  #include <QPushButton>
19  #include <QWidget>
20  #include <string>
21
22  class LoginWindow : public QWidget {
23    Q_OBJECT
24
25    public:
26    LoginWindow(QWidget *parent = nullptr, const std::string &ip = "",
          const int port = 0);
```

```
27    ~LoginWindow();
28
29    private slots:
30    void login();
31    void returnToSelectServer();
32
33    private:
34    std::string ip;
35    int port;
36    QLabel *titleLabel, *usernameLabel, *passwordLabel;
37    QLineEdit *usernameEdit, *passwordEdit;
38    QPushButton *loginButton, *returnButton;
39    std::string username, password;
40 };
```

qt-include/carinfo-manager/manage-account-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/manage-account-window.hpp
3   * @brief Declaration of class ManageAccountWindow
4   *
5   * @details
6   * This file contains the declaration of the ManageAccountWindow class.
7   * The ManageAccountWindow class provides a window to manage an account
        .
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #include <QLabel>
15 #include <QPushButton>
16 #include <QWidget>
17 #include <string>
18 #include "carinfo-manager/accountpool.hpp"
19 #include "carinfo-manager/qwidget-with-subwindows.hpp"
20
21 class ManageAccountWindow : public QWidgetWithSubWindows {
22   Q_OBJECT
23   public:
24   ManageAccountWindow(QWidget *parent,
```

```
25              const std::string &ip,
26              const int port,
27              const Account &acc,
28              QWidgetWithSubWindows *parentWindow = nullptr);
29    ~ManageAccountWindow();
30
31    private:
32    std::string ip;
33    int port;
34    Account acc;
35    QPushButton *addAccountButton, *updateAccountButton, *
          removeAccountButton;
36    QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
37
38    private slots:
39    void addAccount();
40    void removeAccount();
41    void updateAccount();
42 };
```

qt-include/carinfo-manager/manage-car-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/manage-car-window.hpp
3   * @brief Declaration of class ManageAccountWindow
4   *
5   * @details
6   * This file contains the declaration of the ManageAccountWindow class.
7   * The ManageAccountWindow class provides a window to manage an account
        .
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #include <QLabel>
15 #include <QPushButton>
16 #include <QWidget>
17 #include <string>
18 #include "carinfo-manager/accountpool.hpp"
19 #include "carinfo-manager/qwidget-with-subwindows.hpp"
```

```
20
21 class ManageCarWindow : public QWidgetWithSubWindows {
22   Q_OBJECT
23   public:
24   ManageCarWindow(QWidget *parent,
25           const std::string &ip,
26           const int port,
27           const Account &acc,
28           QWidgetWithSubWindows *parentWindow = nullptr);
29   ~ManageCarWindow();
30
31   private:
32   std::string ip;
33   int port;
34   Account acc;
35   QPushButton *addCarButton, *updateCarButton, *removeCarButton;
36   QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
37
38   private slots:
39   void addCar();
40   void removeCar();
41   void updateCar();
42 };
```

qt-include/carinfo-manager/manage-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/manage-window.hpp
3   * @brief Declaration of class ManageWindow
4   *
5   * @details
6   * This file contains the declaration of the ManageWindow class.
7   * The ManageWindow class provides a window to manage accounts and cars
        .
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #pragma execution_character_set("utf-8")
15 #include <QLabel>
```

```
16 #include <QPushButton>
17 #include <QWidget>
18 #include <set>
19 #include <string>
20 #include "carinfo-manager/accountpool.hpp"
21 #include "carinfo-manager/qwidget-with-subwindows.hpp"
22 #include "carinfo-manager/search-car-window.hpp"
23
24 class ManageWindow : public QWidgetWithSubWindows {
25   Q_OBJECT
26
27   public:
28   ManageWindow(QWidget *parent = nullptr,
29           const std::string &ip = "",
30           const int port = 0,
31           const Account &acc = Account());
32   ~ManageWindow();
33
34   private slots:
35   void accountManage();
36   void carManage();
37   void accountSearch();
38   void carSearch();
39   void changePassword();
40   void logout();
41
42   private:
43   QLabel *titleLabel;
44   QLabel *usernameLabel, *accounttypeLabel;
45   QPushButton *accountManageButton, *carManageButton, *
       accountSearchButton, *carSearchButton,
46     *changePasswordButton, *logoutButton;
47   std::string ip;
48   int port;
49   Account acc;
50 };
```

qt-include/carinfo-manager/qwidget-with-subwindows.hpp

```
1 /**
2  * @file qt-include/carinfo-manager/qwidget-with-subwindows.hpp
3  * @brief Declaration of class QWidgetWithSubWindows
```

```
 *
 * @details
 * This file contains the declaration of the QWidgetWithSubWindows class.
 * The QWidgetWithSubWindows class provides a window with sub windows.
 * It can be used as a base class for windows, which close all sub windows when needed.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#include <QWidget>
#include <set>

class QWidgetWithSubWindows : public QWidget {
  Q_OBJECT

  protected:
  std::set<QWidgetWithSubWindows *> subWindows;
  QWidgetWithSubWindows *parentWindow;
  bool closeFromParent = false;

  public:
  QWidgetWithSubWindows(
    QWidget *parent,
    QWidgetWithSubWindows *parentWindow = nullptr,
    const std::set<QWidgetWithSubWindows *> &subWindows = std::set<QWidgetWithSubWindows *>());
  void addSubWindow(QWidgetWithSubWindows *window);
  void removeSubWindow(QWidgetWithSubWindows *window);
  void closeSubWindow(QWidgetWithSubWindows *window);
  void closeAllSubWindows();
  void removeFromParent();
  void setCloseFromParent(const bool closeFromParent);
  bool getCloseFromParent() const;
  virtual void closeEvent(QCloseEvent *event) override;
  ~QWidgetWithSubWindows();
};
```

qt-include/carinfo-manager/remove-account-window.hpp

```
/**
 * @file qt-include/carinfo-manager/remove-account-window.hpp
 * @brief Declaration of class RemoveAccountWindow
 *
 * @details
 * This file contains the declaration of the RemoveAccountWindow class.
 * The RemoveAccountWindow class provides a window to remove an account
    .
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#include <QPushButton>
#include <QWidget>
#include <QLabel>
#include <QLineEdit>
#include <string>
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/qwidget-with-subwindows.hpp"

class RemoveAccountWindow : public QWidgetWithSubWindows {
  Q_OBJECT
  public:
  RemoveAccountWindow(QWidget *parent,
            const std::string &ip,
            const int port,
            const Account &acc,
            QWidgetWithSubWindows *parentWindow = nullptr);
  ~RemoveAccountWindow();

  private:
  std::string ip;
  int port;
  Account acc;
  QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
  QLabel *targetUsernameLabel;
  QLineEdit *targetUsernameEdit;
  QPushButton *removeButton;
```

```
40
41    private slots:
42    void removeAccount();
43 };
```

qt-include/carinfo-manager/remove-car-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/remove-car-window.hpp
3   * @brief Declaration of class RemoveCarWindow
4   *
5   * @details
6   * This file contains the declaration of the RemoveCarWindow class.
7   * The RemoveCarWindow class provides a window to remove a car.
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #include <QLabel>
15 #include <QLineEdit>
16 #include <QPushButton>
17 #include <QWidget>
18 #include <string>
19 #include "carinfo-manager/accountpool.hpp"
20 #include "carinfo-manager/httphandler-client.hpp"
21 #include "carinfo-manager/qwidget-with-subwindows.hpp"
22
23 class RemoveCarWindow : public QWidgetWithSubWindows {
24   Q_OBJECT
25
26   public:
27   RemoveCarWindow(QWidget *parent = nullptr,
28           const std::string &ip = "",
29           const int port = 0,
30           const Account &acc = Account(),
31           QWidgetWithSubWindows *parentWindow = nullptr);
32   ~RemoveCarWindow();
33
34   private:
35   std::string ip;
```

```
36    int port;
37    Account acc;
38    QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
39    QLabel *carIdLabel;
40    QLineEdit *carIdLineEdit;
41    QPushButton *removeCarButton;
42
43    private slots:
44    void removeCar();
45 };
```

qt-include/carinfo-manager/search-account-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/search-account-window.hpp
3   * @brief Declaration of class SearchAccountWindow
4   *
5   * @details
6   * This file contains the declaration of the SearchAccountWindow class.
7   * The SearchAccountWindow class provides a window to search an account
        .
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #pragma execution_character_set("utf-8")
15 #include <QLabel>
16 #include <QLineEdit>
17 #include <QPushButton>
18 #include <QWidget>
19 #include <string>
20 #include "carinfo-manager/accountpool.hpp"
21 #include "carinfo-manager/qwidget-with-subwindows.hpp"
22
23 class SearchAccountWindow : public QWidgetWithSubWindows {
24   Q_OBJECT
25
26   public:
27   SearchAccountWindow(QWidget *parent = nullptr,
28             const std::string &ip = "",
```

```
29              const int port = 0,
30              const Account &acc = Account(),
31              QWidgetWithSubWindows *parentWindow = nullptr);
32    ~SearchAccountWindow();
33
34    private:
35    std::string ip;
36    int port;
37    Account acc;
38    QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
39    QLabel *searchUsernameLabel;
40    QLineEdit *searchUsernameLineEdit;
41    QPushButton *searchButton, *saveAllAccountsButton;
42
43    private slots:
44    void searchAccount();
45    void saveAllAccounts();
46 };
```

qt-include/carinfo-manager/search-car-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/search-car-window.hpp
3   * @brief Declaration of class SearchCarWindow
4   *
5   * @details
6   * This file contains the declaration of the SearchCarWindow class.
7   * The SearchCarWindow class provides a window to search a car.
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
14 #pragma execution_character_set("utf-8")
15 #include <QLabel>
16 #include <QLineEdit>
17 #include <QMessageBox>
18 #include <QPushButton>
19 #include <QWidget>
20 #include <string>
21 #include "carinfo-manager/accountpool.hpp"
```

```
22 #include "carinfo-manager/qwidget-with-subwindows.hpp"
23
24 class SearchCarWindow : public QWidgetWithSubWindows {
25   Q_OBJECT
26
27   public:
28   SearchCarWindow(QWidget *parent = nullptr,
29           const std::string &ip = "",
30           const int port = 0,
31           const Account &acc = Account(),
32           QWidgetWithSubWindows *parentWindow = nullptr);
33   ~SearchCarWindow();
34
35   private slots:
36   void search();
37
38   private:
39   std::string ip;
40   int port;
41   Account acc;
42   QLabel *usernameLabel, *accounttypeLabel;
43   QLabel *titleLabel, *caridLabel, *carownerLabel, *cartypeLabel, *
       carcolorLabel;
44   QLineEdit *caridEdit, *carownerEdit, *cartypeEdit, *carcolorEdit;
45   QPushButton *searchButton;
46 };
```

qt-include/carinfo-manager/select-server-window.hpp

```
1  /**
2   * @file qt-include/carinfo-manager/select-server-window.hpp
3   * @brief Declaration of class SelectServerWindow
4   *
5   * @details
6   * This file contains the declaration of the SelectServerWindow class.
7   * The SelectServerWindow class provides a window to select a server.
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
10  * @version 1.0
11  */
12
13 #pragma once
```

```
#pragma execution_character_set("utf-8")
#include <QLabel>
#include <QLineEdit>
#include <QMessageBox>
#include <QPushButton>
#include <QWidget>
#include <string>

class SelectServerWindow : public QWidget {
  Q_OBJECT

  public:
  SelectServerWindow(QWidget *parent = nullptr);
  ~SelectServerWindow();

  private slots:
  void verifyServer();

  private:
  std::string ip;
  int port;
  QLabel *titleLabel, *serverLabel, *portLabel;
  QLineEdit *serverEdit, *portEdit;
  QPushButton *verifyButton;
};
```

qt-include/carinfo-manager/update-account-window.hpp

```
/**
 * @file qt-include/carinfo-manager/update-account-window.hpp
 * @brief Declaration of class UpdateAccountWindow
 *
 * @details
 * This file contains the declaration of the UpdateAccountWindow class.
 * The UpdateAccountWindow class provides a window to update an account
    .
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
```

```
14 #include <QComboBox>
15 #include <QLabel>
16 #include <QLineEdit>
17 #include <QPushButton>
18 #include <QWidget>
19 #include <string>
20 #include "carinfo-manager/accountpool.hpp"
21 #include "carinfo-manager/qwidget-with-subwindows.hpp"
22
23 class UpdateAccountWindow : public QWidgetWithSubWindows {
24   Q_OBJECT
25   public:
26   UpdateAccountWindow(QWidget *parent,
27             const std::string &ip,
28             const int port,
29             const Account &acc,
30             QWidgetWithSubWindows *parentWindow = nullptr);
31   ~UpdateAccountWindow();
32
33   private:
34   std::string ip;
35   int port;
36   Account acc;
37   QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
38   QLabel *targetUsernameLabel, *newUsernameLabel, *newPasswordLabel, *
       confirmPasswordLabel, *newAccounttypeLabel;
39   QLineEdit *targetUsernameEdit, *newUsernameEdit, *newPasswordEdit, *
       confirmPasswordEdit;
40   QComboBox *newAccounttypeComboBox;
41   QPushButton *updateAccountButton;
42
43   private slots:
44   void updateAccount();
45 };
```

qt-include/carinfo-manager/update-car-window.hpp

```
1 /**
2  * @file qt-include/carinfo-manager/update-car-window.hpp
3  * @brief Declaration of class UpdateCarWindow
4  *
5  * @details
```

```
 * This file contains the declaration of the UpdateCarWindow class.
 * The UpdateCarWindow class provides a window to update a car.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#pragma once
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QWidget>
#include <string>
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/qwidget-with-subwindows.hpp"

class UpdateCarWindow : public QWidgetWithSubWindows {
  Q_OBJECT

  public:
  UpdateCarWindow(QWidget *parent = nullptr,
          const std::string &ip = "",
          const int port = 0,
          const Account &acc = Account(),
          QWidgetWithSubWindows *parentWindow = nullptr);
  ~UpdateCarWindow();

  private:
  std::string ip;
  int port;
  Account acc;
  QLabel *titleLabel, *usernameLabel, *accounttypeLabel;
  QLabel *originalCarIdLabel, *newCarIdLabel, *newCarOwnerLabel, *
     newCarTypeLabel,
    *newCarColorLabel, *newCarYearLabel, *newCarImgPathLabel;
  QLineEdit *originalCarIdLineEdit, *newCarIdLineEdit, *
     newCarOwnerLineEdit, *newCarTypeLineEdit,
    *newCarColorLineEdit, *newCarYearLineEdit, *newCarImgPathLineEdit;
  QPushButton *carImgSelectButton, *updateCarButton;

  private slots:
```

```
45    void getCarImgPath();
46    void updateCar();
47  };
```

qt-src/add-account-window.cpp

```
1   /**
2    * @file qt-src/add-account-window.cpp
3    * @brief Implementation of class AddAccountWindow
4    *
5    * @details
6    * This file contains the implementation of the AddAccountWindow class.
7    *
8    * @author donghy23@mails.tsinghua.edu.cn
9    * @version 1.0
10   */
11
12  #include "carinfo-manager/add-account-window.hpp"
13  #include <QMessageBox>
14  #include "carinfo-manager/hash.hpp"
15  #include "carinfo-manager/httphandler-client.hpp"
16  #include "carinfo-manager/log.hpp"
17
18  AddAccountWindow::AddAccountWindow(QWidget *parent,
19                      const std::string &ip,
20                      const int port,
21                      const Account &acc,
22                      QWidgetWithSubWindows *parentWindow)
23    : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
        acc(acc) {
24    this->setWindowTitle("添加账户");
25    this->setFixedSize(320, 330);
26
27    titleLabel = new QLabel("添加账户", this);
28    titleLabel->setGeometry(110, 50, 100, 30);
29    titleLabel->setAlignment(Qt::AlignCenter);
30    titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
31
32    usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
          this);
33    usernameLabel->setGeometry(250, 20, 50, 15);
34    usernameLabel->setAlignment(Qt::AlignCenter);
```

```
35    accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
36    accounttypeLabel->setGeometry(250, 40, 50, 15);
37    accounttypeLabel->setAlignment(Qt::AlignCenter);
38
39    newUsernameLabel = new QLabel("用户名: ", this);
40    newUsernameLabel->setGeometry(50, 100, 60, 30);
41    newUsernameEdit = new QLineEdit(this);
42    newUsernameEdit->setGeometry(120, 100, 140, 30);
43
44    newPasswordLabel = new QLabel("密码: ", this);
45    newPasswordLabel->setGeometry(50, 140, 60, 30);
46    newPasswordEdit = new QLineEdit(this);
47    newPasswordEdit->setEchoMode(QLineEdit::Password);
48    newPasswordEdit->setGeometry(120, 140, 140, 30);
49
50    confirmPasswordLabel = new QLabel("确认密码: ", this);
51    confirmPasswordLabel->setGeometry(50, 180, 60, 30);
52    confirmPasswordEdit = new QLineEdit(this);
53    confirmPasswordEdit->setEchoMode(QLineEdit::Password);
54    confirmPasswordEdit->setGeometry(120, 180, 140, 30);
55
56    newAccounttypeLabel = new QLabel("账户类型: ", this);
57    newAccounttypeLabel->setGeometry(50, 220, 60, 30);
58    newAccounttypeComboBox = new QComboBox(this);
59    newAccounttypeComboBox->setGeometry(120, 220, 140, 30);
60    newAccounttypeComboBox->addItem("用户");
61    newAccounttypeComboBox->addItem("管理员");
62
63    addAccountButton = new QPushButton("添加", this);
64    addAccountButton->setGeometry(100, 270, 100, 30);
65    connect(this->addAccountButton, &QPushButton::clicked, this, &
          AddAccountWindow::addAccount);
66  }
67
68  AddAccountWindow::~AddAccountWindow() {
69    closeAllSubWindows();
70    removeFromParent();
71  }
72
73  void AddAccountWindow::addAccount() {
74    std::string newUsername = this->newUsernameEdit->text().toStdString()
```

```
    ;
75  std::string newPassword = this->newPasswordEdit->text().toStdString()
    ;
76  std::string confirmPassword = this->confirmPasswordEdit->text().
    toStdString();
77  Account::AccountType newAccountType = this->newAccounttypeComboBox->
    currentIndex() == 0
78                        ? Account::AccountType::USER
79                        : Account::AccountType::ADMIN;
80  if (newPassword != confirmPassword) {
81    QMessageBox::critical(this, "错误", "两次输入的密码不一致");
82    return;
83  }
84  if (newUsername.empty() || newPassword.empty()) {
85    QMessageBox::critical(this, "错误", "用户名和密码不能为空");
86    return;
87  }
88
89  Hash passwordHash(newPassword, "Carinfo-Manager");
90  std::string newPasswdHash = passwordHash.getHash();
91
92  auto res = ClientHttpHandler::handler_add_account(
93    this->ip, this->port, this->acc, newUsername, newPasswdHash,
       newAccountType);
94  if (res)
95    QMessageBox::information(this, "成功", "添加账户成功");
96  else
97    QMessageBox::critical(
98      this, "错误", std::string("添加账户失败
          n错误信息: " + res.message).c_str());
99  }
```

qt-src/add-car-window.cpp

```
1  /**
2   * @file qt-src/add-car-window.cpp
3   * @brief Implementation of class AddCarWindow
4   *
5   * @details
6   * This file contains the implementation of the AddCarWindow class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
```

```
 */

#include "carinfo-manager/add-car-window.hpp"
#include <QFileDialog>
#include <QMessageBox>
#include <fstream>
#include <string>
#include "carinfo-manager/httphandler-client.hpp"

AddCarWindow::AddCarWindow(QWidget *parent,
                const std::string &ip,
                const int port,
                const Account &acc,
                QWidgetWithSubWindows *parentWindow)
  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
     acc(acc) {
  this->setWindowTitle("添加车辆");
  this->setFixedSize(320, 400);

  titleLabel = new QLabel("添加车辆", this);
  titleLabel->setGeometry(50, 50, 200, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
      this);
  usernameLabel->setGeometry(250, 20, 50, 30);
  usernameLabel->setAlignment(Qt::AlignCenter);
  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
  accounttypeLabel->setGeometry(250, 40, 50, 30);
  accounttypeLabel->setAlignment(Qt::AlignCenter);

  carIdLabel = new QLabel("车牌号: ", this);
  carIdLabel->setGeometry(50, 100, 70, 30);
  carIdLineEdit = new QLineEdit(this);
  carIdLineEdit->setGeometry(120, 100, 150, 30);

  carOwnerLabel = new QLabel("车主: ", this);
  carOwnerLabel->setGeometry(50, 140, 70, 30);
  carOwnerLineEdit = new QLineEdit(this);
  carOwnerLineEdit->setGeometry(120, 140, 150, 30);
```

```
49
50    carTypeLabel = new QLabel("车型: ", this);
51    carTypeLabel->setGeometry(50, 180, 70, 30);
52    carTypeLineEdit = new QLineEdit(this);
53    carTypeLineEdit->setGeometry(120, 180, 150, 30);
54
55    carColorLabel = new QLabel("颜色: ", this);
56    carColorLabel->setGeometry(50, 220, 70, 30);
57    carColorLineEdit = new QLineEdit(this);
58    carColorLineEdit->setGeometry(120, 220, 150, 30);
59
60    carYearLabel = new QLabel("年份: ", this);
61    carYearLabel->setGeometry(50, 260, 70, 30);
62    carYearLineEdit = new QLineEdit(this);
63    carYearLineEdit->setGeometry(120, 260, 150, 30);
64
65    carImgPathLabel = new QLabel("图片路径: ", this);
66    carImgPathLabel->setGeometry(50, 300, 70, 30);
67    carImgPathLineEdit = new QLineEdit(this);
68    carImgPathLineEdit->setGeometry(120, 300, 110, 30);
69    carImgSelectButton = new QPushButton("选择", this);
70    carImgSelectButton->setGeometry(230, 300, 40, 30);
71    connect(carImgSelectButton, &QPushButton::clicked, this, &
          AddCarWindow::getCarImagePath);
72
73    addCarButton = new QPushButton("添加", this);
74    addCarButton->setGeometry(110, 340, 100, 30);
75    connect(addCarButton, &QPushButton::clicked, this, &AddCarWindow::
          addCar);
76  }
77
78  AddCarWindow::~AddCarWindow() {
79    closeAllSubWindows();
80    removeFromParent();
81  }
82
83  void AddCarWindow::getCarImagePath() {
84    QString imgPath =
85      QFileDialog::getOpenFileName(this, "选择图
          片", "", "Images (*.png *.jpg *.jpeg)");
86    if (!imgPath.isEmpty())
```

```
87      carImgPathLineEdit->setText(imgPath);
88  }
89
90  void AddCarWindow::addCar() {
91    if (carIdLineEdit->text().isEmpty() || carOwnerLineEdit->text().
        isEmpty() ||
92      carTypeLineEdit->text().isEmpty() || carColorLineEdit->text().
          isEmpty() ||
93      carYearLineEdit->text().isEmpty() || carImgPathLineEdit->text().
          isEmpty()) {
94      QMessageBox::critical(this, "错误", "请填写完整信息");
95      return;
96    }
97    std::string carId = carIdLineEdit->text().toStdString(),
98          carOwner = carOwnerLineEdit->text().toStdString(),
99          carType = carTypeLineEdit->text().toStdString(),
100         carColor = carColorLineEdit->text().toStdString(),
101         carImgPath = carImgPathLineEdit->text().toLocal8Bit().data(),
              carImgType = "",
102         carImgData = "";
103   int carYear = carYearLineEdit->text().toInt();
104
105   if (carImgPath.find_last_of('.') != std::string::npos) {
106     carImgType = carImgPath.substr(carImgPath.find_last_of('.'));
107     if (carImgType != ".png" && carImgType != ".jpg" && carImgType != "
          .jpeg") {
108       QMessageBox::critical(this, "错误", "图片格式不支持");
109       return;
110     }
111   }
112   else {
113     QMessageBox::critical(this, "错误", "图片路径不合法");
114     return;
115   }
116
117   std::ifstream imgFile(carImgPath, std::ios::binary);
118   if (imgFile.is_open()) {
119     carImgData =
120       std::string(std::istreambuf_iterator<char>(imgFile), std::
            istreambuf_iterator<char>());
121     imgFile.close();
```

```
122   }
123   else {
124     QMessageBox::critical(this, "错误", "打开图片失败");
125     return;
126   }
127
128   auto res = ClientHttpHandler::handler_add_car(
129     ip, port, acc, carId, carOwner, carType, carColor, carYear,
          carImgData, carImgType);
130   if (res)
131     QMessageBox::information(this, "成功", "添加车辆成功");
132   else
133     QMessageBox::critical(
134       this, "错误", std::string("添加车辆失败
          n错误信息: " + res.message).c_str());
135 }
```

qt-src/change-password-window.cpp

```
1  /**
2   * @file qt-src/change-password-window.cpp
3   * @brief Implementation of class ChangePasswordWindow
4   *
5   * @details
6   * This file contains the implementation of the ChangePasswordWindow
        class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
10  */
11
12 #include "carinfo-manager/change-password-window.hpp"
13 #include "carinfo-manager/login-window.hpp"
14 #include <QMessageBox>
15 #include "carinfo-manager/accountpool.hpp"
16 #include "carinfo-manager/hash.hpp"
17 #include "carinfo-manager/httphandler-client.hpp"
18
19 ChangePasswordWindow::ChangePasswordWindow(QWidget *parent,
20                       const std::string &ip,
21                       const int port,
22                       const Account &acc,
23                       QWidgetWithSubWindows *parentWindow)
```

```
24  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
        acc(acc) {
25  this->setWindowTitle("修改密码");
26  this->setFixedSize(320, 320);
27
28  titleLabel = new QLabel("修改密码", this);
29  titleLabel->setGeometry(110, 50, 100, 30);
30  titleLabel->setAlignment(Qt::AlignCenter);
31  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
32
33  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
        this);
34  usernameLabel->setGeometry(250, 20, 50, 15);
35  usernameLabel->setAlignment(Qt::AlignCenter);
36  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
37  accounttypeLabel->setGeometry(250, 40, 50, 15);
38  accounttypeLabel->setAlignment(Qt::AlignCenter);
39
40  oldPasswordLabel = new QLabel("原密码: ", this);
41  oldPasswordLabel->setGeometry(50, 100, 60, 30);
42  oldPasswordEdit = new QLineEdit(this);
43  oldPasswordEdit->setEchoMode(QLineEdit::Password);
44  oldPasswordEdit->setGeometry(120, 100, 140, 30);
45
46  newPasswordLabel = new QLabel("新密码: ", this);
47  newPasswordLabel->setGeometry(50, 140, 60, 30);
48  newPasswordEdit = new QLineEdit(this);
49  newPasswordEdit->setEchoMode(QLineEdit::Password);
50  newPasswordEdit->setGeometry(120, 140, 140, 30);
51
52  confirmPasswordLabel = new QLabel("确认密码: ", this);
53  confirmPasswordLabel->setGeometry(50, 180, 60, 30);
54  confirmPasswordEdit = new QLineEdit(this);
55  confirmPasswordEdit->setEchoMode(QLineEdit::Password);
56  confirmPasswordEdit->setGeometry(120, 180, 140, 30);
57
58  changePasswordButton = new QPushButton("修改", this);
59  changePasswordButton->setGeometry(110, 230, 100, 30);
60  connect(this->changePasswordButton,
61      &QPushButton::clicked,
62      this,
```

```
63          &ChangePasswordWindow::changePassword);
64  }
65
66  ChangePasswordWindow::~ChangePasswordWindow() {
67    closeAllSubWindows();
68    removeFromParent();
69  }
70
71  void ChangePasswordWindow::changePassword() {
72    std::string oldPassword = oldPasswordEdit->text().toStdString();
73    std::string newPassword = newPasswordEdit->text().toStdString();
74    std::string confirmPassword = confirmPasswordEdit->text().toStdString
          ();
75
76    if (newPassword != confirmPassword) {
77      QMessageBox::warning(this, "错误", "两次输入的密码不一致");
78      return;
79    }
80
81    Hash oldPasswordHash(oldPassword, "Carinfo-Manager");
82    Hash newPasswordHash(newPassword, "Carinfo-Manager");
83    Account testAcc(acc.getUsername(), oldPasswordHash.getHash(), acc.
          getAccountType());
84
85    auto res = ClientHttpHandler::handler_change_password(ip, port,
          testAcc, newPasswordHash.getHash());
86    if (res) {
87      QMessageBox::information(this, "成功", "修改密码成功, 请重新登录");
88      removeFromParent();
89      LoginWindow *loginWindow = new LoginWindow(nullptr, ip, port);
90      loginWindow->show();
91      parentWindow->close();
92      this->close();
93    }
94    else
95      QMessageBox::critical(
96        this, "错误", std::string("修改密码失败
              n错误信息: " + res.message).c_str());
97  }
```

qt-src/client-window.cpp

```
1  /**
```

```
 * @file qt-src/client-window.cpp
 * @brief Implementation of class ClientMain
 *
 * @details
 * This file contains the implementation of the ClientMain class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/client-window.hpp"
#include <locale>
#include "carinfo-manager/select-server-window.hpp"

ClientMain::ClientMain(int argc, char *argv[]) : QApplication(argc,
    argv) {}

ClientMain::~ClientMain() {}

int ClientMain::run() {
  SelectServerWindow *selectServerWindow = new SelectServerWindow();
  selectServerWindow->show();
  return exec();
}
```

qt-src/display-account-window.cpp

```
/**
 * @file qt-src/display-accountinfo-window.cpp
 * @brief Implementation of class DisplayAccountinfoWindow
 *
 * @details
 * This file contains the implementation of the
     DisplayAccountinfoWindow class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include <QFont>
#include <QString>
#include "carinfo-manager/display-accountinfo-window.hpp"
```

```
15 #include "carinfo-manager/httphandler-client.hpp"
16
17 DisplayAccountinfoWindow::DisplayAccountinfoWindow(QWidget *parent,
18                             const AccountPool &accountpool,
19                             const std::string searchCarId,
20                             const std::string &ip,
21                             const int port,
22                             const Account &acc,
23                             QWidgetWithSubWindows *parentWindow)
24   : QWidgetWithSubWindows(parent, parentWindow),
25     ip(ip),
26     port(port),
27     acc(acc),
28     accountpool(accountpool),
29     searchCarId(searchCarId) {
30   this->setParent(parent);
31   this->setWindowTitle("账户信息");
32   this->setFixedSize(300, 300);
33
34   outerLayout = new QVBoxLayout(this);
35   outerLayout->setContentsMargins(0, 0, 0, 0);
36
37   accountinfoScrollArea = new QScrollArea(this);
38   accountinfoScrollArea->setWidgetResizable(true);
39   outerLayout->addWidget(accountinfoScrollArea);
40
41   accountinfoWidget = new QWidget(this);
42   accountinfoScrollArea->setWidget(accountinfoWidget);
43
44   innerLayout = new QVBoxLayout(accountinfoWidget);
45
46   accountQueryLabel =
47     new QLabel(std::string("查询条件:
       \u3000" + searchCarId).c_str(), accountinfoWidget);
48   accountQueryLabel->setAlignment(Qt::AlignCenter);
49   QFont font = accountQueryLabel->font();
50   font.setBold(true);
51   accountQueryLabel->setFont(font);
52   innerLayout->addWidget(accountQueryLabel, 0, Qt::AlignHCenter | Qt::
       AlignTop);
53
```

```
54  QLabel *placeholderLabel = new QLabel(this);
55  placeholderLabel->setFixedSize(300, 10);
56  innerLayout->addWidget(placeholderLabel, 0, Qt::AlignHCenter | Qt::
        AlignTop);
57
58  auto acc_list = accountpool.list();
59  for (auto it = acc_list.begin(); it != acc_list.end(); ++it) {
60    Account account = *it;
61
62    QWidget *itemWidget = new QWidget(accountinfoWidget);
63    QVBoxLayout *itemLayout = new QVBoxLayout(itemWidget);
64    itemLayout->setContentsMargins(5, 5, 5, 5);
65    itemLayout->setSpacing(10);
66
67    QLabel *itemUsernameLabel = new QLabel(
68           QString::fromStdString("账号:
                \u3000" + account.getUsername()), itemWidget),
69         *itemPasswdHashTitleLabel = new QLabel("密码哈希: ", itemWidget),
70         *itemPasswdHashLabel_1 = new QLabel(
71           QString::fromStdString(account.getPasswdHash().substr(0, 16)
                ), itemWidget),
72         *itemPasswdHashLabel_2 = new QLabel(
73           QString::fromStdString(account.getPasswdHash().substr(16,
                16)), itemWidget),
74         *itemPasswdHashLabel_3 = new QLabel(
75           QString::fromStdString(account.getPasswdHash().substr(32,
                16)), itemWidget),
76         *itemPasswdHashLabel_4 = new QLabel(
77           QString::fromStdString(account.getPasswdHash().substr(48,
                16)), itemWidget),
78         *itemAccountTypeLabel = new QLabel(
79           QString::fromStdString("账户类型: \u3000" +
80                      std::string((account.isAdmin() ? "管理员" : "普通用
                          户"))),
81           itemWidget);
82    itemUsernameLabel->setAlignment(Qt::AlignCenter);
83    itemPasswdHashTitleLabel->setAlignment(Qt::AlignCenter);
84    itemPasswdHashLabel_1->setAlignment(Qt::AlignCenter);
85    itemPasswdHashLabel_2->setAlignment(Qt::AlignCenter);
86    itemPasswdHashLabel_3->setAlignment(Qt::AlignCenter);
87    itemPasswdHashLabel_4->setAlignment(Qt::AlignCenter);
88    itemAccountTypeLabel->setAlignment(Qt::AlignCenter);
```

```
89      itemUsernameLabel->resize(250, 30);
90      itemPasswdHashTitleLabel->resize(250, 30);
91      itemPasswdHashLabel_1->resize(250, 30);
92      itemPasswdHashLabel_2->resize(250, 30);
93      itemPasswdHashLabel_3->resize(250, 30);
94      itemPasswdHashLabel_4->resize(250, 30);
95      itemAccountTypeLabel->resize(250, 30);
96      itemLayout->addWidget(itemUsernameLabel, 0, Qt::AlignHCenter);
97      itemLayout->addWidget(itemPasswdHashTitleLabel, 0, Qt::AlignHCenter
            );
98      itemLayout->addWidget(itemPasswdHashLabel_1, 0, Qt::AlignHCenter);
99      itemLayout->addWidget(itemPasswdHashLabel_2, 0, Qt::AlignHCenter);
100     itemLayout->addWidget(itemPasswdHashLabel_3, 0, Qt::AlignHCenter);
101     itemLayout->addWidget(itemPasswdHashLabel_4, 0, Qt::AlignHCenter);
102     itemLayout->addWidget(itemAccountTypeLabel, 0, Qt::AlignHCenter);
103
104     innerLayout->addWidget(itemWidget, 0, Qt::AlignHCenter | Qt::
            AlignTop);
105
106     if (it != acc_list.end() - 1) {
107       QFrame *line = new QFrame(accountinfoWidget);
108       line->setFrameShape(QFrame::HLine);
109       line->setFrameShadow(QFrame::Sunken);
110       innerLayout->addWidget(line);
111     }
112   }
113 }
114
115 DisplayAccountinfoWindow::~DisplayAccountinfoWindow() {
116   closeAllSubWindows();
117   removeFromParent();
118 }
```

qt-src/display-carinfo-window.cpp

```
1  /**
2   * @file qt-src/display-carinfo-window.cpp
3   * @brief Implementation of class DisplayCarInfoWindow
4   *
5   * @details
6   * This file contains the implementation of the DisplayCarInfoWindow
        class.
```

```
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/display-carinfo-window.hpp"
#include <QByteArray>
#include <QImage>
#include <QMessageBox>
#include <QPixmap>
#include <QString>
#include <sstream>
#include <string>
#include "carinfo-manager/httphandler-client.hpp"
#include "json/json.hpp"

SearchCarQuery::SearchCarQuery(const std::string &searchCarId,
                               const std::string &searchCarOwner,
                               const std::string &searchCarType,
                               const std::string &searchCarColor) {
  this->searchCarId = searchCarId;
  this->searchCarOwner = searchCarOwner;
  this->searchCarType = searchCarType;
  this->searchCarColor = searchCarColor;
}

SearchCarQuery::SearchCarQuery(const SearchCarQuery &query) {
  this->searchCarId = query.searchCarId;
  this->searchCarOwner = query.searchCarOwner;
  this->searchCarType = query.searchCarType;
  this->searchCarColor = query.searchCarColor;
}

SearchCarQuery::~SearchCarQuery() {}

DisplayCarInfoWindow::DisplayCarInfoWindow(QWidget *parent,
                                           const CarPool &carpool,
                                           const SearchCarQuery &query,
                                           const std::string &ip,
                                           const int port,
                                           const Account &acc,
```

```
48                          QWidgetWithSubWindows *parentWindow)
49  : QWidgetWithSubWindows(parent, parentWindow),
50    SearchCarQuery(query),
51    ip(ip),
52    port(port),
53    acc(acc),
54    carpool(carpool) {
55  this->setParent(parent);
56  this->setWindowTitle("车辆信息");
57  this->setFixedSize(800, 600);
58
59  outerLayout = new QVBoxLayout(this);
60  outerLayout->setContentsMargins(0, 0, 0, 0);
61
62  carinfoScrollArea = new QScrollArea(this);
63  carinfoScrollArea->setSizePolicy(QSizePolicy(QSizePolicy::Expanding,
       QSizePolicy::Expanding));
64  carinfoScrollArea->setWidgetResizable(true);
65  outerLayout->addWidget(carinfoScrollArea);
66
67  carinfoWidget = new QWidget(this);
68  carinfoScrollArea->setWidget(carinfoWidget);
69
70  innerLayout = new QVBoxLayout(carinfoWidget);
71  innerLayout->addSpacing(10);
72
73  carQueryTitleLabel = new QLabel(QString::fromStdString("查询条件:
       "), carinfoWidget);
74  QFont font = carQueryTitleLabel->font();
75  font.setBold(true);
76  carQueryTitleLabel->setFont(font);
77  carQueryTitleLabel->setAlignment(Qt::AlignCenter);
78  innerLayout->addWidget(carQueryTitleLabel);
79
80  if (!searchCarId.empty()) {
81    carQueryIdLabel =
82      new QLabel(QString::fromStdString("车牌号:
           \u3000" + searchCarId), carinfoWidget);
83    carQueryIdLabel->setAlignment(Qt::AlignCenter);
84    carQueryIdLabel->setFont(font);
85    innerLayout->addWidget(carQueryIdLabel);
86  }
```

```
87  if (!searchCarOwner.empty()) {
88    carQueryOwnerLabel =
89      new QLabel(QString::fromStdString("车主:
          \u3000" + searchCarOwner), carinfoWidget);
90    carQueryOwnerLabel->setAlignment(Qt::AlignCenter);
91    carQueryOwnerLabel->setFont(font);
92    innerLayout->addWidget(carQueryOwnerLabel);
93  }
94  if (!searchCarType.empty()) {
95    carQueryTypeLabel =
96      new QLabel(QString::fromStdString("车型:
          \u3000" + searchCarType), carinfoWidget);
97    carQueryTypeLabel->setAlignment(Qt::AlignCenter);
98    carQueryTypeLabel->setFont(font);
99    innerLayout->addWidget(carQueryTypeLabel);
100 }
101 if (!searchCarColor.empty()) {
102   carQueryColorLabel =
103     new QLabel(QString::fromStdString("颜色:
          \u3000" + searchCarColor), carinfoWidget);
104   carQueryColorLabel->setAlignment(Qt::AlignCenter);
105   carQueryColorLabel->setFont(font);
106   innerLayout->addWidget(carQueryColorLabel);
107 }
108
109 QLabel *placeholderLabel = new QLabel("", carinfoWidget);
110 placeholderLabel->setFixedSize(800, 10);
111 innerLayout->addWidget(placeholderLabel);
112
113 auto car_list = carpool.list();
114 for (auto it = car_list.begin(); it != car_list.end(); ++it) {
115   Car car = *it;
116
117   QWidget *itemWidget = new QWidget(carinfoWidget);
118   QHBoxLayout *itemLayout = new QHBoxLayout(itemWidget);
119   itemLayout->setContentsMargins(5, 5, 5, 5);
120
121   QWidget *itemTextWidget = new QWidget(itemWidget), *itemPicWidget =
        new QWidget(itemWidget);
122   itemLayout->addWidget(itemTextWidget);
123   itemLayout->addWidget(itemPicWidget);
124
```

```
125     QVBoxLayout *itemTextLayout = new QVBoxLayout(itemTextWidget),
126           *itemPicLayout = new QVBoxLayout(itemPicWidget);
127     itemTextLayout->setSpacing(10);
128     itemPicLayout->setContentsMargins(0, 0, 0, 0);
129
130     QLabel *itemIdLabel = new QLabel(QString::fromStdString("车牌号:
            \u3000" + car.getId()),
131                         itemTextWidget),
132          *itemOwnerLabel = new QLabel(QString::fromStdString("车主:
                \u3000" + car.getOwner()),
133                          itemTextWidget),
134          *itemTypeLabel = new QLabel(QString::fromStdString("车型:
                \u3000" + car.getType()),
135                           itemTextWidget),
136          *itemColorLabel = new QLabel(QString::fromStdString("颜色:
                \u3000" + car.getColor()),
137                          itemTextWidget),
138          *itemPicLabel = new QLabel(itemPicWidget);
139     itemIdLabel->setAlignment(Qt::AlignCenter);
140     itemColorLabel->setAlignment(Qt::AlignCenter);
141     itemTypeLabel->setAlignment(Qt::AlignCenter);
142     itemOwnerLabel->setAlignment(Qt::AlignCenter);
143     itemIdLabel->resize(350, 30);
144     itemColorLabel->resize(350, 30);
145     itemTypeLabel->resize(350, 30);
146     itemOwnerLabel->resize(350, 30);
147     itemPicLabel->resize(360, 280);
148
149     auto imgRes = ClientHttpHandler::handler_get_carimg(ip, port, acc,
            car.getImagePath());
150     if (!imgRes) {
151       QMessageBox::critical(this,
152                   "错误",
153                   std::string("获取车辆图片失败
                        n车牌号: " + car.getId() +
154                            "
                                n错误信息: " + imgRes.message)
155                       .c_str());
156       continue;
157     }
158     QByteArray imgData = QByteArray::fromStdString(imgRes.message);
159     QPixmap img = QPixmap::fromImage(QImage::fromData(imgData));
160
```

```
161     itemPicLabel->setPixmap(img.scaled(360, 280, Qt::KeepAspectRatio));
162     itemPicLabel->setAlignment(Qt::AlignCenter);
163     itemTextLayout->addStretch();
164     itemTextLayout->addWidget(itemIdLabel, 0, Qt::AlignCenter);
165     itemTextLayout->addWidget(itemColorLabel, 0, Qt::AlignCenter);
166     itemTextLayout->addWidget(itemTypeLabel, 0, Qt::AlignCenter);
167     itemTextLayout->addWidget(itemOwnerLabel, 0, Qt::AlignCenter);
168     itemTextLayout->addStretch();
169     itemPicLayout->addWidget(itemPicLabel);
170     innerLayout->addWidget(itemWidget);
171
172     if (it != car_list.end() - 1) {
173       QFrame *line = new QFrame(carinfoWidget);
174       line->setFrameShape(QFrame::HLine);
175       line->setFrameShadow(QFrame::Sunken);
176       innerLayout->addWidget(line);
177     }
178   }
179 }
180
181 DisplayCarInfoWindow::~DisplayCarInfoWindow() {
182   closeAllSubWindows();
183   removeFromParent();
184 }
```

qt-src/login-window.cpp

```
1  /**
2   * @file qt-src/login-window.cpp
3   * @brief Implementation of class LoginWindow
4   *
5   * @details
6   * This file contains the implementation of the LoginWindow class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
10  */
11
12 #include "carinfo-manager/login-window.hpp"
13 #include "carinfo-manager/accountpool.hpp"
14 #include "carinfo-manager/hash.hpp"
15 #include "carinfo-manager/httphandler-client.hpp"
```

```cpp
#include "carinfo-manager/manage-window.hpp"
#include "carinfo-manager/search-car-window.hpp"
#include "carinfo-manager/select-server-window.hpp"

LoginWindow::LoginWindow(QWidget *parent, const std::string &ip, const int port)
  : QWidget(parent), ip(ip), port(port) {
  this->setWindowTitle("登录");
  this->setFixedSize(400, 300);

  titleLabel = new QLabel("车辆管理系统", this);
  titleLabel->setGeometry(100, 50, 200, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  usernameLabel = new QLabel("用户名: ", this);
  usernameLabel->setGeometry(50, 100, 100, 30);
  usernameLabel->setAlignment(Qt::AlignCenter);
  usernameEdit = new QLineEdit(this);
  usernameEdit->setGeometry(150, 100, 200, 30);
  usernameEdit->setPlaceholderText("请输入用户名");

  passwordLabel = new QLabel("密码: ", this);
  passwordLabel->setGeometry(50, 150, 100, 30);
  passwordLabel->setAlignment(Qt::AlignCenter);
  passwordEdit = new QLineEdit(this);
  passwordEdit->setGeometry(150, 150, 200, 30);
  passwordEdit->setEchoMode(QLineEdit::Password);
  passwordEdit->setPlaceholderText("请输入密码");

  loginButton = new QPushButton("登录", this);
  loginButton->setGeometry(200, 200, 100, 30);
  connect(loginButton, &QPushButton::clicked, this, &LoginWindow::login);
  returnButton = new QPushButton("返回", this);
  returnButton->setGeometry(100, 200, 100, 30);
  connect(returnButton, &QPushButton::clicked, this, &LoginWindow::returnToSelectServer);
}

LoginWindow::~LoginWindow() {}
```

```
54
55 void LoginWindow::login() {
56   if (usernameEdit->text().isEmpty() || passwordEdit->text().isEmpty())
         {
57     QMessageBox::critical(this, "错误", "用户名或密码不能为空");
58     return;
59   }
60   try {
61     username = usernameEdit->text().toStdString();
62     password = passwordEdit->text().toStdString();
63   }
64   catch (std::exception &e) {
65     QMessageBox::critical(this, "错误", "不合法的用户名或密码");
66     return;
67   }
68   Hash passwordHash(password, "Carinfo-Manager");
69   Account acc(username, passwordHash.getHash(), Account::AccountType::
        NONETYPE);
70   ClientHttpHandler::AccountVerifyResult result = ClientHttpHandler::
        handler_login(ip, port, acc);
71   if (result == ClientHttpHandler::AccountVerifyResult::SUCCESS) {
72     QMessageBox::information(this, "成功", "登录成功");
73     if (acc.getAccountType() == Account::AccountType::ADMIN ||
74       acc.getAccountType() == Account::AccountType::USER) {
75       ManageWindow *manageWindow = new ManageWindow(nullptr, ip, port,
            acc);
76       this->close();
77       manageWindow->show();
78       return;
79     }
80     else
81       QMessageBox::critical(this, "错误", "未知账户类型");
82   }
83   else if (result ==
84        ClientHttpHandler::AccountVerifyResult::
           ACCOUNT_NOT_FOUND_OR_WRONG_PASSWORD) {
85     QMessageBox::critical(this, "错误", "用户名或密码错误");
86   }
87   else if (result == ClientHttpHandler::AccountVerifyResult::
        NETWORK_ERROR) {
88     QMessageBox::critical(this, "错误", "无法连接到服务器");
```

```
89    }
90    else if (result == ClientHttpHandler::AccountVerifyResult::
          BAD_REQUEST) {
91      QMessageBox::critical(this, "错误", "请求错误");
92    }
93    else {
94      QMessageBox::critical(this, "错误", "未知错误");
95    }
96  }
97
98  void LoginWindow::returnToSelectServer() {
99    SelectServerWindow *selectServerWindow = new SelectServerWindow;
100   this->close();
101   selectServerWindow->show();
102 }
```

qt-src/manage-account-window.cpp

```
1   /**
2    * @file qt-src/manage-account-window.cpp
3    * @brief Implementation of class ManageAccountWindow
4    *
5    * @details
6    * This file contains the implementation of the ManageAccountWindow
         class.
7    *
8    * @author donghy23@mails.tsinghua.edu.cn
9    * @version 1.0
10   */
11
12  #include "carinfo-manager/manage-account-window.hpp"
13  #include "carinfo-manager/add-account-window.hpp"
14  #include "carinfo-manager/log.hpp"
15  #include "carinfo-manager/remove-account-window.hpp"
16  #include "carinfo-manager/update-account-window.hpp"
17
18  ManageAccountWindow::ManageAccountWindow(QWidget *parent,
19                       const std::string &ip,
20                       const int port,
21                       const Account &acc,
22                       QWidgetWithSubWindows *parentWindow)
23    : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
```

```
    acc(acc) {
24  this->setWindowTitle("账户管理");
25  this->setFixedSize(300, 330);
26
27  titleLabel = new QLabel("账户管理", this);
28  titleLabel->setGeometry(50, 50, 200, 30);
29  titleLabel->setAlignment(Qt::AlignCenter);
30  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
31
32  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
        this);
33  usernameLabel->setGeometry(240, 20, 50, 15);
34  usernameLabel->setAlignment(Qt::AlignCenter);
35  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
36  accounttypeLabel->setGeometry(240, 40, 50, 15);
37  accounttypeLabel->setAlignment(Qt::AlignCenter);
38
39  addAccountButton = new QPushButton("添加账户", this);
40  addAccountButton->setGeometry(50, 100, 200, 30);
41  connect(addAccountButton, &QPushButton::clicked, this, &
       ManageAccountWindow::addAccount);
42
43  updateAccountButton = new QPushButton("修改账户", this);
44  updateAccountButton->setGeometry(50, 140, 200, 30);
45  connect(updateAccountButton, &QPushButton::clicked, this, &
       ManageAccountWindow::updateAccount);
46
47  removeAccountButton = new QPushButton("删除账户", this);
48  removeAccountButton->setGeometry(50, 180, 200, 30);
49  connect(removeAccountButton, &QPushButton::clicked, this, &
       ManageAccountWindow::removeAccount);
50
51
52 }
53
54 ManageAccountWindow::~ManageAccountWindow() {
55  closeAllSubWindows();
56  removeFromParent();
57 }
58
59 void ManageAccountWindow::addAccount() {
```

```
60    AddAccountWindow *addAccountWindow = new AddAccountWindow(nullptr, ip
          , port, acc, this);
61    addSubWindow(addAccountWindow);
62    addAccountWindow->show();
63  }
64
65  void ManageAccountWindow::removeAccount() {
66    RemoveAccountWindow *removeAccountWindow =
67      new RemoveAccountWindow(nullptr, ip, port, acc, this);
68    addSubWindow(removeAccountWindow);
69    removeAccountWindow->show();
70  }
71
72  void ManageAccountWindow::updateAccount() {
73    UpdateAccountWindow *updateAccountWindow =
74      new UpdateAccountWindow(nullptr, ip, port, acc, this);
75    addSubWindow(updateAccountWindow);
76    updateAccountWindow->show();
77  }
```

qt-src/manage-car-window.cpp

```
1   /**
2    * @file qt-src/manage-car-window.cpp
3    * @brief Implementation of class ManageCarWindow
4    *
5    * @details
6    * This file contains the implementation of the ManageCarWindow class.
7    *
8    * @author donghy23@mails.tsinghua.edu.cn
9    * @version 1.0
10   */
11
12  #include "carinfo-manager/manage-car-window.hpp"
13  #include "carinfo-manager/add-car-window.hpp"
14  #include "carinfo-manager/remove-car-window.hpp"
15  #include "carinfo-manager/update-car-window.hpp"
16
17  ManageCarWindow::ManageCarWindow(QWidget *parent,
18                   const std::string &ip,
19                   const int port,
20                   const Account &acc,
```

```
21                  QWidgetWithSubWindows *parentWindow)
22   : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
        acc(acc) {
23   this->setWindowTitle("车辆管理");
24   this->setFixedSize(300, 330);
25
26   titleLabel = new QLabel("车辆管理", this);
27   titleLabel->setGeometry(50, 50, 200, 30);
28   titleLabel->setAlignment(Qt::AlignCenter);
29   titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
30
31   usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
         this);
32   usernameLabel->setGeometry(240, 20, 50, 15);
33   usernameLabel->setAlignment(Qt::AlignCenter);
34   accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
35   accounttypeLabel->setGeometry(240, 40, 50, 15);
36   accounttypeLabel->setAlignment(Qt::AlignCenter);
37
38   addCarButton = new QPushButton("添加车辆", this);
39   addCarButton->setGeometry(50, 100, 200, 30);
40   connect(addCarButton, &QPushButton::clicked, this, &ManageCarWindow::
        addCar);
41
42   updateCarButton = new QPushButton("修改车辆", this);
43   updateCarButton->setGeometry(50, 140, 200, 30);
44   connect(updateCarButton, &QPushButton::clicked, this, &
        ManageCarWindow::updateCar);
45
46   removeCarButton = new QPushButton("删除车辆", this);
47   removeCarButton->setGeometry(50, 180, 200, 30);
48   connect(removeCarButton, &QPushButton::clicked, this, &
        ManageCarWindow::removeCar);
49 }
50
51 ManageCarWindow::~ManageCarWindow() {
52   closeAllSubWindows();
53   removeFromParent();
54 }
55
56 void ManageCarWindow::addCar() {
```

```
57    AddCarWindow *addCarWindow = new AddCarWindow(nullptr, ip, port, acc,
          this);
58    addSubWindow(addCarWindow);
59    addCarWindow->show();
60  }
61
62  void ManageCarWindow::removeCar() {
63    RemoveCarWindow *removeCarWindow = new RemoveCarWindow(nullptr, ip,
          port, acc, this);
64    addSubWindow(removeCarWindow);
65    removeCarWindow->show();
66  }
67
68  void ManageCarWindow::updateCar() {
69    UpdateCarWindow *updateCarWindow = new UpdateCarWindow(nullptr, ip,
          port, acc, this);
70    addSubWindow(updateCarWindow);
71    updateCarWindow->show();
72  }
```

qt-src/manage-window.cpp

```
1  /**
2   * @file qt-src/manage-window.cpp
3   * @brief Implementation of class ManageWindow
4   *
5   * @details
6   * This file contains the implementation of the ManageWindow class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
10  */
11
12 #include "carinfo-manager/manage-window.hpp"
13 #include "carinfo-manager/accountpool.hpp"
14 #include "carinfo-manager/login-window.hpp"
15 #include "carinfo-manager/manage-account-window.hpp"
16 #include "carinfo-manager/manage-car-window.hpp"
17 #include "carinfo-manager/search-account-window.hpp"
18 #include "carinfo-manager/search-car-window.hpp"
19 #include "carinfo-manager/change-password-window.hpp"
20
```

```
21 ManageWindow::ManageWindow(QWidget *parent,
22                 const std::string &ip,
23                 const int port,
24                 const Account &acc)
25   : QWidgetWithSubWindows(parent), ip(ip), port(port), acc(acc) {
26   this->setWindowTitle(acc.isAdmin() ? "管理员管理" : "用户查询");
27   this->setFixedSize(300, 330);
28
29   titleLabel = new QLabel(acc.isAdmin() ? "管理员管理" : "用户查询", this);
30   titleLabel->setGeometry(50, 50, 200, 30);
31   titleLabel->setAlignment(Qt::AlignCenter);
32   titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
33
34   usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
         this);
35   usernameLabel->setGeometry(240, 20, 50, 15);
36   usernameLabel->setAlignment(Qt::AlignCenter);
37   accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
38   accounttypeLabel->setGeometry(240, 40, 50, 15);
39   accounttypeLabel->setAlignment(Qt::AlignCenter);
40
41   if (acc.isAdmin()) {
42     carManageButton = new QPushButton("车辆管理", this);
43     carManageButton->setGeometry(50, 100, 200, 30);
44     connect(carManageButton, &QPushButton::clicked, this, &ManageWindow
          ::carManage);
45
46     accountManageButton = new QPushButton("账户管理", this);
47     accountManageButton->setGeometry(50, 140, 200, 30);
48     connect(accountManageButton, &QPushButton::clicked, this, &
          ManageWindow::accountManage);
49
50     carSearchButton = new QPushButton("查询车辆", this);
51     carSearchButton->setGeometry(50, 180, 200, 30);
52     connect(carSearchButton, &QPushButton::clicked, this, &ManageWindow
          ::carSearch);
53
54     accountSearchButton = new QPushButton("查询账户", this);
55     accountSearchButton->setGeometry(50, 220, 200, 30);
56     connect(accountSearchButton, &QPushButton::clicked, this, &
          ManageWindow::accountSearch);
```

```
57
58      changePasswordButton = new QPushButton("修改密码", this);
59      changePasswordButton->setGeometry(50, 260, 95, 30);
60      connect(changePasswordButton, &QPushButton::clicked, this, &
            ManageWindow::changePassword);
61
62      logoutButton = new QPushButton("注销", this);
63      logoutButton->setGeometry(155, 260, 95, 30);
64      connect(logoutButton, &QPushButton::clicked, this, &ManageWindow::
            logout);
65    }
66    else {
67      carSearchButton = new QPushButton("查询车辆", this);
68      carSearchButton->setGeometry(50, 140, 200, 30);
69      connect(carSearchButton, &QPushButton::clicked, this, &ManageWindow
            ::carSearch);
70
71      changePasswordButton = new QPushButton("修改密码", this);
72      changePasswordButton->setGeometry(50, 260, 95, 30);
73      connect(changePasswordButton, &QPushButton::clicked, this, &
            ManageWindow::changePassword);
74
75      logoutButton = new QPushButton("注销", this);
76      logoutButton->setGeometry(155, 260, 95, 30);
77      connect(logoutButton, &QPushButton::clicked, this, &ManageWindow::
            logout);
78    }
79  }
80
81  ManageWindow::~ManageWindow() {
82    closeAllSubWindows();
83  }
84
85  void ManageWindow::accountManage() {
86    ManageAccountWindow *manageAccountWindow =
87      new ManageAccountWindow(nullptr, ip, port, acc, this);
88    addSubWindow(manageAccountWindow);
89    manageAccountWindow->show();
90  }
91
92  void ManageWindow::carManage() {
```

```
93    ManageCarWindow *manageCarWindow = new ManageCarWindow(nullptr, ip,
          port, acc, this);
94    addSubWindow(manageCarWindow);
95    manageCarWindow->show();
96  }
97
98  void ManageWindow::accountSearch() {
99    SearchAccountWindow *searchAccountWindow =
100     new SearchAccountWindow(nullptr, ip, port, acc, this);
101   addSubWindow(searchAccountWindow);
102   searchAccountWindow->show();
103 }
104
105 void ManageWindow::carSearch() {
106   SearchCarWindow *searchCarWindow = new SearchCarWindow(nullptr, ip,
          port, acc, this);
107   addSubWindow(searchCarWindow);
108   searchCarWindow->show();
109 }
110
111 void ManageWindow::changePassword() {
112   ChangePasswordWindow *changePasswordWindow =
113     new ChangePasswordWindow(nullptr, ip, port, acc, this);
114   addSubWindow(changePasswordWindow);
115   changePasswordWindow->show();
116 }
117
118 void ManageWindow::logout() {
119   LoginWindow *loginWindow = new LoginWindow(nullptr, ip, port);
120   loginWindow->show();
121   closeAllSubWindows();
122   this->close();
123 }
```

qt-src/qwidget-with-subwindows.cpp

```
1  /**
2   * @file qt-src/qwidget-with-subwindows.cpp
3   * @brief Implementation of class QWidgetWith
4   *
5   * @details
6   * This file contains the implementation of the QWidgetWithSubWindows
```

```
     class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/qwidget-with-subwindows.hpp"
#include <QCloseEvent>

QWidgetWithSubWindows::QWidgetWithSubWindows(QWidget *parent,
                       QWidgetWithSubWindows *parentWindow,
                       const std::set<QWidgetWithSubWindows *> &
                          subWindows)
  : QWidget(parent), parentWindow(parentWindow), subWindows(subWindows)
      {}

void QWidgetWithSubWindows::addSubWindow(QWidgetWithSubWindows *window)
    {
  if (subWindows.find(window) == subWindows.end())
    subWindows.insert(window);
}

void QWidgetWithSubWindows::removeSubWindow(QWidgetWithSubWindows *
   window) {
  if (subWindows.find(window) != subWindows.end())
    subWindows.erase(window);
}

void QWidgetWithSubWindows::closeSubWindow(QWidgetWithSubWindows *
   window) {
  if (subWindows.find(window) != subWindows.end()) {
    window->setCloseFromParent(true);
    window->close();
    delete window;
    subWindows.erase(window);
  }
}

void QWidgetWithSubWindows::closeAllSubWindows() {
  for (QWidgetWithSubWindows *window : subWindows) {
    window->setCloseFromParent(true);
```

```
42      window->close();
43      delete window;
44    }
45    subWindows.clear();
46  }
47
48  void QWidgetWithSubWindows::removeFromParent() {
49    if (parentWindow != nullptr)
50      parentWindow->removeSubWindow(this);
51  }
52
53  QWidgetWithSubWindows::~QWidgetWithSubWindows() {
54    closeAllSubWindows();
55  }
56
57  void QWidgetWithSubWindows::setCloseFromParent(const bool
        closeFromParent) {
58    this->closeFromParent = closeFromParent;
59  }
60
61  bool QWidgetWithSubWindows::getCloseFromParent() const {
62    return closeFromParent;
63  }
64
65  void QWidgetWithSubWindows::closeEvent(QCloseEvent *event) {
66    if (!closeFromParent)
67      removeFromParent();
68    closeAllSubWindows();
69    event->accept();
70  }
```

qt-src/remove-account-window.cpp

```
1  /**
2   * @file qt-src/remove-account-window.cpp
3   * @brief Implementation of class RemoveAccountWindow
4   *
5   * @details
6   * This file contains the implementation of the RemoveAccountWindow
        class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
```

```
 * @version 1.0
 */

#include "carinfo-manager/remove-account-window.hpp"
#include <QMessageBox>
#include "carinfo-manager/httphandler-client.hpp"
#include "carinfo-manager/log.hpp"

RemoveAccountWindow::RemoveAccountWindow(QWidget *parent,
                     const std::string &ip,
                     const int port,
                     const Account &acc,
                     QWidgetWithSubWindows *parentWindow)
  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
     acc(acc) {
  this->setWindowTitle("删除账户");
  this->setFixedSize(300, 300);

  titleLabel = new QLabel("删除账户", this);
  titleLabel->setGeometry(50, 50, 200, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
      this);
  usernameLabel->setGeometry(240, 20, 50, 30);
  usernameLabel->setAlignment(Qt::AlignCenter);
  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
  accounttypeLabel->setGeometry(240, 40, 50, 30);
  accounttypeLabel->setAlignment(Qt::AlignCenter);

  targetUsernameLabel = new QLabel("目标用户名: ", this);
  targetUsernameLabel->setGeometry(50, 100, 70, 30);
  targetUsernameEdit = new QLineEdit(this);
  targetUsernameEdit->setGeometry(130, 100, 120, 30);

  removeButton = new QPushButton("删除", this);
  removeButton->setGeometry(100, 150, 100, 30);
  connect(removeButton, &QPushButton::clicked, this, &
     RemoveAccountWindow::removeAccount);
}
```

```
47
48 RemoveAccountWindow::~RemoveAccountWindow() {
49   closeAllSubWindows();
50   removeFromParent();
51 }
52
53 void RemoveAccountWindow::removeAccount() {
54   std::string target_username = this->acc.getUsername();
55   if (target_username.empty()) {
56     QMessageBox::warning(this, "错误", "用户名不能为空");
57     return;
58   }
59   if (target_username == this->acc.getUsername()) {
60     QMessageBox::warning(this, "错误", "不能删除自己");
61     return;
62   }
63   auto res = ClientHttpHandler::handler_remove_account(
64     this->ip, this->port, this->acc, target_username);
65   if (res)
66     QMessageBox::information(this, "成功", "删除账户成功");
67   else
68     QMessageBox::critical(
69       this, "错误", std::string("删除账户失败
          n错误信息: " + res.message).c_str());
70 }
```

qt-src/remove-car-window.cpp

```
1  /**
2   * @file qt-src/remove-car-window.cpp
3   * @brief Implementation of class RemoveCarWindow
4   *
5   * @details
6   * This file contains the implementation of the RemoveCarWindow class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
10  */
11
12 #include "carinfo-manager/remove-car-window.hpp"
13 #include <QMessageBox>
14
15 RemoveCarWindow::RemoveCarWindow(QWidget *parent,
```

```
                    const std::string &ip,
                    const int port,
                    const Account &acc,
                    QWidgetWithSubWindows *parentWindow)
    : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
        acc(acc) {
    this->setWindowTitle("删除车辆");
    this->setFixedSize(300, 300);

    titleLabel = new QLabel("删除车辆", this);
    titleLabel->setGeometry(50, 50, 200, 30);
    titleLabel->setAlignment(Qt::AlignCenter);
    titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

    usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
        this);
    usernameLabel->setGeometry(240, 20, 50, 30);
    usernameLabel->setAlignment(Qt::AlignCenter);
    accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
    accounttypeLabel->setGeometry(240, 40, 50, 30);
    accounttypeLabel->setAlignment(Qt::AlignCenter);

    carIdLabel = new QLabel("车牌号: ", this);
    carIdLabel->setGeometry(50, 100, 70, 30);
    carIdLineEdit = new QLineEdit(this);
    carIdLineEdit->setGeometry(120, 100, 150, 30);

    removeCarButton = new QPushButton("删除", this);
    removeCarButton->setGeometry(100, 150, 100, 30);
    connect(removeCarButton, &QPushButton::clicked, this, &
        RemoveCarWindow::removeCar);
}

RemoveCarWindow::~RemoveCarWindow() {
    closeAllSubWindows();
    removeFromParent();
}

void RemoveCarWindow::removeCar() {
    std::string car_id = carIdLineEdit->text().toStdString();
    if (car_id.empty()) {
```

```
54      QMessageBox::warning(this, "错误", "车牌号不能为空");
55      return;
56    }
57
58    auto res = ClientHttpHandler::handler_remove_car(ip, port, acc,
          car_id);
59    if (res) {
60      QMessageBox::information(this, "成功", "车辆删除成功");
61    }
62    else {
63      QMessageBox::critical(
64        this, "错误", std::string("车辆删除失败
            n错误信息: " + res.message).c_str());
65    }
66  }
```

qt-src/search-account-window.cpp

```
1   /**
2    * @file qt-src/search-account-window.cpp
3    * @brief Implementation of class SearchAccountWindow
4    *
5    * @details
6    * This file contains the implementation of the SearchAccountWindow
         class.
7    *
8    * @author donghy23@mails.tsinghua.edu.cn
9    * @version 1.0
10   */
11
12  #include "carinfo-manager/search-account-window.hpp"
13  #include <QFileDialog>
14  #include <QMessageBox>
15  #include <fstream>
16  #include <sstream>
17  #include "carinfo-manager/display-accountinfo-window.hpp"
18  #include "carinfo-manager/httphandler-client.hpp"
19  #include "json/json.hpp"
20
21  using json = nlohmann::json;
22
23  SearchAccountWindow::SearchAccountWindow(QWidget *parent,
24                      const std::string &ip,
```

```
                        const int port,
                        const Account &acc,
                        QWidgetWithSubWindows *parentWindow)
    : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
      acc(acc) {
    this->setWindowTitle("查询账户");
    this->setFixedSize(300, 300);

    titleLabel = new QLabel("查询账户", this);
    titleLabel->setGeometry(50, 50, 200, 30);
    titleLabel->setAlignment(Qt::AlignCenter);
    titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

    usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
        this);
    usernameLabel->setGeometry(240, 20, 50, 30);
    usernameLabel->setAlignment(Qt::AlignCenter);
    accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
    accounttypeLabel->setGeometry(240, 40, 50, 30);
    accounttypeLabel->setAlignment(Qt::AlignCenter);

    searchUsernameLabel = new QLabel("用户名: ", this);
    searchUsernameLabel->setGeometry(50, 100, 70, 30);
    searchUsernameLineEdit = new QLineEdit(this);
    searchUsernameLineEdit->setGeometry(120, 100, 150, 30);

    searchButton = new QPushButton("查询", this);
    searchButton->setGeometry(100, 150, 100, 30);
    connect(searchButton, &QPushButton::clicked, this, &
       SearchAccountWindow::searchAccount);

    saveAllAccountsButton = new QPushButton("保存所有账户", this);
    saveAllAccountsButton->setGeometry(100, 190, 100, 30);
    connect(
      saveAllAccountsButton, &QPushButton::clicked, this, &
         SearchAccountWindow::saveAllAccounts);
}

SearchAccountWindow::~SearchAccountWindow() {
    closeAllSubWindows();
    removeFromParent();
```

```
62 }
63
64 void SearchAccountWindow::searchAccount() {
65   std::string searchUsername = searchUsernameLineEdit->text().
       toStdString();
66   if (searchUsername.empty()) {
67     QMessageBox::critical(this, "错误", "用户名不能为空");
68     return;
69   }
70
71   auto res = ClientHttpHandler::handler_search_account(ip, port, acc,
       searchUsername);
72   if (!res) {
73     QMessageBox::critical(
74       this, "错误", std::string("查询账户失败
          n错误信息: " + res.message).c_str());
75     return;
76   }
77   std::stringstream ss;
78   ss << res.json_obj.dump(4);
79   AccountPool accountpool;
80   accountpool.load(ss);
81
82   DisplayAccountinfoWindow *displayAccountinfoWindow =
83     new DisplayAccountinfoWindow(nullptr, accountpool, searchUsername,
         ip, port, acc, this);
84   addSubWindow(displayAccountinfoWindow);
85   displayAccountinfoWindow->show();
86 }
87
88 void SearchAccountWindow::saveAllAccounts() {
89   auto res = ClientHttpHandler::handler_get_all_accounts(ip, port, acc)
       ;
90   if (!res) {
91     QMessageBox::critical(
92       this, "错误", std::string("获取所有账户失败
          n错误信息: " + res.message).c_str());
93     return;
94   }
95
96   std::string filename =
97     QFileDialog::getSaveFileName(this, "保存文件", "", "JOSN文
```

```
        件(*.json)").toLocal8Bit().data();
98    if (filename.empty()) {
99      QMessageBox::critical(this, "错误", "文件名不合法");
100     return;
101   }
102
103   std::ofstream file(filename, std::ios::out);
104   if (!file.is_open()) {
105     QMessageBox::critical(this, "错误", "无法打开文件");
106     return;
107   }
108   file << res.json_obj.dump(4);
109   file.close();
110   QMessageBox::information(this, "成功", "保存成功");
111 }
```

qt-src/search-car-window.cpp

```
1  /**
2   * @file qt-src/search-car-window.cpp
3   * @brief Implementation of class SearchCarWindow
4   *
5   * @details
6   * This file contains the implementation of the SearchCarWindow class.
7   *
8   * @author donghy23@mails.tsinghua.edu.cn
9   * @version 1.0
10  */
11
12 #include "carinfo-manager/search-car-window.hpp"
13 #include "carinfo-manager/accountpool.hpp"
14 #include "carinfo-manager/carpool.hpp"
15 #include "carinfo-manager/display-carinfo-window.hpp"
16 #include "carinfo-manager/httphandler-client.hpp"
17 #include "json/json.hpp"
18
19 using json = nlohmann::json;
20
21 SearchCarWindow::SearchCarWindow(QWidget *parent,
22                                  const std::string &ip,
23                                  const int port,
24                                  const Account &acc,
```

```
25                  QWidgetWithSubWindows *parentWindow)
26  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
        acc(acc) {
27  this->setWindowTitle("查找车辆");
28  this->setFixedSize(300, 300);
29
30  titleLabel = new QLabel("查询车辆信息", this);
31  titleLabel->setGeometry(50, 50, 200, 30);
32  titleLabel->setAlignment(Qt::AlignCenter);
33  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));
34
35  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
        this);
36  usernameLabel->setGeometry(240, 20, 50, 15);
37  usernameLabel->setAlignment(Qt::AlignCenter);
38  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
39  accounttypeLabel->setGeometry(240, 40, 50, 15);
40  accounttypeLabel->setAlignment(Qt::AlignCenter);
41
42  caridLabel = new QLabel("车牌号: ", this);
43  caridLabel->setGeometry(50, 100, 50, 30);
44  caridLabel->setAlignment(Qt::AlignVCenter | Qt::AlignRight);
45  carownerLabel = new QLabel("车主: ", this);
46  carownerLabel->setGeometry(50, 130, 50, 30);
47  carownerLabel->setAlignment(Qt::AlignVCenter | Qt::AlignRight);
48  cartypeLabel = new QLabel("车型: ", this);
49  cartypeLabel->setGeometry(50, 160, 50, 30);
50  cartypeLabel->setAlignment(Qt::AlignVCenter | Qt::AlignRight);
51  carcolorLabel = new QLabel("颜色: ", this);
52  carcolorLabel->setGeometry(50, 190, 50, 30);
53  carcolorLabel->setAlignment(Qt::AlignVCenter | Qt::AlignRight);
54
55  caridEdit = new QLineEdit(this);
56  caridEdit->setGeometry(110, 100, 100, 30);
57  carownerEdit = new QLineEdit(this);
58  carownerEdit->setGeometry(110, 130, 100, 30);
59  cartypeEdit = new QLineEdit(this);
60  cartypeEdit->setGeometry(110, 160, 100, 30);
61  carcolorEdit = new QLineEdit(this);
62  carcolorEdit->setGeometry(110, 190, 100, 30);
63
```

```
64    searchButton = new QPushButton("查询", this);
65    searchButton->setGeometry(100, 240, 100, 30);
66    connect(searchButton, &QPushButton::clicked, this, &SearchCarWindow::
          search);
67  }
68
69  SearchCarWindow::~SearchCarWindow() {
70    removeFromParent();
71  }
72
73  void SearchCarWindow::search() {
74    std::string car_id = caridEdit->text().toStdString();
75    std::string car_owner = carownerEdit->text().toStdString();
76    std::string car_type = cartypeEdit->text().toStdString();
77    std::string car_color = carcolorEdit->text().toStdString();
78
79    auto res = ClientHttpHandler::handler_get_carinfo(
80      ip, port, acc, car_id, car_owner, car_color, car_type);
81    if (!res) {
82      QMessageBox::critical(
83        this, "错误", std::string("查询失败
            n错误信息: " + res.message).c_str());
84      return;
85    }
86
87    std::stringstream ss;
88    ss << res.json_obj.dump(4);
89    CarPool searchRes;
90    searchRes.load(ss);
91
92    DisplayCarInfoWindow *displayCarinfoWindow =
93      new DisplayCarInfoWindow(nullptr,
94                   searchRes,
95                   SearchCarQuery(car_id, car_owner, car_type, car_color)
                         ,
96                   ip,
97                   port,
98                   acc,
99                   this);
100   addSubWindow(displayCarinfoWindow);
101   displayCarinfoWindow->show();
```

```
}
```

qt-src/select-server-window.cpp

```
/**
 * @file qt-src/select-server-window.cpp
 * @brief Implementation of class SelectServerWindow
 *
 * @details
 * This file contains the implementation of the SelectServerWindow
    class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/select-server-window.hpp"
#include "carinfo-manager/accountpool.hpp"
#include "carinfo-manager/hash.hpp"
#include "carinfo-manager/httphandler-client.hpp"
#include "carinfo-manager/login-window.hpp"
#include "carinfo-manager/search-car-window.hpp"

SelectServerWindow::SelectServerWindow(QWidget *parent) : QWidget(
   parent) {
  this->setWindowTitle("选择服务器");
  this->setFixedSize(400, 300);

  titleLabel = new QLabel("车辆管理系统", this);
  titleLabel->setGeometry(100, 50, 200, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  serverLabel = new QLabel("服务器地址: ", this);
  serverLabel->setGeometry(50, 100, 100, 30);
  serverLabel->setAlignment(Qt::AlignCenter);
  serverEdit = new QLineEdit(this);
  serverEdit->setGeometry(150, 100, 200, 30);
  serverEdit->setPlaceholderText("请输入服务器地址");

  portLabel = new QLabel("端口: ", this);
  portLabel->setGeometry(50, 150, 100, 30);
```

```
37    portLabel->setAlignment(Qt::AlignCenter);
38    portEdit = new QLineEdit(this);
39    portEdit->setGeometry(150, 150, 200, 30);
40    portEdit->setPlaceholderText("请输入端口");
41
42    verifyButton = new QPushButton("验证", this);
43    verifyButton->setGeometry(150, 200, 100, 30);
44    connect(verifyButton, &QPushButton::clicked, this, &
          SelectServerWindow::verifyServer);
45  }
46
47  SelectServerWindow::~SelectServerWindow() {}
48
49  void SelectServerWindow::verifyServer() {
50    if (serverEdit->text().isEmpty() || portEdit->text().isEmpty()) {
51      QMessageBox::critical(this, "错误", "服务器地址或端口不能为空");
52      return;
53    }
54    try {
55      ip = serverEdit->text().toStdString();
56      port = portEdit->text().toInt();
57    }
58    catch (std::exception &e) {
59      QMessageBox::critical(this, "错误", "不合法的地址或端口");
60      return;
61    }
62
63    bool isServerValid = ClientHttpHandler::handler_verify_server(ip,
          port);
64    if (isServerValid) {
65      LoginWindow *loginWindow = new LoginWindow(nullptr, ip, port);
66      this->close();
67      loginWindow->show();
68    }
69    else {
70      QMessageBox::critical(this, "错误", "无法连接到服务器");
71    }
72  }
```

qt-src/update-account-window.cpp

```
1  /**
```

```
 * @file qt-src/update-account-window.cpp
 * @brief Implementation of class UpdateAccountWindow
 *
 * @details
 * This file contains the implementation of the UpdateAccountWindow
    class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/update-account-window.hpp"
#include <QMessageBox>
#include "carinfo-manager/hash.hpp"
#include "carinfo-manager/httphandler-client.hpp"
#include "carinfo-manager/log.hpp"

UpdateAccountWindow::UpdateAccountWindow(QWidget *parent,
                     const std::string &ip,
                     const int port,
                     const Account &acc,
                     QWidgetWithSubWindows *parentWindow)
  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
     acc(acc) {
  this->setWindowTitle("修改账户");
  this->setFixedSize(320, 370);

  titleLabel = new QLabel("修改账户", this);
  titleLabel->setGeometry(110, 50, 100, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
      this);
  usernameLabel->setGeometry(250, 20, 50, 30);
  usernameLabel->setAlignment(Qt::AlignCenter);
  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
  accounttypeLabel->setGeometry(250, 40, 50, 30);
  accounttypeLabel->setAlignment(Qt::AlignCenter);

  targetUsernameLabel = new QLabel("原用户名: ", this);
```

```
40    targetUsernameLabel->setGeometry(50, 100, 60, 30);
41    targetUsernameEdit = new QLineEdit(this);
42    targetUsernameEdit->setGeometry(120, 100, 140, 30);
43
44    newUsernameLabel = new QLabel("新用户名: ", this);
45    newUsernameLabel->setGeometry(50, 140, 60, 30);
46    newUsernameEdit = new QLineEdit(this);
47    newUsernameEdit->setGeometry(120, 140, 140, 30);
48
49    newPasswordLabel = new QLabel("新密码: ", this);
50    newPasswordLabel->setGeometry(50, 180, 60, 30);
51    newPasswordEdit = new QLineEdit(this);
52    newPasswordEdit->setEchoMode(QLineEdit::Password);
53    newPasswordEdit->setGeometry(120, 180, 140, 30);
54
55    confirmPasswordLabel = new QLabel("确认密码: ", this);
56    confirmPasswordLabel->setGeometry(50, 220, 60, 30);
57    confirmPasswordEdit = new QLineEdit(this);
58    confirmPasswordEdit->setEchoMode(QLineEdit::Password);
59    confirmPasswordEdit->setGeometry(120, 220, 140, 30);
60
61    newAccounttypeLabel = new QLabel("新账户类型: ", this);
62    newAccounttypeLabel->setGeometry(50, 260, 70, 30);
63    newAccounttypeComboBox = new QComboBox(this);
64    newAccounttypeComboBox->setGeometry(130, 260, 130, 30);
65    newAccounttypeComboBox->addItem("用户");
66    newAccounttypeComboBox->addItem("管理员");
67
68    updateAccountButton = new QPushButton("修改", this);
69    updateAccountButton->setGeometry(110, 310, 100, 30);
70    connect(updateAccountButton, &QPushButton::clicked, this, &
          UpdateAccountWindow::updateAccount);
71  }
72
73  UpdateAccountWindow::~UpdateAccountWindow() {
74    closeAllSubWindows();
75    removeFromParent();
76  }
77
78  void UpdateAccountWindow::updateAccount() {
79    std::string targetUsername = this->targetUsernameEdit->text().
```

```
    toStdString();
80  std::string newUsername = this->newUsernameEdit->text().toStdString()
    ;
81  std::string newPassword = this->newPasswordEdit->text().toStdString()
    ;
82  std::string confirmPassword = this->confirmPasswordEdit->text().
    toStdString();
83  Account::AccountType newAccountType = this->newAccounttypeComboBox->
    currentIndex() == 0
84                        ? Account::AccountType::USER
85                        : Account::AccountType::ADMIN;
86  if (newPassword != confirmPassword) {
87    QMessageBox::critical(this, "错误", "两次输入的密码不一致");
88    return;
89  }
90  if (newUsername.empty() || newPassword.empty()) {
91    QMessageBox::critical(this, "错误", "用户名和密码不能为空");
92    return;
93  }
94
95  Hash passwordHash(newPassword, "Carinfo-Manager");
96  std::string newPasswdHash = passwordHash.getHash();
97
98  auto res = ClientHttpHandler::handler_update_account(this->ip,
99                               this->port,
100                              this->acc,
101                              targetUsername,
102                              newUsername,
103                              newPasswdHash,
104                              newAccountType);
105 if (res)
106   QMessageBox::information(this, "成功", "修改账户成功");
107 else
108   QMessageBox::critical(
109     this, "错误", std::string("修改账户失败
        n错误信息: " + res.message).c_str());
110 }
```

qt-src/update-car-window.cpp

```
1  /**
2   * @file qt-src/update-car-window.cpp
3   * @brief Implementation of class UpdateCarWindow
```

```
 *
 * @details
 * This file contains the implementation of the UpdateCarWindow class.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/update-car-window.hpp"
#include <QFileDialog>
#include <QMessageBox>
#include <fstream>
#include <string>
#include "carinfo-manager/httphandler-client.hpp"

UpdateCarWindow::UpdateCarWindow(QWidget *parent,
                 const std::string &ip,
                 const int port,
                 const Account &acc,
                 QWidgetWithSubWindows *parentWindow)
  : QWidgetWithSubWindows(parent, parentWindow), ip(ip), port(port),
     acc(acc) {
  this->setWindowTitle("修改车辆");
  this->setFixedSize(320, 460);

  titleLabel = new QLabel("修改车辆", this);
  titleLabel->setGeometry(50, 50, 200, 30);
  titleLabel->setAlignment(Qt::AlignCenter);
  titleLabel->setFont(QFont("Arial", 16, QFont::Bold));

  usernameLabel = new QLabel(QString::fromStdString(acc.getUsername()),
      this);
  usernameLabel->setGeometry(250, 20, 50, 30);
  usernameLabel->setAlignment(Qt::AlignCenter);
  accounttypeLabel = new QLabel(acc.isAdmin() ? "管理员" : "用户", this);
  accounttypeLabel->setGeometry(250, 40, 50, 30);
  accounttypeLabel->setAlignment(Qt::AlignCenter);

  originalCarIdLabel = new QLabel("原车牌号: ", this);
  originalCarIdLabel->setGeometry(50, 100, 70, 30);
  originalCarIdLineEdit = new QLineEdit(this);
```

```
43      originalCarIdLineEdit->setGeometry(120, 100, 150, 30);
44
45      newCarIdLabel = new QLabel("新车牌号: ", this);
46      newCarIdLabel->setGeometry(50, 140, 70, 30);
47      newCarIdLineEdit = new QLineEdit(this);
48      newCarIdLineEdit->setGeometry(120, 140, 150, 30);
49
50      newCarOwnerLabel = new QLabel("新车主: ", this);
51      newCarOwnerLabel->setGeometry(50, 180, 70, 30);
52      newCarOwnerLineEdit = new QLineEdit(this);
53      newCarOwnerLineEdit->setGeometry(120, 180, 150, 30);
54
55      newCarTypeLabel = new QLabel("新车型: ", this);
56      newCarTypeLabel->setGeometry(50, 220, 70, 30);
57      newCarTypeLineEdit = new QLineEdit(this);
58      newCarTypeLineEdit->setGeometry(120, 220, 150, 30);
59
60      newCarColorLabel = new QLabel("新颜色: ", this);
61      newCarColorLabel->setGeometry(50, 260, 70, 30);
62      newCarColorLineEdit = new QLineEdit(this);
63      newCarColorLineEdit->setGeometry(120, 260, 150, 30);
64
65      newCarYearLabel = new QLabel("新年份: ", this);
66      newCarYearLabel->setGeometry(50, 300, 70, 30);
67      newCarYearLineEdit = new QLineEdit(this);
68      newCarYearLineEdit->setGeometry(120, 300, 150, 30);
69
70      newCarImgPathLabel = new QLabel("新图片路径: ", this);
71      newCarImgPathLabel->setGeometry(50, 340, 70, 30);
72      newCarImgPathLineEdit = new QLineEdit(this);
73      newCarImgPathLineEdit->setGeometry(120, 340, 110, 30);
74      carImgSelectButton = new QPushButton("选择", this);
75      carImgSelectButton->setGeometry(230, 340, 40, 30);
76      connect(carImgSelectButton, &QPushButton::clicked, this, &
            UpdateCarWindow::getCarImgPath);
77
78      updateCarButton = new QPushButton("修改", this);
79      updateCarButton->setGeometry(110, 380, 100, 30);
80      connect(updateCarButton, &QPushButton::clicked, this, &
            UpdateCarWindow::updateCar);
81  }
```

```
82
83  UpdateCarWindow::~UpdateCarWindow() {
84    closeAllSubWindows();
85    removeFromParent();
86  }
87
88  void UpdateCarWindow::getCarImgPath() {
89    QString imgPath =
90      QFileDialog::getOpenFileName(this, "选择图
          片", "", "Images (*.png *.jpg *.jpeg)");
91    if (!imgPath.isEmpty())
92      newCarImgPathLineEdit->setText(imgPath);
93  }
94
95  void UpdateCarWindow::updateCar() {
96    if (originalCarIdLineEdit->text().isEmpty() || newCarIdLineEdit->text
       ().isEmpty() ||
97      newCarOwnerLineEdit->text().isEmpty() || newCarTypeLineEdit->text()
         .isEmpty() ||
98      newCarColorLineEdit->text().isEmpty() || newCarYearLineEdit->text()
         .isEmpty() ||
99      newCarImgPathLineEdit->text().isEmpty()) {
100     QMessageBox::warning(this, "错误", "请填写完整信息");
101     return;
102   }
103
104   std::string originalCarId = originalCarIdLineEdit->text().toStdString
       (),
105         newCarId = newCarIdLineEdit->text().toStdString(),
106         newCarOwner = newCarOwnerLineEdit->text().toStdString(),
107         newCarType = newCarTypeLineEdit->text().toStdString(),
108         newCarColor = newCarColorLineEdit->text().toStdString(),
109         newCarImgPath = newCarImgPathLineEdit->text().toLocal8Bit().
            data(),
110         newCarImgType = "", newCarImgData = "";
111   int newCarYear = newCarYearLineEdit->text().toInt();
112
113   if (newCarImgPath.find_last_of('.') != std::string::npos) {
114     newCarImgType = newCarImgPath.substr(newCarImgPath.find_last_of('.'
         ));
115     if (newCarImgType != ".png" && newCarImgType != ".jpg" &&
```

```
        newCarImgType != ".jpeg") {
116       QMessageBox::critical(this, "错误", "图片格式不支持");
117       return;
118     }
119   }
120   else {
121     QMessageBox::critical(this, "错误", "图片路径不合法");
122     return;
123   }
124
125   std::ifstream imgFile(newCarImgPath, std::ios::binary);
126   if (imgFile.is_open()) {
127     newCarImgData =
128       std::string(std::istreambuf_iterator<char>(imgFile), std::
            istreambuf_iterator<char>());
129     imgFile.close();
130   }
131   else {
132     QMessageBox::critical(this, "错误", "打开图片失败");
133     return;
134   }
135
136   auto res = ClientHttpHandler::handler_update_car(ip,
137                            port,
138                            acc,
139                            originalCarId,
140                            newCarId,
141                            newCarOwner,
142                            newCarType,
143                            newCarColor,
144                            newCarYear,
145                            newCarImgData,
146                            newCarImgType);
147   if (res)
148     QMessageBox::information(this, "成功", "修改成功");
149   else
150     QMessageBox::critical(
151       this, "错误", std::string("修改车辆失败
          n错误信息: " + res.message).c_str());
152 }
```

src/accountpool.cpp

```cpp
/**
 * @file src/accountpool.cpp
 * @brief Implementation of class Account and class AccountPool, which are used to manage accounts in the system, including loading and saving accounts from and to files, adding, removing, updating and verifying accounts, etc. The class Account is used to represent an account, and the class AccountPool is used to manage a pool of accounts. The class AccountPool is derived from the class BasicPool, which is a base class for all pool classes in the system.
 *
 * @details
 * The Account class represents an account in the system. It stores the username, password hash, and account type.
 * The AccountPool class manages a pool of accounts. It provides functions to add, remove, update, and verify accounts.
 * The AccountPool class is derived from the BasicPool class, which is a base class for all pool classes in the system.
 * The AccountPool class uses three internal maps to store the accounts: accountpool, adminpool, and userpool. The accountpool map stores all accounts, while the adminpool and userpool maps store only the admin and user accounts, respectively.
 * The AccountPool class provides functions to load and save accounts from and to files, as well as functions to retrieve the size of the account pool and check if it is empty.
 * The AccountPool class also provides functions to clear the account pool and get a list of all accounts.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/accountpool.hpp"
#include <assert.h>
#include <fstream>
#include "carinfo-manager/log.hpp"
#include "json/json.hpp"
using json = nlohmann::json;

Account::Account() {
  username = "";
```

```
26    passwd_hash = "";
27    account_type = AccountType::NONETYPE;
28  }
29
30  Account::Account(const std::string &username,
31           const std::string &passwd_hash,
32           const AccountType &account_type) {
33    assert(passwd_hash.length() == 64);
34    this->username = username;
35    this->passwd_hash = passwd_hash;
36    this->account_type = account_type;
37  }
38
39  Account::Account(const Account &acc) {
40    username = acc.username;
41    passwd_hash = acc.passwd_hash;
42    account_type = acc.account_type;
43  }
44
45  Account::~Account() {}
46
47  std::string Account::getUsername() const {
48    return username;
49  }
50
51  std::string Account::getPasswdHash() const {
52    return passwd_hash;
53  }
54
55  Account::AccountType Account::getAccountType() const {
56    return account_type;
57  }
58
59  bool Account::isAdmin() const {
60    return account_type == AccountType::ADMIN;
61  }
62
63  bool Account::isUser() const {
64    return account_type == AccountType::USER;
65  }
66
```

```
67  void Account::setUsername(const std::string &username) {
68    this->username = username;
69  }
70
71  void Account::setPasswdHash(const std::string &passwd_hash) {
72    assert(passwd_hash.length() == 64);
73    this->passwd_hash = passwd_hash;
74  }
75
76  void Account::setAccountType(const AccountType &account_type) {
77    this->account_type = account_type;
78  }
79
80  bool Account::operator==(const Account &acc) const {
81    return username == acc.username;
82  }
83
84  bool Account::operator!=(const Account &acc) const {
85    return username != acc.username;
86  }
87
88  bool Account::operator<(const Account &acc) const {
89    return username < acc.username;
90  }
91
92  bool Account::operator>(const Account &acc) const {
93    return username > acc.username;
94  }
95
96  bool Account::operator<=(const Account &acc) const {
97    return username <= acc.username;
98  }
99
100 bool Account::operator>=(const Account &acc) const {
101   return username >= acc.username;
102 }
103
104 Account &Account::operator=(const Account &acc) {
105   username = acc.username;
106   passwd_hash = acc.passwd_hash;
107   account_type = acc.account_type;
```

```
108   return *this;
109 }
110
111 const Account Account::NULL_ACCOUNT = Account();
112
113 /**
114  * @brief Default constructor for the AccountPool class.
115  *
116  * This constructor initializes the account pool with an empty map of
        accounts and sets the size of the account pool to 0.
117  */
118 AccountPool::AccountPool() {
119   accountpool = std::map<std::string, Account>();
120   adminpool = std::map<std::string, Account>();
121   userpool = std::map<std::string, Account>();
122   sz = 0;
123 }
124
125 /**
126  * @brief Constructor for the AccountPool class that initializes the
        account pool with a range of accounts.
127  *
128  * This constructor initializes the account pool with the accounts in
        the range [begin, end). It adds each account in the range to the
        account pool.
129  *
130  * @param begin An iterator pointing to the beginning of the range of
        accounts.
131  * @param end An iterator pointing to the end of the range of accounts.
132  */
133 AccountPool::AccountPool(Account *begin, Account *end) {
134   accountpool = std::map<std::string, Account>();
135   adminpool = std::map<std::string, Account>();
136   userpool = std::map<std::string, Account>();
137   sz = 0;
138   for (Account *it = begin; it != end; it++) {
139     if (addAccount(*it) != 0)
140       throw "Error: AccountPool::AccountPool(Account* begin, Account*
            end)";
141   }
142 }
```

```
143
144 /**
145  * @brief Constructor for the AccountPool class that initializes the
        account pool with a vector of accounts.
146  *
147  * This constructor initializes the account pool with the accounts in
        the specified vector. It adds each account in the vector to the
        account pool.
148  *
149  * @param accounts The vector of accounts to initialize the account
        pool with.
150  */
151 AccountPool::AccountPool(const std::vector<Account> &accounts) {
152   accountpool = std::map<std::string, Account>();
153   adminpool = std::map<std::string, Account>();
154   userpool = std::map<std::string, Account>();
155   sz = 0;
156   for (Account acc : accounts) {
157     if (addAccount(acc) != 0)
158       throw "Error: AccountPool::AccountPool(std::vector<Account>
            accounts)";
159   }
160 }
161
162 /**
163  * @brief Copy constructor for the AccountPool class.
164  *
165  * This constructor initializes the account pool with the accounts in
        the specified AccountPool object.
166  *
167  * @param ap The AccountPool object to copy.
168  */
169 AccountPool::AccountPool(const AccountPool &ap) {
170   accountpool = ap.accountpool;
171   adminpool = ap.adminpool;
172   userpool = ap.userpool;
173   sz = ap.sz;
174 }
175
176 /**
177  * @brief Destructor for the AccountPool class.
```

```
178   *
179   * This destructor clears the account pool and frees the memory used by
          the account pool.
180   */
181  AccountPool::~AccountPool() {
182    accountpool.clear();
183    adminpool.clear();
184    userpool.clear();
185    sz = 0;
186  }
187
188  /**
189   * @brief Adds an account to the account pool.
190   *
191   * This function adds the specified account to the account pool. It
          checks if the account already exists in the pool and returns an
          error code if it does. If the account type is ADMIN, it also adds
          the account to the admin pool. If the account type is USER, it adds
           the account to the user pool. Finally, it increments the size of
          the account pool.
192   *
193   * @param acc The account to be added.
194   * @return Returns 0 if the account was successfully added, else an
          error code:
195   *      - 0x10: The account already exists in the pool.
196   *      - 0x1F: An unknown error occurred.
197   */
198  int AccountPool::addAccount(const Account &acc) {
199    try {
200      if (accountpool.find(acc.getUsername()) != accountpool.end()) {
201        MyLogger::log("carinfo-manager-logger",
202                MyLogger::LOG_LEVEL::DEBUG,
203                "[AccountPool Add Account] \n- Username: " + acc.
                      getUsername() +
204                "\n- PasswdHash: " + acc.getPasswdHash() + "\n-
                      AccountType: " +
205                std::to_string((int)acc.getAccountType()) + "\n-
                      Stuatus: 0x10");
206        return 0x10;
207      }
208      accountpool[acc.getUsername()] = acc;
```

```
209     if (acc.getAccountType() == Account::AccountType::ADMIN) {
210       adminpool[acc.getUsername()] = acc;
211     }
212     else if (acc.getAccountType() == Account::AccountType::USER) {
213       userpool[acc.getUsername()] = acc;
214     }
215     sz++;
216     MyLogger::log("carinfo-manager-logger",
217             MyLogger::LOG_LEVEL::DEBUG,
218             "[AccountPool Add Account] \n- Username: " + acc.
                getUsername() +
219               "\n- PasswdHash: " + acc.getPasswdHash() + "\n-
                  AccountType: " +
220               std::to_string((int)acc.getAccountType()) + "\n- Stuatus:
                  0");
221     return 0;
222   }
223   catch (...) {
224     MyLogger::log("carinfo-manager-logger",
225             MyLogger::LOG_LEVEL::ERROR,
226             "[AccountPool Add Account] \n- Username: " + acc.
                getUsername() +
227               "\n- PasswdHash: " + acc.getPasswdHash() + "\n-
                  AccountType: " +
228               std::to_string((int)acc.getAccountType()) + "\n- Stuatus:
                  0x1F");
229     return 0x1F;
230   }
231 }
232
233 /**
234  * @brief Removes an account from the account pool.
235  *
236  * This function removes the account with the specified username from
       the account pool.
237  * If the account is successfully removed, it returns 0. If the account
       does not exist in the account pool, it returns 0x20.
238  * If an exception occurs during the removal process, it returns 0x2F.
239  *
240  * @param username The username of the account to be removed.
241  * @return Returns 0 if the account is successfully removed, else an
```

```
     error code:
242  *      - 0x20: The account does not exist in the pool.
243  *      - 0x2F: An unknown error occurred.
244  */
245 int AccountPool::removeAccount(const std::string &username) {
246   try {
247     if (accountpool.find(username) == accountpool.end()) {
248       MyLogger::log("carinfo-manager-logger",
249               MyLogger::LOG_LEVEL::DEBUG,
250               "[AccountPool Remove Account] " + username + ". Stuatus:
                    0x20");
251       return 0x20;
252     }
253
254     Account acc = accountpool[username];
255     accountpool.erase(username);
256     if (acc.getAccountType() == Account::AccountType::ADMIN) {
257       adminpool.erase(username);
258     }
259     else if (acc.getAccountType() == Account::AccountType::USER) {
260       userpool.erase(username);
261     }
262     sz--;
263     MyLogger::log("carinfo-manager-logger",
264             MyLogger::LOG_LEVEL::DEBUG,
265             "[AccountPool Remove Account] " + username + ". Stuatus: 0"
                  );
266     return 0;
267   }
268   catch (...) {
269     MyLogger::log("carinfo-manager-logger",
270             MyLogger::LOG_LEVEL::ERROR,
271             "[AccountPool Remove Account] " + username + ". Stuatus: 0
                  x2F");
272     return 0x2F;
273   }
274 }
275
276 /**
277  * @brief Removes an account from the account pool.
278  *
```

```
279  * This function removes the account with the specified username from
         the account pool.
280  * If the account is successfully removed, it returns 0. If the account
         does not exist in the account pool, it returns 0x30.
281  * If an exception occurs during the removal process, it returns 0x3F.
282  *
283  * @param acc The account to be removed.
284  * @return Returns 0 if the account is successfully removed, else an
         error code:
285  *     - 0x30: The account does not exist in the pool.
286  *     - 0x3F: An unknown error occurred.
287  */
288 int AccountPool::removeAccount(const Account &acc) {
289   return removeAccount(acc.getUsername());
290 }
291
292 /**
293  * @brief Updates an account in the account pool.
294  *
295  * This function removes the original account from the account pool and
         adds the new account to the pool.
296  *
297  * @param original_acc The original account to be updated.
298  * @param new_acc The new account to replace the original account.
299  * @return Returns 0 if the update is successful, else an error code:
300  *     - 0x30: The original account does not exist in the pool.
301  *     - 0x31: The new account could not be added to the pool.
302  *     - 0x3F: An unknown error occurred.
303  */
304 int AccountPool::updateAccount(const Account &original_acc, const
       Account &new_acc) {
305   try {
306     if (removeAccount(original_acc) != 0) {
307       MyLogger::log(
308         "carinfo-manager-logger",
309         MyLogger::LOG_LEVEL::DEBUG,
310         "[AccountPool Update Account] \n- Original Username: " +
311           original_acc.getUsername() + "\n- New Username: " + new_acc.
                getUsername() +
312           "\n- New PasswdHash: " + new_acc.getPasswdHash() + "\n- New
                AccountType: " +
```

```
313             std::to_string((int)new_acc.getAccountType()) + "\n- Stuatus:
                    0x30");
314         return 0x30;
315       }
316       if (addAccount(new_acc) != 0) {
317         MyLogger::log(
318           "carinfo-manager-logger",
319           MyLogger::LOG_LEVEL::DEBUG,
320           "[AccountPool Update Account] \n- Original Username: " +
321             original_acc.getUsername() + "\n- New Username: " + new_acc.
                  getUsername() +
322             "\n- New PasswdHash: " + new_acc.getPasswdHash() + "\n- New
                  AccountType: " +
323             std::to_string((int)new_acc.getAccountType()) + "\n- Stuatus:
                    0x31");
324         return 0x31;
325       }
326       MyLogger::log(
327         "carinfo-manager-logger",
328         MyLogger::LOG_LEVEL::DEBUG,
329         "[AccountPool Update Account] \n- Original Username: " +
              original_acc.getUsername() +
330           "\n- New Username: " + new_acc.getUsername() +
331           "\n- New PasswdHash: " + new_acc.getPasswdHash() + "\n- New
                AccountType: " +
332           std::to_string((int)new_acc.getAccountType()) + "\n- Stuatus: 0
                ");
333       return 0;
334     }
335     catch (...) {
336       MyLogger::log(
337         "carinfo-manager-logger",
338         MyLogger::LOG_LEVEL::ERROR,
339         "[AccountPool Update Account] \n- Original Username: " +
              original_acc.getUsername() +
340           "\n- New Username: " + new_acc.getUsername() +
341           "\n- New PasswdHash: " + new_acc.getPasswdHash() + "\n- New
                AccountType: " +
342           std::to_string((int)new_acc.getAccountType()) + "\n- Stuatus: 0
                x3F");
343       return 0x3F;
```

```
344   }
345 }
346
347 /**
348  * Retrieves the account associated with the given username from the
        account pool.
349  * If the account is not found, returns a null account.
350  *
351  * @param username The username of the account to retrieve.
352  * @return The account associated with the given username, or a null
        account if not found.
353  */
354 Account AccountPool::getAccount(const std::string &username) const {
355   if (accountpool.find(username) == accountpool.end())
356     return Account::NULL_ACCOUNT;
357   MyLogger::log(
358     "carinfo-manager-logger",
359     MyLogger::LOG_LEVEL::DEBUG,
360     "[AccountPool Get Account] \n- Username: " + username +
361       "\n- PasswdHash: " + accountpool.at(username).getPasswdHash() +
362       "\n- AccountType: " + std::to_string((int)accountpool.at(username
            ).getAccountType()));
363   return accountpool.at(username);
364 }
365
366 /**
367  * Retrieves a list of accounts that match the given username.
368  *
369  * This function searches for accounts in the account pool whose
        usernames contain the given username as a substring.
370  *
371  * @param username The username to search for.
372  * @return A vector containing all accounts whose usernames contain the
        given username as a substring.
373  *       If no accounts are found, returns an empty vector.
374  */
375 AccountPool AccountPool::getAccountLike(const std::string &username)
      const {
376   AccountPool ap;
377   for (auto it = accountpool.begin(); it != accountpool.end(); it++) {
378     if (it->first.find(username) != std::string::npos)
```

```
379         ap.addAccount(it->second);
380     }
381     std::stringstream ss;
382     ap.save(ss);
383     MyLogger::log("carinfo-manager-logger",
384             MyLogger::LOG_LEVEL::DEBUG,
385             "[AccountPool Get Account Like] \n- Username: " + username +
386               "\n- AccountPool: " + ss.str());
387     return ap;
388 }
389 
390 /**
391  * Verifies the account with the given username and password hash.
392  *
393  * @param username The username of the account to verify.
394  * @param passwd_hash The password hash of the account to verify.
395  * @return The result of the account verification.
396  *         - ACCOUNT_NOT_FOUND if the account with the given username is
       not found.
397  *         - WRONG_PASSWORD if the password hash does not match the account
       's password hash.
398  *         - SUCCESS if the account is successfully verified.
399  */
400 AccountPool::AccountVerifyResult AccountPool::verifyAccount(const std::
       string &username,
401                                 const std::string &passwd_hash) const {
402     if (accountpool.find(username) == accountpool.end()) {
403         MyLogger::log("carinfo-manager-logger",
404                 MyLogger::LOG_LEVEL::DEBUG,
405                 "[AccountPool Verify Account] \n- Username: " + username +
406                   "\n- PasswdHash: " + passwd_hash + "\n- Stuatus: 1");
407         return AccountVerifyResult::ACCOUNT_NOT_FOUND;
408     }
409     if (accountpool.at(username).getPasswdHash() != passwd_hash) {
410         MyLogger::log("carinfo-manager-logger",
411                 MyLogger::LOG_LEVEL::DEBUG,
412                 "[AccountPool Verify Account] \n- Username: " + username +
413                   "\n- PasswdHash: " + passwd_hash + "\n- Stuatus: 2");
414         return AccountVerifyResult::WRONG_PASSWORD;
415     }
416     MyLogger::log("carinfo-manager-logger",
```

```
417             MyLogger::LOG_LEVEL::DEBUG,
418             "[AccountPool Verify Account] \n- Username: " + username +
419               "\n- PasswdHash: " + passwd_hash + "\n- Stuatus: 0");
420   return AccountVerifyResult::SUCCESS;
421 }
422
423 /**
424  * Retrieves the account type associated with the given username.
425  *
426  * @param username The username of the account.
427  * @return The account type associated with the username. If the
       username is not found in the account pool, returns Account::
       AccountType::NONETYPE.
428  */
429 Account::AccountType AccountPool::getAccountType(const std::string &
      username) const {
430   if (accountpool.find(username) == accountpool.end()) {
431     MyLogger::log(
432       "carinfo-manager-logger",
433       MyLogger::LOG_LEVEL::DEBUG,
434       "[AccountPool Get Account Type] \n- Username: " + username + "\n-
            Stuatus: 1");
435     return Account::AccountType::NONETYPE;
436   }
437   MyLogger::log("carinfo-manager-logger",
438             MyLogger::LOG_LEVEL::DEBUG,
439             "[AccountPool Get Account Type] \n- Username: " + username +
                "\n- AccountType: " +
440               std::to_string((int)accountpool.at(username).getAccountType
                    ()) +
441               "\n- Stuatus: 0");
442   return accountpool.at(username).getAccountType();
443 }
444
445 /**
446  * Retrieves the size of the account pool.
447  *
448  * @return The size of the account pool.
449  */
450 size_t AccountPool::size() const {
451   MyLogger::log("carinfo-manager-logger",
```

```
452             MyLogger::LOG_LEVEL::DEBUG,
453             "[AccountPool Size] \n- Size: " + std::to_string(sz));
454     return sz;
455 }
456
457 /**
458  * Checks if the account pool is empty.
459  *
460  * @return True if the account pool is empty, false otherwise.
461  */
462 bool AccountPool::empty() const {
463     MyLogger::log("carinfo-manager-logger",
464             MyLogger::LOG_LEVEL::DEBUG,
465             "[AccountPool Empty] \n- Stuatus: " + std::to_string(sz == 0)
                );
466     return sz == 0;
467 }
468
469 /**
470  * @brief Clears the account pool.
471  *
472  * This function clears the account pool by removing all accounts from
       the account pool and setting the size of the account pool to 0.
473  *
474  * @return Returns 0 if the account pool is successfully cleared, else
       an error code:
475  *     - 0x4F: An unknown error occurred.
476  */
477 int AccountPool::clear() {
478     try {
479         accountpool.clear();
480         adminpool.clear();
481         userpool.clear();
482         sz = 0;
483         MyLogger::log("carinfo-manager-logger",
484                 MyLogger::LOG_LEVEL::DEBUG,
485                 "[AccountPool Clear] \n- Stuatus: 0");
486         return 0;
487     }
488     catch (...) {
489         MyLogger::log("carinfo-manager-logger",
```

```
490             MyLogger::LOG_LEVEL::ERROR,
491             "[AccountPool Clear] \n- Stuatus: 0x4F");
492     return 0x4F;
493   }
494 }
495
496 /**
497  * @brief Loads account data from an input stream.
498  *
499  * This function reads account data from the specified input stream and
        populates the AccountPool object with the loaded accounts.
500  *
501  * @param is The input stream to read from.
502  * @return Returns 0 if the accounts are loaded successfully, else an
       error code:
503  *     - 0x50: The input stream is invalid.
504  *     - 0x51: An error occurred while clearing the account pool.
505  *     - 0x52: The loaded data is not in the correct format.
506  *     - 0x53: The loaded data is missing required fields.
507  *     - 0x54: The loaded data has incorrect field types.
508  *     - 0x55: An error occurred while adding an account to the account
        pool.
509  *     - 0x5F: An unknown error occurred.
510  */
511 int AccountPool::load(std::istream &is) {
512   if (!is) {
513     MyLogger::log("carinfo-manager-logger",
514             MyLogger::LOG_LEVEL::ERROR,
515             "[AccountPool Load] \n- Stuatus: 0x50");
516     return 0x50;
517   }
518   try {
519     if (clear() != 0) {
520       MyLogger::log("carinfo-manager-logger",
521               MyLogger::LOG_LEVEL::ERROR,
522               "[AccountPool Load] \n- Stuatus: 0x51");
523       return 0x51;
524     }
525     json load_json_obj = json::parse(is);
526     for (json::iterator it = load_json_obj.begin(); it != load_json_obj
          .end(); it++) {
```

```
527       if (!it.value().is_object()) {
528         MyLogger::log("carinfo-manager-logger",
529                 MyLogger::LOG_LEVEL::ERROR,
530                 "[AccountPool Load] \n- Stuatus: 0x52");
531         return 0x52;
532       }
533       if (!it.value().contains("username") || !it.value().contains("
              passwd_hash") ||
534         !it.value().contains("account_type")) {
535         MyLogger::log("carinfo-manager-logger",
536                 MyLogger::LOG_LEVEL::ERROR,
537                 "[AccountPool Load] \n- Stuatus: 0x53");
538         return 0x53;
539       }
540       if (!it.value()["username"].is_string() || !it.value()["
              passwd_hash"].is_string() ||
541         !it.value()["account_type"].is_number_integer()) {
542         MyLogger::log("carinfo-manager-logger",
543                 MyLogger::LOG_LEVEL::ERROR,
544                 "[AccountPool Load] \n- Stuatus: 0x54");
545         return 0x54;
546       }
547       Account acc(std::string(it.key()),
548             std::string(it.value()["passwd_hash"]),
549             (Account::AccountType)(int)(it.value()["account_type"]));
550       if (addAccount(acc) != 0) {
551         MyLogger::log("carinfo-manager-logger",
552                 MyLogger::LOG_LEVEL::ERROR,
553                 "[AccountPool Load] \n- Stuatus: 0x55");
554         return 0x55;
555       }
556     }
557     MyLogger::log("carinfo-manager-logger",
558             MyLogger::LOG_LEVEL::DEBUG,
559             "[AccountPool Load] \n- Stuatus: 0");
560     return 0;
561   }
562   catch (...) {
563     MyLogger::log("carinfo-manager-logger",
564             MyLogger::LOG_LEVEL::ERROR,
565             "[AccountPool Load] \n- Stuatus: 0x5F");
```

```
566     return 0x5F;
567   }
568 }
569
570 /**
571  * Saves the AccountPool to an output stream.
572  *
573  * @param os The output stream to save the AccountPool to.
574  * @return Returns 0 if the AccountPool is successfully saved, else an
        error code:
575  *      - 0x60: The output stream is invalid.
576  *      - 0x6F: An unknown error occurred.
577  */
578 int AccountPool::save(std::ostream &os) const {
579   if (!os) {
580     MyLogger::log("carinfo-manager-logger",
581             MyLogger::LOG_LEVEL::ERROR,
582             "[AccountPool Save] \n- Stuatus: 0x60");
583     return 0x60;
584   }
585   try {
586     // Write size
587     json save_json_obj;
588     for (auto it = accountpool.begin(); it != accountpool.end(); it++)
          {
589       json acc_json_obj;
590       acc_json_obj["username"] = it->second.getUsername();
591       acc_json_obj["passwd_hash"] = it->second.getPasswdHash();
592       acc_json_obj["account_type"] = (int)it->second.getAccountType();
593       save_json_obj[it->first] = acc_json_obj;
594     }
595     os << save_json_obj.dump(4);
596     MyLogger::log("carinfo-manager-logger",
597             MyLogger::LOG_LEVEL::DEBUG,
598             "[AccountPool Save] \n- Stuatus: 0");
599     return 0;
600   }
601   catch (...) {
602     MyLogger::log("carinfo-manager-logger",
603             MyLogger::LOG_LEVEL::ERROR,
604             "[AccountPool Save] \n- Stuatus: 0x6F");
```

```
605       return 0x6F;
606     }
607   }
608
609   /**
610    * Retrieves a list of all accounts in the account pool.
611    *
612    * @return A vector containing all the accounts in the account pool.
613    */
614   std::vector<Account> AccountPool::list() const {
615     std::vector<Account> accounts;
616     for (auto it = accountpool.begin(); it != accountpool.end(); it++) {
617       accounts.push_back(it->second);
618     }
619     MyLogger::log(
620       "carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "[AccountPool
              List] \n- Stuatus: 0");
621     return accounts;
622   }
623
624   /**
625    * @brief Equality operator for the AccountPool class.
626    *
627    * This operator checks if two AccountPool objects are equal by
         comparing their size, account pools, admin pools, and user pools.
628    *
629    * @param ap The AccountPool object to compare with.
630    * @return True if the AccountPool objects are equal, false otherwise.
631    */
632   bool AccountPool::operator==(const AccountPool &ap) const {
633     return sz == ap.sz && accountpool == ap.accountpool && adminpool ==
         ap.adminpool &&
634            userpool == ap.userpool;
635   }
636
637   /**
638    * @brief Inequality operator for the AccountPool class.
639    *
640    * This operator checks if two AccountPool objects are not equal by
         comparing their size, account pools, admin pools, and user pools.
641    *
```

```
642  * @param ap The AccountPool object to compare with.
643  * @return True if the AccountPool objects are not equal, false
         otherwise.
644  */
645 bool AccountPool::operator!=(const AccountPool &ap) const {
646   return sz != ap.sz || accountpool != ap.accountpool || adminpool !=
         ap.adminpool ||
647          userpool != ap.userpool;
648 }
649
650 /**
651  * @brief Less than operator for the AccountPool class.
652  *
653  * This operator compares two AccountPool objects by comparing their
         size.
654  *
655  * @param ap The AccountPool object to compare with.
656  * @return True if the size of the AccountPool object is less than the
         size of the other AccountPool object, false otherwise.
657  */
658 AccountPool &AccountPool::operator=(const AccountPool &ap) {
659   sz = ap.sz;
660   accountpool = ap.accountpool;
661   adminpool = ap.adminpool;
662   userpool = ap.userpool;
663   return *this;
664 }
```

src/base64.cpp

```
1  /**
2  * @file src/base64.cpp
3  * @brief Implementation of the Base64 class.
4  *
5  * @details
6  * This file contains the implementation of the Base64 class, which
       provides functions for encoding and decoding data in Base64 format.
7  *
8  * @author donghy23@mails.tsinghua.edu.cn
9  * @version 1.0
10 */
11
```

```
12 #include "carinfo-manager/base64.hpp"
13 #include <cstring>
14
15 const char Base64::encodeTable[] =
16   "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
17 const char Base64::decodeTable[] = {
18   0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
19   0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
20   0,  0,  0,  0,  0,  0,  0,  0,  0,
21   62,  // '+'
22   0,  0,  0,
23   63,                    // '/'
24   52, 53, 54, 55, 56, 57, 58, 59, 60, 61,  // '0'-'9'
25   0,  0,  0,  0,  0,  0,  0,  0,  1,  2,  3,  4,  5,  6,  7,  8,  9,
26   10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25,  // '
       A'-'Z'
27   0,  0,  0,  0,  0,  0,  26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,
28   37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51,  // 'a'-'
       z'
29 };
30
31 /**
32  * @brief Encodes the given data into Base64 format.
33  *
34  * @param data The data to be encoded.
35  * @param data_bytes_length The length of the data in bytes.
36  *
37  * @return The Base64 encoded string.
38  */
39 std::string Base64::encode(const unsigned char *data,
40                            size_t data_bytes_length) {
41   std::string strEncode;
42   unsigned char Tmp[4] = {0};
43   int LineLength = 0;
44   for (int i = 0; i < (int)(data_bytes_length / 3); i++) {
45     Tmp[1] = *data++;
46     Tmp[2] = *data++;
47     Tmp[3] = *data++;
48     strEncode += encodeTable[Tmp[1] >> 2];
49     strEncode += encodeTable[((Tmp[1] << 4) | (Tmp[2] >> 4)) & 0x3F];
50     strEncode += encodeTable[((Tmp[2] << 2) | (Tmp[3] >> 6)) & 0x3F];
```

```
    strEncode += encodeTable[Tmp[3] & 0x3F];
    if (LineLength += 4, LineLength == 76)
      strEncode += "\r\n";
    LineLength = 0;
  }

  int Mod = data_bytes_length % 3;
  if (Mod == 1) {
    Tmp[1] = *data++;
    strEncode += encodeTable[(Tmp[1] & 0xFC) >> 2];
    strEncode += encodeTable[((Tmp[1] & 0x03) << 4)];
    strEncode += "==";
  }
  else if (Mod == 2) {
    Tmp[1] = *data++;
    Tmp[2] = *data++;
    strEncode += encodeTable[(Tmp[1] & 0xFC) >> 2];
    strEncode +=
      encodeTable[((Tmp[1] & 0x03) << 4) | ((Tmp[2] & 0xF0) >> 4)];
    strEncode += encodeTable[((Tmp[2] & 0x0F) << 2)];
    strEncode += "=";
  }

  return strEncode;
}

/**
 * @brief Decodes a Base64 encoded string.
 *
 * @param data The Base64 encoded data to decode.
 * @param data_bytes_length The length of the data in bytes.
 * @param output_bytes_length The length of the decoded output in bytes
    .
 *
 * @return The decoded string.
 */
std::string Base64::decode(const char *data,
                size_t data_bytes_length,
                size_t &output_bytes_length) {
  std::string strDecode;
  int nValue;
```

```
  size_t i = 0;
  while (i < data_bytes_length) {
    if (*data != '\r' && *data != '\n') {
      nValue = decodeTable[*data++] << 18;
      nValue += decodeTable[*data++] << 12;
      strDecode += (nValue & 0x00FF0000) >> 16;
      output_bytes_length++;
      if (*data != '=') {
        nValue += decodeTable[*data++] << 6;
        strDecode += (nValue & 0x0000FF00) >> 8;
        output_bytes_length++;
        if (*data != '=') {
          nValue += decodeTable[*data++];
          strDecode += nValue & 0x000000FF;
          output_bytes_length++;
        }
      }
      i += 4;
    }
    else {
      data++;
      i++;
    }
  }
  return strDecode;
}
```

src/carpool.cpp

```
/**
 * @file src/carpool.cpp
 * @brief Implementation of class Car, class Color, and class CarPool
 *
 * @details
 * This file contains the implementation of the Car, Color, and CarPool
     classes.
 * The Car class represents a car object with properties such as ID,
     type, color, year, and image path.
 * The Color class represents a color object with RGB values.
 * The CarPool class represents a collection of cars and provides
     operations to add and remove cars.
 * It also provides iterators to iterate over the cars in different
```

```
     orders (by ID, color, or type).
 * The implementation of these classes is provided in this file.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/carpool.hpp"
#include "carinfo-manager/log.hpp"
#include <fstream>
#include "json/json.hpp"
using nlohmann::json;

Car::Car() {
  car_id = "";
  car_type = "";
  car_color = "";
  car_owner = "";
  car_year = 0;
  car_img_path = "";
}

Car::Car(const std::string &id,
     const std::string &type,
     const std::string &owner,
     const std::string &color,
     const int year,
     const std::string &img_path) {
  car_id = id;
  car_type = type;
  car_owner = owner;
  car_color = color;
  car_year = year;
  car_img_path = img_path;
}

Car::Car(const Car &c)
  : car_id(c.car_id),
    car_type(c.car_type),
    car_color(c.car_color),
    car_owner(c.car_owner),
```

```
51      car_year(c.car_year),
52      car_img_path(c.car_img_path) {}
53
54  Car::~Car() {}
55
56  void Car::setId(const std::string &id) {
57    car_id = id;
58  }
59
60  void Car::setType(const std::string &type) {
61    car_type = type;
62  }
63
64  void Car::setOwner(const std::string &owner) {
65    car_owner = owner;
66  }
67
68  void Car::setColor(const std::string &color) {
69    car_color = color;
70  }
71
72  void Car::setYear(const int year) {
73    car_year = year;
74  }
75
76  void Car::setImagePath(const std::string &img_path) {
77    car_img_path = img_path;
78  }
79
80  std::string Car::getId() const {
81    return car_id;
82  }
83
84  std::string Car::getType() const {
85    return car_type;
86  }
87
88  std::string Car::getOwner() const {
89    return car_owner;
90  }
91
```

```
92  std::string Car::getColor() const {
93    return car_color;
94  }
95
96  int Car::getYear() const {
97    return car_year;
98  }
99
100 std::string Car::getImagePath() const {
101   return car_img_path;
102 }
103
104 bool Car::operator==(const Car &c) const {
105   return car_id == c.car_id;
106 }
107
108 bool Car::operator!=(const Car &c) const {
109   return car_id != c.car_id;
110 }
111
112 bool Car::operator<(const Car &c) const {
113   return car_id < c.car_id;
114 }
115
116 bool Car::operator>(const Car &c) const {
117   return car_id > c.car_id;
118 }
119
120 bool Car::operator<=(const Car &c) const {
121   return car_id <= c.car_id;
122 }
123
124 bool Car::operator>=(const Car &c) const {
125   return car_id >= c.car_id;
126 }
127
128 Car &Car::operator=(const Car &c) {
129   car_id = c.car_id;
130   car_type = c.car_type;
131   car_owner = c.car_owner;
132   car_color = c.car_color;
```

```
133   car_year = c.car_year;
134   car_img_path = c.car_img_path;
135   return *this;
136 }
137
138 const Car Car::NULL_CAR = Car();
139
140 /**
141  * @brief Constructs a new CarPool object.
142  *
143  * This constructor initializes the CarPool object by creating empty
        maps for carpool_byid, carpool_bycolor, and carpool_bytype.
144  * It also sets the initial size of the carpool to 0.
145  */
146 CarPool::CarPool() {
147   carpool_byid = std::map<std::string, Car>();
148   carpool_bycolor = std::multimap<std::string, Car>();
149   carpool_bytype = std::multimap<std::string, Car>();
150   carpool_byowner = std::multimap<std::string, Car>();
151   sz = 0;
152 }
153
154 /**
155  * @brief Constructs a new CarPool object with cars in the range [begin
        , end).
156  *
157  * This constructor initializes the CarPool object by adding cars in
        the range [begin, end) to the carpool.
158  *
159  * @param begin An iterator pointing to the first car in the range.
160  * @param end An iterator pointing to the end of the range.
161  */
162 CarPool::CarPool(Car *begin, Car *end) {
163   carpool_byid = std::map<std::string, Car>();
164   carpool_bycolor = std::multimap<std::string, Car>();
165   carpool_bytype = std::multimap<std::string, Car>();
166   carpool_byowner = std::multimap<std::string, Car>();
167   sz = 0;
168   for (Car *i = begin; i != end; i++)
169     addCar(*i);
170 }
```

```
171
172 /**
173  * @brief Constructs a new CarPool object with cars in the vector.
174  *
175  * This constructor initializes the CarPool object by adding cars in
        the vector to the carpool.
176  *
177  * @param cars A vector containing the cars to be added to the carpool.
178  */
179 CarPool::CarPool(const std::vector<Car> &cars) {
180   carpool_byid = std::map<std::string, Car>();
181   carpool_bycolor = std::multimap<std::string, Car>();
182   carpool_bytype = std::multimap<std::string, Car>();
183   carpool_byowner = std::multimap<std::string, Car>();
184   sz = 0;
185   for (Car car : cars)
186     addCar(car);
187 }
188
189 /**
190  * @brief Constructs a new CarPool object by copying an existing
        CarPool object.
191  *
192  * This constructor initializes the CarPool object by copying the
        contents of the provided CarPool object.
193  *
194  * @param cp The CarPool object to be copied.
195  */
196 CarPool::CarPool(const CarPool &cp) {
197   carpool_byid = cp.carpool_byid;
198   carpool_bycolor = cp.carpool_bycolor;
199   carpool_bytype = cp.carpool_bytype;
200   carpool_byowner = cp.carpool_byowner;
201   sz = cp.sz;
202 }
203
204 /**
205  * @brief Destroys the CarPool object.
206  *
207  * This destructor clears the carpool data by removing all cars from
        the carpool containers.
```

```
208  */
209 CarPool::~CarPool() {
210   carpool_byid.clear();
211   carpool_bycolor.clear();
212   carpool_bytype.clear();
213   carpool_byowner.clear();
214   sz = 0;
215 }
216
217 /**
218  * Adds a car to the carpool.
219  *
220  * @param car The car object to be added.
221  * @return Returns 0 if the car is added successfully, else an error
       code:
222  *      - 0x70: If the car with the same ID already exists in the
       carpool.
223  *      - 0x7F: If an unknown exception occurs while adding the car.
224  */
225 int CarPool::addCar(const Car &car) {
226   try {
227     if (carpool_byid.find(car.getId()) != carpool_byid.end()){
228       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
             ERROR, "[CarPool Add Car] \n- Car ID: " + car.getId() + "\n-
             Car Owner: " + car.getOwner() + "\n- Car Type: " + car.getType
             () + "\n- Car Color: " + car.getColor() + "\n- Car Year: " +
             std::to_string(car.getYear()) + "\n- Car Image Path: " + car.
             getImagePath() + "\n- Status: 0x70");
229       return 0x70;}
230     carpool_byid[car.getId()] = car;
231     carpool_bycolor.insert(std::make_pair(car.getColor(), car));
232     carpool_bytype.insert(std::make_pair(car.getType(), car));
233     carpool_byowner.insert(std::make_pair(car.getOwner(), car));
234     sz++;
235     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
           "[CarPool Add Car] \n- Car ID: " + car.getId() + "\n- Car Owner
           : " + car.getOwner() + "\n- Car Type: " + car.getType() + "\n-
           Car Color: " + car.getColor() + "\n- Car Year: " + std::
           to_string(car.getYear()) + "\n- Car Image Path: " + car.
           getImagePath() + "\n- Status: 0");
236     return 0;
```

```
237   }
238   catch (...) {
239     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
          "[CarPool Add Car] \n- Car ID: " + car.getId() + "\n- Car Owner
          : " + car.getOwner() + "\n- Car Type: " + car.getType() + "\n-
          Car Color: " + car.getColor() + "\n- Car Year: " + std::
          to_string(car.getYear()) + "\n- Car Image Path: " + car.
          getImagePath() + "\n- Status: 0x7F");
240     return 0x7F;
241   }
242 }
243
244 /**
245  * Removes a car from the car pool.
246  *
247  * @param car The car to be removed.
248  * @return The number of cars removed (0 or 1).
249  */
250 int CarPool::removeCar(const Car &car) {
251   return removeCar(car.getId());
252 }
253
254 /**
255  * @brief Removes a car from the carpool based on its ID.
256  *
257  * This function removes a car from the carpool based on the provided
       ID. It searches for the car with the given ID in the carpool_byid
       map and removes it if found. Additionally, it removes the car from
       the carpool_bycolor and carpool_bytype maps if it exists in those
       maps as well. Finally, it decrements the size of the carpool by one
       .
258  *
259  * @param id The ID of the car to be removed.
260  * @return Returns 0 if the car was successfully removed, else an error
        code:
261  *     - 0x80: If the car with the specified ID does not exist in the
       carpool.
262  *     - 0x8F: If an unknown exception occurs while removing the car.
263  */
264 int CarPool::removeCar(const std::string &id) {
265   try {
```

```
266     auto it_id = carpool_byid.find(id);
267     if (it_id == carpool_byid.end()){
268       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
            ERROR, "[CarPool Remove Car] \n- Car ID: " + id + "\n- Status:
             0x80");
269       return 0x80;}
270
271     Car car = it_id->second;
272     carpool_byid.erase(it_id);
273     auto it_cl_bg = carpool_bycolor.lower_bound(car.getColor()),
274        it_cl_ed = carpool_bycolor.upper_bound(car.getColor());
275     for (auto it_cl = it_cl_bg; it_cl != it_cl_ed; it_cl++) {
276       if (it_cl->second.getId() == car.getId()) {
277         carpool_bycolor.erase(it_cl);
278         break;
279       }
280     }
281     auto it_on_bg = carpool_byowner.lower_bound(car.getOwner()),
282        it_on_ed = carpool_byowner.upper_bound(car.getOwner());
283     for (auto it_on = it_on_bg; it_on != it_on_ed; it_on++) {
284       if (it_on->second.getId() == car.getId()) {
285         carpool_byowner.erase(it_on);
286         break;
287       }
288     }
289     auto it_ty_bg = carpool_bytype.lower_bound(car.getType()),
290        it_ty_ed = carpool_bytype.upper_bound(car.getType());
291     for (auto it_ty = it_ty_bg; it_ty != it_ty_ed; it_ty++) {
292       if (it_ty->second.getId() == car.getId()) {
293         carpool_bytype.erase(it_ty);
294         break;
295       }
296     }
297     sz--;
298     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
           "[CarPool Remove Car] \n- Car ID: " + id + "\n- Status: 0");
299     return 0;
300   }
301   catch (...) {
302     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
           "[CarPool Remove Car] \n- Car ID: " + id + "\n- Status: 0x8F");
```

```
303     return 0x8F;
304   }
305 }
306
307 /**
308  * @brief Updates a car in the car pool.
309  *
310  * This function replaces the original car with the new car in the car
        pool.
311  * It first removes the original car from the car pool using the
        removeCar function,
312  * and then adds the new car to the car pool using the addCar function.
313  *
314  * @param original_car The original car to be replaced.
315  * @param new_car The new car to replace the original car.
316  * @return Returns 0 if the update is successful, else an error code:
317  *     - 0x90: If the removal of the original car is successful but the
         addition of the new car fails.
318  *     - 0x91: If the addition of the new car fails.
319  *     - 0x9F: If an exception occurs during the update process.
320  */
321 int CarPool::updateCar(const Car &original_car, const Car &new_car) {
322   try {
323     if (removeCar(original_car)){
324       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
             ERROR, "[CarPool Update Car] \n- Original Car ID: " +
             original_car.getId() + "\n- New Car ID: " + new_car.getId() +
             "\n- New Car Owner: " + new_car.getOwner() + "\n- New Car Type
             : " + new_car.getType() + "\n- New Car Color: " + new_car.
             getColor() + "\n- New Car Year: " + std::to_string(new_car.
             getYear()) + "\n- New Car Image Path: " + new_car.getImagePath
             () + "\n- Status: 0x90");
325       return 0x90;}
326     if (addCar(new_car)){
327       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
             ERROR, "[CarPool Update Car] \n- Original Car ID: " +
             original_car.getId() + "\n- New Car ID: " + new_car.getId() +
             "\n- New Car Owner: " + new_car.getOwner() + "\n- New Car Type
             : " + new_car.getType() + "\n- New Car Color: " + new_car.
             getColor() + "\n- New Car Year: " + std::to_string(new_car.
             getYear()) + "\n- New Car Image Path: " + new_car.getImagePath
```

```
        () + "\n- Status: 0x91");
328       return 0x91;}
329     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
          "[CarPool Update Car] \n- Original Car ID: " + original_car.
          getId() + "\n- New Car ID: " + new_car.getId() + "\n- New Car
          Owner: " + new_car.getOwner() + "\n- New Car Type: " + new_car.
          getType() + "\n- New Car Color: " + new_car.getColor() + "\n-
          New Car Year: " + std::to_string(new_car.getYear()) + "\n- New
          Car Image Path: " + new_car.getImagePath() + "\n- Status: 0");
330     return 0;
331   }
332   catch (...) {
333     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
          "[CarPool Update Car] \n- Original Car ID: " + original_car.
          getId() + "\n- New Car ID: " + new_car.getId() + "\n- New Car
          Owner: " + new_car.getOwner() + "\n- New Car Type: " + new_car.
          getType() + "\n- New Car Color: " + new_car.getColor() + "\n-
          New Car Year: " + std::to_string(new_car.getYear()) + "\n- New
          Car Image Path: " + new_car.getImagePath() + "\n- Status: 0x9F")
          ;
334     return 0x9F;
335   }
336 }
337
338 /**
339  * @brief Updates a car in the car pool.
340  *
341  * This function updates a car in the car pool by first removing the
       car with the specified ID and then adding the new car.
342  *
343  * @param id The ID of the car to be updated.
344  * @param new_car The new car object to replace the existing car.
345  * @return Returns 0 if the car was successfully updated, else an error
       code:
346  *     - 0x90: If the removal of the original car is successful but the
       addition of the new car fails.
347  *     - 0x91: If the addition of the new car fails.
348  *     - 0x9F: If an exception occurs during the update process.
349  */
350 int CarPool::updateCar(const std::string &id, const Car &new_car) {
351   try {
```

```
352     if (removeCar(id)){
353       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              ERROR, "[CarPool Update Car] \n- Original Car ID: " + id + "\n
              - New Car ID: " + new_car.getId() + "\n- New Car Owner: " +
              new_car.getOwner() + "\n- New Car Type: " + new_car.getType()
              + "\n- New Car Color: " + new_car.getColor() + "\n- New Car
              Year: " + std::to_string(new_car.getYear()) + "\n- New Car
              Image Path: " + new_car.getImagePath() + "\n- Status: 0x90");
354       return 0x90;}
355     if (addCar(new_car)){
356       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              ERROR, "[CarPool Update Car] \n- Original Car ID: " + id + "\n
              - New Car ID: " + new_car.getId() + "\n- New Car Owner: " +
              new_car.getOwner() + "\n- New Car Type: " + new_car.getType()
              + "\n- New Car Color: " + new_car.getColor() + "\n- New Car
              Year: " + std::to_string(new_car.getYear()) + "\n- New Car
              Image Path: " + new_car.getImagePath() + "\n- Status: 0x91");
357       return 0x91;}
358       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              DEBUG, "[CarPool Update Car] \n- Original Car ID: " + id + "\n
              - New Car ID: " + new_car.getId() + "\n- New Car Owner: " +
              new_car.getOwner() + "\n- New Car Type: " + new_car.getType()
              + "\n- New Car Color: " + new_car.getColor() + "\n- New Car
              Year: " + std::to_string(new_car.getYear()) + "\n- New Car
              Image Path: " + new_car.getImagePath() + "\n- Status: 0");
359     return 0;
360   }
361   catch (...) {
362     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
            "[CarPool Update Car] \n- Original Car ID: " + id + "\n- New
          Car ID: " + new_car.getId() + "\n- New Car Owner: " + new_car.
          getOwner() + "\n- New Car Type: " + new_car.getType() + "\n- New
           Car Color: " + new_car.getColor() + "\n- New Car Year: " + std
          ::to_string(new_car.getYear()) + "\n- New Car Image Path: " +
          new_car.getImagePath() + "\n- Status: 0x9F");
363     return 0x9F;
364   }
365 }
366
367 /**
368  * Retrieves a CarPool object containing the car with the specified ID.
```

```
369   *
370   * @param id The ID of the car to retrieve.
371   * @return A CarPool object containing the car with the specified ID,
          or an empty CarPool object if the car is not found.
372   */
373 CarPool CarPool::getCarbyId(const std::string &id) const {
374   CarPool cars;
375   if (carpool_byid.find(id) != carpool_byid.end())
376     cars.addCar(carpool_byid.at(id));
377   std::stringstream ss;
378   cars.save(ss);
379   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
          [CarPool Get Car by ID] \n- Car ID: " + id + "\n- Result: " + ss.
          str());
380   return cars;
381 }
382
383 /**
384   * Retrieves a CarPool object containing all cars with the specified
          color.
385   *
386   * @param color The color of the cars to retrieve.
387   * @return A CarPool object containing all cars with the specified
          color.
388   */
389 CarPool CarPool::getCarbyColor(const std::string &color) const {
390   CarPool cars;
391   auto it_bg = carpool_bycolor.lower_bound(color), it_ed =
          carpool_bycolor.upper_bound(color);
392   for (auto it = it_bg; it != it_ed; it++)
393     cars.addCar(it->second);
394   std::stringstream ss;
395   cars.save(ss);
396   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
          [CarPool Get Car by Color] \n- Car Color: " + color + "\n- Result:
           " + ss.str());
397   return cars;
398 }
399
400 /**
401   * Retrieves a CarPool object containing all cars owned by a specific
```

```
     owner.
402  *
403  * @param owner The owner of the cars to retrieve.
404  * @return A CarPool object containing all cars owned by the specified
         owner.
405  */
406 CarPool CarPool::getCarbyOwner(const std::string &owner) const {
407   CarPool cars;
408   auto it_bg = carpool_byowner.lower_bound(owner), it_ed =
         carpool_byowner.upper_bound(owner);
409   for (auto it = it_bg; it != it_ed; it++)
410     cars.addCar(it->second);
411   std::stringstream ss;
412   cars.save(ss);
413   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
         [CarPool Get Car by Owner] \n- Car Owner: " + owner + "\n- Result:
          " + ss.str());
414   return cars;
415 }
416
417 /**
418  * Retrieves a CarPool object containing all cars of a specific type.
419  *
420  * @param type The type of cars to retrieve.
421  * @return A CarPool object containing all cars of the specified type.
422  */
423 CarPool CarPool::getCarbyType(const std::string &type) const {
424   CarPool cars;
425   auto it_bg = carpool_bytype.lower_bound(type), it_ed = carpool_bytype
         .upper_bound(type);
426   for (auto it = it_bg; it != it_ed; it++)
427     cars.addCar(it->second);
428   std::stringstream ss;
429   cars.save(ss);
430   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
         [CarPool Get Car by Type] \n- Car Type: " + type + "\n- Result: "
         + ss.str());
431   return cars;
432 }
433
434 /**
```

```
435  * Retrieves a car from the car pool based on the specified criteria.
436  *
437  * @param id The ID of the car to search for.
438  * @param color The color of the car to search for.
439  * @param owner The owner of the car to search for.
440  * @param type The type of the car to search for.
441  * @return The car that matches the specified criteria.
442  */
443 CarPool CarPool::getCar(const std::string &id,
444             const std::string &color,
445             const std::string &owner,
446             const std::string &type) const {
447   std::string _id = id, _color = color, _type = type, _owner = owner;
448   CarPool cars = *this;
449   while (!_id.empty() || !_color.empty() || !_type.empty() || !_owner.
        empty()) {
450     if (_id != "") {
451       cars = cars.getCarbyId(_id);
452       _id = "";
453     }
454     if (_owner != "") {
455       cars = cars.getCarbyOwner(_owner);
456       _owner = "";
457     }
458     if (_color != "") {
459       cars = cars.getCarbyColor(_color);
460       _color = "";
461     }
462     if (_type != "") {
463       cars = cars.getCarbyType(_type);
464       _type = "";
465     }
466   }
467   std::stringstream ss;
468   cars.save(ss);
469   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
        [CarPool Get Car] \n- Car ID: " + id + "\n- Car Owner: " + owner +
        "\n- Car Type: " + type + "\n- Car Color: " + color + "\n- Result
        : " + ss.str());
470   return cars;
471 }
```

```
472
473 /**
474  * @brief Retrieves the number of cars in the carpool.
475  *
476  * This function returns the number of cars in the carpool.
477  *
478  * @return The number of cars in the carpool.
479  */
480 size_t CarPool::size() const {
481   return sz;
482 }
483
484 /**
485  * @brief Checks if the carpool is empty.
486  *
487  * This function checks if the carpool is empty by checking if the size
       of the carpool is zero.
488  *
489  * @return True if the carpool is empty, otherwise false.
490  */
491 bool CarPool::empty() const {
492   return sz == 0;
493 }
494
495 /**
496  * @brief Clears the carpool data.
497  *
498  * This function clears the carpool data by removing all cars from the
       carpool containers and resetting the size to zero.
499  *
500  * @return 0 if the carpool data is cleared successfully, 0xAF if an
       exception occurs.
501  */
502 int CarPool::clear() {
503   try {
504     carpool_byid.clear();
505     carpool_bycolor.clear();
506     carpool_bytype.clear();
507     carpool_byowner.clear();
508     sz = 0;
509     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
```

```
        "[CarPool Clear] \n- Status: 0");
510     return 0;
511   }
512   catch (...) {
513     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
            "[CarPool Clear] \n- Status: 0xAF");
514     return 0xAF;
515   }
516 }
517
518 /**
519  * @brief Loads car data from an input stream.
520  *
521  * This function reads car data from the provided input stream and
        populates the CarPool object with the loaded cars.
522  *
523  * @param is The input stream to read car data from.
524  * @return Returns 0 if the car data is successfully loaded, otherwise
        returns an error code:
525  *     - 0xB0: If the input stream is not valid.
526  *     - 0xB1: If there is an error while clearing the existing car
        data in the CarPool object.
527  *     - 0xB2: If the JSON object is not an object.
528  *     - 0xB3: If the JSON object does not contain the required fields.
529  *     - 0xB4: If the JSON object contains fields with incorrect types.
530  *     - 0xB5: If there is an error while adding a car to the CarPool
        object.
531  *     - 0xBF: If an unknown exception occurs while loading the car
        data.
532  */
533 int CarPool::load(std::istream &is) {
534   if (!is){
535     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
            "[CarPool Load] \n- Status: 0xB0");
536     return 0xB0;}
537   try {
538     if (clear() != 0){
539       MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              ERROR, "[CarPool Load] \n- Status: 0xB1");
540       return 0xB1;}
541     json load_json_obj = json::parse(is);
```

```
542     for (auto it = load_json_obj.begin(); it != load_json_obj.end(); it
            ++) {
543       if (!it.value().is_object()){
544         MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              ERROR, "[CarPool Load] \n- Status: 0xB2");
545         return 0xB2;}
546       if (!it.value().contains("id") || !it.value().contains("type") ||
547         !it.value().contains("owner") || !it.value().contains("color")
              ||
548         !it.value().contains("year") || !it.value().contains("img_path"
              )){
549           MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
                ERROR, "[CarPool Load] \n- Status: 0xB3");
550         return 0xB3;}
551       if (!it.value()["id"].is_string() || !it.value()["type"].
            is_string() ||
552         !it.value()["owner"].is_string() || !it.value()["color"].
              is_string() ||
553         !it.value()["year"].is_number_integer() || !it.value()["
              img_path"].is_string()){
554           MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
                ERROR, "[CarPool Load] \n- Status: 0xB4");
555         return 0xB4;}
556       Car car(std::string(it.value()["id"]),
557           std::string(it.value()["type"]),
558           std::string(it.value()["owner"]),
559           std::string(it.value()["color"]),
560           int(it.value()["year"]),
561           std::string(it.value()["img_path"]));
562       if (addCar(car)){
563         MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::
              ERROR, "[CarPool Load] \n- Status: 0xB5");
564         return 0xB5;}
565     }
566     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
          "[CarPool Load] \n- Status: 0");
567     return 0;
568   }
569   catch (...) {
570     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
          "[CarPool Load] \n- Status: 0xBF");
```

```
571     return 0xBF;
572   }
573 }
574
575 /**
576  * Saves the CarPool object to an output stream.
577  *
578  * @param os The output stream to save the CarPool object to.
579  * @return Returns 0 if the CarPool object is successfully saved, else
        an error code:
580  *     - 0xC0: If the output stream is not valid.
581  *     - 0xCF: If an unknown exception occurs during the saving process
        .
582  */
583 int CarPool::save(std::ostream &os) const {
584   if (!os){
585     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
          "[CarPool Save] \n- Status: 0xC0");
586     return 0xC0;}
587   try {
588     json save_json_obj;
589     for (auto it = carpool_byid.begin(); it != carpool_byid.end(); it
          ++) {
590       json car_json_obj;
591       car_json_obj["id"] = it->second.getId();
592       car_json_obj["type"] = it->second.getType();
593       car_json_obj["owner"] = it->second.getOwner();
594       car_json_obj["color"] = it->second.getColor();
595       car_json_obj["year"] = it->second.getYear();
596       car_json_obj["img_path"] = it->second.getImagePath();
597       save_json_obj[it->first] = car_json_obj;
598     }
599     os << save_json_obj.dump(4);
600     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG,
          "[CarPool Save] \n- Status: 0");
601     return 0;
602   }
603   catch (...) {
604     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
          "[CarPool Save] \n- Status: 0xCF");
605     return 0xCF;
```

```
606   }
607 }
608
609 /**
610  * Retrieves a list of cars in the carpool.
611  *
612  * This function returns a vector containing all the cars in the
        carpool.
613  *
614  * @return A vector of Car objects representing the cars in the carpool
        .
615  */
616 std::vector<Car> CarPool::list() const {
617   std::vector<Car> cars;
618   for (auto it = carpool_byid.begin(); it != carpool_byid.end(); it++)
619     cars.push_back(it->second);
620   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::DEBUG, "
        [CarPool List] \n- Status: 0");
621   return cars;
622 }
623
624 /**
625  * @brief Compares two CarPool objects for equality.
626  *
627  * This function compares two CarPool objects for equality by comparing
        the size of the carpool and the cars in the carpool.
628  *
629  * @param cp The CarPool object to compare with.
630  * @return True if the CarPool objects are equal, otherwise false.
631  */
632 bool CarPool::operator==(const CarPool &cp) const {
633   return sz == cp.sz && cp.carpool_byid == cp.carpool_byid &&
634         carpool_bycolor == cp.carpool_bycolor && carpool_bytype == cp.
            carpool_bytype &&
635         carpool_byowner == cp.carpool_byowner;
636 }
637
638 /**
639  * @brief Compares two CarPool objects for inequality.
640  *
641  * This function compares two CarPool objects for inequality by
```

```
     comparing the size of the carpool and the cars in the carpool.
642  *
643  * @param cp The CarPool object to compare with.
644  * @return True if the CarPool objects are not equal, otherwise false.
645  */
646 bool CarPool::operator!=(const CarPool &cp) const {
647   return sz != cp.sz || cp.carpool_byid != cp.carpool_byid ||
648         carpool_bycolor != cp.carpool_bycolor || carpool_bytype != cp.
              carpool_bytype ||
649         carpool_byowner != cp.carpool_byowner;
650 }
651
652 /**
653  * @brief Compares two CarPool objects for less than.
654  *
655  * This function compares two CarPool objects for less than by
       comparing the size of the carpool.
656  *
657  * @param cp The CarPool object to compare with.
658  * @return True if the size of the carpool is less than the size of the
       carpool in the provided CarPool object, otherwise false.
659  */
660 CarPool &CarPool::operator=(const CarPool &cp) {
661   sz = cp.sz;
662   carpool_byid = cp.carpool_byid;
663   carpool_bycolor = cp.carpool_bycolor;
664   carpool_bytype = cp.carpool_bytype;
665   carpool_byowner = cp.carpool_byowner;
666   return *this;
667 }
```

src/client-main.cpp

```
1  /**
2   * @file src/client-main.cpp
3   * @brief Main entry of the client program
4   *
5   * @details
6   * This file contains the main entry of the client program.
7   * The main function starts the logging system and the client program.
8   *
9   * @author donghy23@mails.tsinghua.edu.cn
```

```
 * @version 1.0
 */

#include <QApplication>
#include "carinfo-manager/client-window.hpp"
#include "carinfo-manager/log.hpp"

int main(int argc, char *argv[]) {
  // start logging
  MyLogger::registerLogger("carinfo-manager-logger",
                MyLogger::LOG_LEVEL::DEBUG,
                MyLogger::LOG_TYPE::CONSOLE | MyLogger::LOG_TYPE::FILE,
                "log/carinfo-manager-client.log");

  // start client
  ClientMain clientMain(argc, argv);
  return clientMain.run();
}
```

src/hash.cpp

```
/**
 * @file src/hash.cpp
 * @brief Implementation of class Hash for hashing data with SHA-256 algorithm
 *
 * @details
 * This file contains the implementation of the Hash class, which provides functionality for hashing data using the SHA-256 algorithm.
 * It includes methods for performing various operations required for the SHA-256 algorithm, such as message scheduling, choice, majority, and rounds.
 * The class also provides methods for setting the data and salt values, computing the hash, and retrieving the computed hash value.
 *
 * @author donghy23@mails.tsinghua.edu.cn & kibonga@github.com
 * @version 1.0
 */

#include "carinfo-manager/hash.hpp"
#include <iomanip>
```

```
16 #include <sstream>
17 #include <vector>
18
19 const int Hash::a = 0, Hash::b = 1, Hash::c = 2, Hash::d = 3;
20 const int Hash::e = 4, Hash::f = 5, Hash::g = 6, Hash::h = 7;
21 const uint32_t Hash::K[64] = {
22   0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0
       x59f111f1,
23   0x923f82a4, 0xab1c5ed5, 0xd807aa98, 0x12835b01, 0x243185be, 0
       x550c7dc3,
24   0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174, 0xe49b69c1, 0
       xefbe4786,
25   0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0
       x76f988da,
26   0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0
       xd5a79147,
27   0x06ca6351, 0x14292967, 0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0
       x53380d13,
28   0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85, 0xa2bfe8a1, 0
       xa81a664b,
29   0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0
       x106aa070,
30   0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0
       x4ed8aa4a,
31   0x5b9cca4f, 0x682e6ff3, 0x748f82ee, 0x78a5636f, 0x84c87814, 0
       x8cc70208,
32   0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
33 };
34
35 uint32_t Hash::right_rotate(uint32_t x, uint32_t n) {
36   return (x >> n) | (x << (32 - n));
37 }
38
39 uint32_t Hash::little_sigma_0(uint32_t x) {
40   return right_rotate(x, 7) ^ right_rotate(x, 18) ^ (x >> 3);
41 }
42
43 uint32_t Hash::little_sigma_1(uint32_t x) {
44   return right_rotate(x, 17) ^ right_rotate(x, 19) ^ (x >> 10);
45 }
46
```

```
47 uint32_t Hash::big_sigma_0(uint32_t x) {
48   return right_rotate(x, 2) ^ right_rotate(x, 13) ^ right_rotate(x, 22)
       ;
49 }
50
51 uint32_t Hash::big_sigma_1(uint32_t x) {
52   return right_rotate(x, 6) ^ right_rotate(x, 11) ^ right_rotate(x, 25)
       ;
53 }
54
55 void Hash::message_schedule(uint32_t (&W)[64], uint8_t (&block)[64]) {
56   for (int i = 0; i < 16; i++) {
57     W[i] = (block[i * 4] << 24) | (block[i * 4 + 1] << 16) |
58          (block[i * 4 + 2] << 8) | (block[i * 4 + 3]);
59   }
60
61   for (int i = 16; i < 64; i++) {
62     W[i] = little_sigma_1(W[i - 2]) + W[i - 7] + little_sigma_0(W[i -
         15]) +
63           W[i - 16];
64   }
65 }
66
67 uint32_t Hash::choice(uint32_t x, uint32_t y, uint32_t z) {
68   return (x & y) ^ (~x & z);
69 }
70
71 uint32_t Hash::majority(uint32_t x, uint32_t y, uint32_t z) {
72   return (x & y) ^ (x & z) ^ (y & z);
73 }
74
75 void Hash::round(uint32_t (&H)[8],
76          uint32_t round_constant,
77          uint32_t schedule_word) {
78   uint32_t T1 = H[h] + big_sigma_1(H[e]) + choice(H[e], H[f], H[g]) +
79           round_constant + schedule_word;
80
81   uint32_t T2 = big_sigma_0(H[a]) + majority(H[a], H[b], H[c]);
82
83   for (int i = 7; i > 0; i--) {
84     H[i] = H[i - 1];
```

```
85    }
86
87    H[a] = T1 + T2;
88    H[e] += T1;
89  }
90
91  void Hash::compress_block(uint32_t (&H)[8], uint8_t (&block)[64]) {
92    uint32_t W[64];
93    uint32_t h[8];
94
95    message_schedule(W, block);
96
97    for (int i = 0; i < 8; i++)
98      h[i] = H[i];
99    for (int i = 0; i < 64; i++)
100     round(h, W[i], K[i]);
101   for (int i = 0; i < 8; i++)
102     H[i] += h[i];
103 }
104
105 /**
106  * Calculates the SHA-256 hash of the data combined with the salt.
107  * The resulting hash is stored in the hash member variable.
108  */
109 void Hash::sha256() {
110   uint32_t H[8] = {0x6a09e667,
111             0xbb67ae85,
112             0x3c6ef372,
113             0xa54ff53a,
114             0x510e527f,
115             0x9b05688c,
116             0x1f83d9ab,
117             0x5be0cd19};
118   std::string m = data + salt;
119   size_t size = m.size();
120   size_t l = (size * 8);
121   std::vector<uint8_t> message;
122
123   size_t N = 1;
124   if ((l + 64) > 512) {
125     N += ((l + 64) / 512);
```

```
126   }
127
128   int k = 0;
129   for (size_t i = l; i < ((N * 512) - 8 - 64); i += 8) {
130     k++;
131   }
132
133   message.reserve(size + k + 1 + 8);
134   copy(m.c_str(), m.c_str() + size, back_inserter(message));
135   message.push_back(128);
136   message.insert(message.end(), k, 0);
137
138   for (int i = 0; i < 8; ++i) {
139     message.push_back(
140       ((((unsigned long long)l) >> (56 - (8 * i))) & 0xFFu));
141   }
142
143   uint8_t block[64];
144   for (int i = 0; i < N; i++) {
145     memcpy(block, &message[i * 64], 64);
146     compress_block(H, block);
147   }
148
149   std::stringstream ss;
150   for (size_t i = 0; i < 8; ++i) {
151     ss << std::hex << std::setw(8) << std::setfill('0') << H[i];
152   }
153
154   hash = ss.str();
155 }
156
157 Hash::Hash() : data(""), salt(""), hash("") {}
158
159 Hash::Hash(std::string data, std::string salt)
160   : data(data), salt(salt), hash("") {
161   sha256();
162 }
163
164 Hash::Hash(const Hash &hashObj)
165   : data(hashObj.data), salt(hashObj.salt), hash(hashObj.hash) {}
166
```

```
167 Hash::~Hash() {}
168
169 /**
170  * @brief Sets the data for the Hash object.
171  *
172  * This function sets the data for the Hash object. The data parameter
        is used to update the data member of the object.
173  * After setting the data, the function calls the sha256() function to
        compute the SHA-256 hash of the data.
174  *
175  * @param data The data to be set for the Hash object.
176  */
177 void Hash::setData(std::string data) {
178   this->data = data;
179   sha256();
180 }
181
182 /**
183  * @brief Sets the salt value for the Hash object.
184  *
185  * This function sets the salt value for the Hash object. The salt
        value is used in the hashing process
186  * to add an extra layer of security to the hashed data.
187  *
188  * @param salt The salt value to be set.
189  */
190 void Hash::setSalt(std::string salt) {
191   this->salt = salt;
192   sha256();
193 }
194
195 /**
196  * @brief Retrieves the hash value.
197  *
198  * This function returns the hash value stored in the Hash object.
199  *
200  * @return The hash value.
201  */
202 std::string Hash::getHash() {
203   return hash;
204 }
```

src/httphandler-client.cpp

```cpp
/**
 * @file src/httphandler-client.cpp
 * @brief Implementation of class ClientHttpHandler
 *
 * @details
 * This file contains the implementation of the ClientHttpHandler class.
 * The ClientHttpHandler class is used to handle HTTP requests from the client side.
 * It sends HTTP requests to the server and receives responses.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include "carinfo-manager/httphandler-client.hpp"
#include <sstream>
#include "carinfo-manager/log.hpp"
using json = nlohmann::json;

ClientHttpHandler::ClientHttpHandler() {}

/**
 * @brief Parse the response content of a HTTP response
 *
 * @param resp The HTTP response
 * @return json The parsed JSON object of the response content
 */
json ClientHttpHandler::parse_resp_content(const httplib::Response &resp) {
  json resp_json_obj;
  try {
    resp_json_obj = json::parse(resp.body);
  }
  catch (json::parse_error &e) {
    resp_json_obj["error"] = "Invalid response content";
  }
  return resp_json_obj;
}

```

```
38 /**
39  * @brief Verify the connection to the server
40  *
41  * @param ip The IP address of the server
42  * @param port The port of the server
43  *
44  * @return bool True if the connection is successful, false otherwise
45  */
46 bool ClientHttpHandler::handler_verify_server(const std::string &ip,
       int port) {
47   httplib::Client client(ip, port);
48   client.set_read_timeout(5);
49   httplib::Result res = client.Get("/test_connection");
50   if (!res) {
51     MyLogger::log("carinfo-manager-logger",
52             MyLogger::LOG_LEVEL::ERROR,
53             "[HTTP Verify Server] Failed to connect to " + ip + ":" +
54               std::to_string(port) +
55               ". \n- Error: " + std::to_string((int)res.error()));
56     return false;
57   }
58   httplib::Response resp = res.value();
59   if (resp.status == 200 && resp.body == "Connection is OK") {
60     MyLogger::log("carinfo-manager-logger",
61             MyLogger::LOG_LEVEL::DEBUG,
62             "[HTTP Verify Server] Connect to " + ip + ":" + std::
                to_string(port) +
63               ". \n- Status Code: " + std::to_string(resp.status) +
64               "\n- Response Body: " + resp.body);
65     return true;
66   }
67   else {
68     MyLogger::log("carinfo-manager-logger",
69             MyLogger::LOG_LEVEL::ERROR,
70             "[HTTP Verify Server] Failed to connect to " + ip + ":" +
71               std::to_string(port) + ". \n- Status Code: " +
72               std::to_string(resp.status) + "\n- Response Body: " +
                  resp.body);
73     return false;
74   }
75 }
```

```
76
77  /**
78   * @brief Verify the account by sending a login request to the server
79   *
80   * @param ip The IP address of the server
81   * @param port The port of the server
82   * @param acc The account to be verified
83   *
84   * @return AccountVerifyResult The result of the account verification
85   */
86  ClientHttpHandler::AccountVerifyResult ClientHttpHandler::handler_login
        (const std::string &ip,
87                                       int port,
88                                       Account &acc) {
89    httplib::Client client(ip, port);
90    client.set_read_timeout(5);
91
92    httplib::MultipartFormDataItems items = {{"username", acc.getUsername
          ()},
93                         {"passwd_hash", acc.getPasswdHash()}};
94    httplib::Result res = client.Post("/login", items);
95    if (!res) {
96      MyLogger::log("carinfo-manager-logger",
97              MyLogger::LOG_LEVEL::ERROR,
98              "[HTTP Login] Failed to connect to " + ip + ":" + std::
                  to_string(port) +
99                ". \n- Error: " + std::to_string((int)res.error()));
100     return AccountVerifyResult::NETWORK_ERROR;
101   }
102   httplib::Response resp = res.value();
103   if (resp.status == 200) {
104     json resp_json_obj = parse_resp_content(resp);
105     if (resp_json_obj.find("error") != resp_json_obj.end()) {
106       MyLogger::log("carinfo-manager-logger",
107               MyLogger::LOG_LEVEL::ERROR,
108               "[HTTP Login] Failed to login to " + ip + ":" + std::
                    to_string(port) +
109                 ". \n- Status: Invalid Response. \n- Error: " +
110                 resp_json_obj["error"].get<std::string>());
111       return AccountVerifyResult::INVALID_RESPONSE;
112     }
```

```
113     else
114       acc.setAccountType(Account::AccountType(resp_json_obj["
              account_type"].get<int>()));
115     MyLogger::log("carinfo-manager-logger",
116             MyLogger::LOG_LEVEL::DEBUG,
117             "[HTTP Login] Login to " + ip + ":" + std::to_string(port)
                  +
118               ". \n- Status Code: " + std::to_string(resp.status) +
119               "\n- Response Body: " + resp.body);
120     return AccountVerifyResult::SUCCESS;
121   }
122   else if (resp.status == 401 && resp.body == "Account Not Found or
        Wrong Password") {
123     MyLogger::log("carinfo-manager-logger",
124             MyLogger::LOG_LEVEL::ERROR,
125             "[HTTP Login] Failed to login to " + ip + ":" + std::
                to_string(port) +
126               ". \n- Status Code: " + std::to_string(resp.status) +
127               "\n- Response Body: " + resp.body);
128     return AccountVerifyResult::ACCOUNT_NOT_FOUND_OR_WRONG_PASSWORD;
129   }
130   else if (resp.status == 400 && resp.body == "Bad Request") {
131     MyLogger::log("carinfo-manager-logger",
132             MyLogger::LOG_LEVEL::ERROR,
133             "[HTTP Login] Failed to login to " + ip + ":" + std::
                to_string(port) +
134               ". \n- Status Code: " + std::to_string(resp.status) +
135               "\n- Response Body: " + resp.body);
136     return AccountVerifyResult::BAD_REQUEST;
137   }
138   else {
139     MyLogger::log("carinfo-manager-logger",
140             MyLogger::LOG_LEVEL::ERROR,
141             "[HTTP Login] Failed to login to " + ip + ":" + std::
                to_string(port) +
142               ". \n- Status Code: " + std::to_string(resp.status) +
143               "\n- Response Body: " + resp.body);
144     return AccountVerifyResult::INVALID_RESPONSE;
145   }
146 }
147
```

```
148 /**
149  * @brief Get the information of a car by sending a request to the
        server
150  *
151  * @param ip The IP address of the server
152  * @param port The port of the server
153  * @param acc The account to be verified
154  * @param car_id The ID of the car
155  * @param car_owner The owner of the car
156  * @param car_color The color of the car
157  * @param car_type The type of the car
158  *
159  * @return HttpResult The result of the HTTP request
160  */
161 ClientHttpHandler::HttpResult ClientHttpHandler::handler_get_carinfo(
       const std::string &ip,
162                                      int port,
163                                      const Account &acc,
164                                      const std::string &car_id,
165                                      const std::string &car_owner,
166                                      const std::string &car_color,
167                                      const std::string &car_type) {
168   httplib::Client client(ip, port);
169   client.set_read_timeout(5);
170
171   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
        ()},
172                         {"passwd_hash", acc.getPasswdHash()},
173                         {"car_id", car_id},
174                         {"car_owner", car_owner},
175                         {"car_color", car_color},
176                         {"car_type", car_type}};
177   httplib::Result res = client.Post("/get_carinfo", items);
178   if (!res) {
179     MyLogger::log("carinfo-manager-logger",
180             MyLogger::LOG_LEVEL::ERROR,
181             "[HTTP Get Car Info] Failed to connect to " + ip + ":" +
182               std::to_string(port) +
183               ". \n- Error: " + std::to_string((int)res.error()));
184     return HttpResult(0, std::to_string((int)(res.error())));
185   }
```

```
186   httplib::Response resp = res.value();
187   if (resp.status == 200) {
188     json resp_json_obj = parse_resp_content(resp);
189     if (resp_json_obj.find("error") != resp_json_obj.end()) {
190       MyLogger::log("carinfo-manager-logger",
191               MyLogger::LOG_LEVEL::ERROR,
192               "[HTTP Get Car Info] Failed to get car info from " + ip +
                    ":" +
193                 std::to_string(port) + ". \n- Status: Invalid Response.
                      \n- Error: " +
194                 resp_json_obj["error"].get<std::string>());
195       return HttpResult(0, resp_json_obj["error"].get<std::string>());
196     }
197     else {
198       MyLogger::log("carinfo-manager-logger",
199               MyLogger::LOG_LEVEL::DEBUG,
200               "[HTTP Get Car Info] Get car info from " + ip + ":" +
201                 std::to_string(port) + ". \n- Status Code: " +
202                 std::to_string(resp.status) + "\n- Response Body: " +
                      resp.body);
203       return HttpResult(resp.status, resp.body, resp_json_obj);
204     }
205   }
206   else {
207     MyLogger::log("carinfo-manager-logger",
208             MyLogger::LOG_LEVEL::ERROR,
209             "[HTTP Get Car Info] Failed to get car info from " + ip + "
                  :" +
210               std::to_string(port) + ". \n- Status Code: " +
211               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
212     return HttpResult(resp.status, resp.body);
213   }
214 }
215
216 /**
217  * @brief Get the image of a car by sending a request to the server
218  *
219  * @param ip The IP address of the server
220  * @param port The port of the server
221  * @param acc The account to be verified
```

```
222  * @param car_img_path The path of the car image
223  *
224  * @return HttpResult The result of the HTTP request
225  */
226 ClientHttpHandler::HttpResult ClientHttpHandler::handler_get_carimg(
       const std::string &ip,
227                                    int port,
228                                    const Account &acc,
229                                    std::string car_img_path) {
230   httplib::Client client(ip, port);
231   client.set_read_timeout(5);
232
233   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
        ()},
234                        {"passwd_hash", acc.getPasswdHash()},
235                        {"car_img_path", car_img_path}};
236   httplib::Result res = client.Post("/get_carimg", items);
237   if (!res) {
238     MyLogger::log("carinfo-manager-logger",
239             MyLogger::LOG_LEVEL::ERROR,
240             "[HTTP Get Car Image] Failed to connect to " + ip + ":" +
241               std::to_string(port) +
242               ". \n- Error: " + std::to_string((int)res.error()));
243     return HttpResult(0, std::to_string((int)(res.error())));
244   }
245   httplib::Response resp = res.value();
246   if (resp.status == 200)
247     MyLogger::log("carinfo-manager-logger",
248             MyLogger::LOG_LEVEL::DEBUG,
249             "[HTTP Get Car Image] Get car image from " + ip + ":" + std
                ::to_string(port) +
250               ". \n- Status Code: " + std::to_string(resp.status));
251   else
252     MyLogger::log("carinfo-manager-logger",
253             MyLogger::LOG_LEVEL::ERROR,
254             "[HTTP Get Car Image] Failed to get car image from " + ip +
                  ":" +
255               std::to_string(port) + ". \n- Status Code: " +
256               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
257   return HttpResult(resp.status, resp.body);
```

```
258 }
259
260 /**
261  * @brief Add a car by sending a request to the server
262  *
263  * @param ip The IP address of the server
264  * @param port The port of the server
265  * @param acc The account to be verified
266  * @param car_id The ID of the car
267  * @param car_owner The owner of the car
268  * @param car_type The type of the car
269  * @param car_color The color of the car
270  * @param car_year The year of the car
271  * @param car_img The image of the car
272  * @param car_img_type The type of the image of the car
273  *
274  * @return HttpResult The result of the HTTP request
275  */
276 ClientHttpHandler::HttpResult ClientHttpHandler::handler_add_car(const
       std::string &ip,
277                                     const int port,
278                                     const Account &acc,
279                                     const std::string &car_id,
280                                     const std::string &car_owner,
281                                     const std::string &car_type,
282                                     const std::string &car_color,
283                                     const int car_year,
284                                     const std::string &car_img,
285                                     const std::string &car_img_type) {
286   httplib::Client client(ip, port);
287   client.set_read_timeout(5);
288
289   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
          ()},
290                         {"passwd_hash", acc.getPasswdHash()},
291                         {"car_id", car_id},
292                         {"car_owner", car_owner},
293                         {"car_type", car_type},
294                         {"car_color", car_color},
295                         {"car_year", std::to_string(car_year)},
296                         {"car_img", car_img},
```

```
297                         {"car_img_type", car_img_type}};
298   httplib::Result res = client.Post("/add_car", items);
299   if (!res) {
300     MyLogger::log("carinfo-manager-logger",
301             MyLogger::LOG_LEVEL::ERROR,
302             "[HTTP Add Car] Failed to connect to " + ip + ":" + std::
                  to_string(port) +
303               ". \n- Error: " + std::to_string((int)res.error()));
304     return HttpResult(0, std::to_string((int)(res.error())));
305   }
306   httplib::Response resp = res.value();
307   if (resp.status == 200)
308     MyLogger::log("carinfo-manager-logger",
309             MyLogger::LOG_LEVEL::DEBUG,
310             "[HTTP Add Car] Add car to " + ip + ":" + std::to_string(
                  port) +
311               ". \n- Status Code: " + std::to_string(resp.status) +
312               "\n- Response Body: " + resp.body);
313   else
314     MyLogger::log("carinfo-manager-logger",
315             MyLogger::LOG_LEVEL::ERROR,
316             "[HTTP Add Car] Failed to add car to " + ip + ":" + std::
                  to_string(port) +
317               ". \n- Status Code: " + std::to_string(resp.status) +
318               "\n- Response Body: " + resp.body);
319   return HttpResult(resp.status, resp.body);
320 }
321
322 /**
323  * @brief Remove a car by sending a request to the server
324  *
325  * @param ip The IP address of the server
326  * @param port The port of the server
327  * @param acc The account to be verified
328  * @param car_id The ID of the car
329  *
330  * @return HttpResult The result of the HTTP request
331  */
332 ClientHttpHandler::HttpResult ClientHttpHandler::handler_remove_car(
      const std::string &ip,
333                                     const int port,
```

```
334                                     const Account &acc,
335                                     const std::string &car_id) {
336   httplib::Client client(ip, port);
337   client.set_read_timeout(5);
338 
339   httplib::MultipartFormDataItems items = {
340     {"username", acc.getUsername()}, {"passwd_hash", acc.getPasswdHash
          ()}, {"car_id", car_id}};
341   httplib::Result res = client.Post("/remove_car", items);
342   if (!res) {
343     MyLogger::log("carinfo-manager-logger",
344             MyLogger::LOG_LEVEL::ERROR,
345             "[HTTP Remove Car] Failed to connect to " + ip + ":" + std
                ::to_string(port) +
346               ". \n- Error: " + std::to_string((int)res.error()));
347     return HttpResult(0, std::to_string((int)(res.error())));
348   }
349   httplib::Response resp = res.value();
350   if (resp.status == 200)
351     MyLogger::log("carinfo-manager-logger",
352             MyLogger::LOG_LEVEL::DEBUG,
353             "[HTTP Remove Car] Remove car from " + ip + ":" + std::
                to_string(port) +
354               ". \n- Status Code: " + std::to_string(resp.status) +
355               "\n- Response Body: " + resp.body);
356   else
357     MyLogger::log("carinfo-manager-logger",
358             MyLogger::LOG_LEVEL::ERROR,
359             "[HTTP Remove Car] Failed to remove car from " + ip + ":" +
360               std::to_string(port) + ". \n- Status Code: " +
361               std::to_string(resp.status) + "\n- Response Body: " +
                  resp.body);
362   return HttpResult(resp.status, resp.body);
363 }
364 
365 /**
366  * @brief Update a car by sending a request to the server
367  *
368  * @param ip The IP address of the server
369  * @param port The port of the server
370  * @param acc The account to be verified
```

```
371   * @param original_car_id The original ID of the car
372   * @param new_car_id The new ID of the car
373   * @param new_car_owner The new owner of the car
374   * @param new_car_type The new type of the car
375   * @param new_car_color The new color of the car
376   * @param new_car_year The new year of the car
377   * @param new_car_img The new image of the car
378   * @param new_car_img_type The new type of the image of the car
379   *
380   * @return HttpResult The result of the HTTP request
381   */
382 ClientHttpHandler::HttpResult ClientHttpHandler::handler_update_car(
383     const std::string &ip,
384     const int port,
385     const Account &acc,
386     const std::string &original_car_id,
387     const std::string &new_car_id,
388     const std::string &new_car_owner,
389     const std::string &new_car_type,
390     const std::string &new_car_color,
391     const int new_car_year,
392     const std::string &new_car_img,
393     const std::string &new_car_img_type) {
394     httplib::Client client(ip, port);
395     client.set_read_timeout(5);
396 
397     httplib::MultipartFormDataItems items = {{"username", acc.getUsername
        ()},
398                                              {"passwd_hash", acc.getPasswdHash()},
399                                              {"original_car_id", original_car_id},
400                                              {"new_car_id", new_car_id},
401                                              {"new_car_owner", new_car_owner},
402                                              {"new_car_type", new_car_type},
403                                              {"new_car_color", new_car_color},
404                                              {"new_car_year", std::to_string(new_car_year)},
405                                              {"new_car_img", new_car_img},
406                                              {"new_car_img_type", new_car_img_type}};
407     httplib::Result res = client.Post("/update_car", items);
408     if (!res) {
409       MyLogger::log("carinfo-manager-logger",
410               MyLogger::LOG_LEVEL::ERROR,
```

```
411             "[HTTP Update Car] Failed to connect to " + ip + ":" + std
                    ::to_string(port) +
412                 ". \n- Error: " + std::to_string((int)res.error()));
413     return HttpResult(0, std::to_string((int)(res.error())));
414   }
415   httplib::Response resp = res.value();
416   MyLogger::log("carinfo-manager-logger",
417           MyLogger::LOG_LEVEL::DEBUG,
418           "[HTTP Update Car] Update car from " + ip + ":" + std::
                to_string(port) +
419             ". \n- Status Code: " + std::to_string(resp.status) +
420             "\n- Response Body: " + resp.body);
421   return HttpResult(resp.status, resp.body);
422 }
423
424 /**
425  * @brief Search an account by sending a request to the server
426  *
427  * @param ip The IP address of the server
428  * @param port The port of the server
429  * @param acc The account to be verified
430  * @param search_username The username to be searched
431  *
432  * @return HttpResult The result of the HTTP request
433  */
434 ClientHttpHandler::HttpResult ClientHttpHandler::handler_search_account
       (
435   const std::string &ip, const int port, const Account &acc, const std
        ::string &search_username) {
436   httplib::Client client(ip, port);
437   client.set_read_timeout(5);
438
439   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
        ()},
440                       {"passwd_hash", acc.getPasswdHash()},
441                       {"target_username", search_username}};
442   httplib::Result res = client.Post("/get_accountinfo", items);
443   if (!res) {
444     MyLogger::log("carinfo-manager-logger",
445             MyLogger::LOG_LEVEL::ERROR,
446             "[HTTP Search Account] Failed to connect to " + ip + ":" +
```

```
447                 std::to_string(port) +
448                 ". \n- Error: " + std::to_string((int)res.error()));
449       return HttpResult(0, std::to_string((int)(res.error())));
450     }
451     httplib::Response resp = res.value();
452     if (resp.status == 200) {
453       json resp_json_obj = parse_resp_content(resp);
454       if (resp_json_obj.find("error") != resp_json_obj.end()) {
455         MyLogger::log("carinfo-manager-logger",
456                 MyLogger::LOG_LEVEL::ERROR,
457                 "[HTTP Search Account] Failed to search account from " +
                      ip + ":" +
458                   std::to_string(port) + ". \n- Status: Invalid Response.
                        \n- Error: " +
459                   resp_json_obj["error"].get<std::string>());
460         return HttpResult(0, resp_json_obj["error"].get<std::string>());
461       }
462       else {
463         MyLogger::log("carinfo-manager-logger",
464                 MyLogger::LOG_LEVEL::DEBUG,
465                 "[HTTP Search Account] Search account from " + ip + ":" +
466                   std::to_string(port) + ". \n- Status Code: " +
467                   std::to_string(resp.status) + "\n- Response Body: " +
                        resp.body);
468         return HttpResult(resp.status, resp.body, resp_json_obj);
469       }
470     }
471     else {
472       MyLogger::log("carinfo-manager-logger",
473               MyLogger::LOG_LEVEL::ERROR,
474               "[HTTP Search Account] Failed to search account from " + ip
                    + ":" +
475                 std::to_string(port) + ". \n- Status Code: " +
476                 std::to_string(resp.status) + "\n- Response Body: " +
                      resp.body);
477       return HttpResult(resp.status, resp.body);
478     }
479   }
480
481   /**
482    * @brief Get all accounts by sending a request to the server
```

```
483  *
484  * @param ip The IP address of the server
485  * @param port The port of the server
486  * @param acc The account to be verified
487  *
488  * @return HttpResult The result of the HTTP request
489  */
490 ClientHttpHandler::HttpResult ClientHttpHandler::
       handler_get_all_accounts(const std::string &ip,
491                                        const int port,
492                                        const Account &acc) {
493   httplib::Client client(ip, port);
494   client.set_read_timeout(5);
495
496   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
          ()},
497                         {"passwd_hash", acc.getPasswdHash()}};
498   httplib::Result res = client.Post("/get_all_account", items);
499   if (!res) {
500     MyLogger::log("carinfo-manager-logger",
501             MyLogger::LOG_LEVEL::ERROR,
502             "[HTTP Get All Accounts] Failed to connect to " + ip + ":"
                  +
503               std::to_string(port) +
504               ". \n- Error: " + std::to_string((int)res.error()));
505     return HttpResult(0, std::to_string((int)(res.error())));
506   }
507   httplib::Response resp = res.value();
508   if (resp.status == 200) {
509     json resp_json_obj = parse_resp_content(resp);
510     if (resp_json_obj.find("error") != resp_json_obj.end()) {
511       MyLogger::log("carinfo-manager-logger",
512               MyLogger::LOG_LEVEL::ERROR,
513               "[HTTP Get All Accounts] Failed to get all accounts from
                    " + ip + ":" +
514                 std::to_string(port) + ". \n- Status: Invalid Response.
                      \n- Error: " +
515                 resp_json_obj["error"].get<std::string>());
516       return HttpResult(0, resp_json_obj["error"].get<std::string>());
517     }
518     else {
```

```
519         MyLogger::log("carinfo-manager-logger",
520                 MyLogger::LOG_LEVEL::DEBUG,
521                 "[HTTP Get All Accounts] Get all accounts from " + ip + "
                        :" +
522                   std::to_string(port) + ". \n- Status Code: " +
523                   std::to_string(resp.status) + "\n- Response Body: " +
                          resp.body);
524         return HttpResult(resp.status, resp.body, resp_json_obj);
525       }
526     }
527     else {
528       MyLogger::log("carinfo-manager-logger",
529               MyLogger::LOG_LEVEL::ERROR,
530               "[HTTP Get All Accounts] Failed to get all accounts from "
                        + ip + ":" +
531                 std::to_string(port) + ". \n- Status Code: " +
532                 std::to_string(resp.status) + "\n- Response Body: " +
                        resp.body);
533       return HttpResult(resp.status, resp.body);
534     }
535 }
536
537 /**
538  * @brief Add an account by sending a request to the server
539  *
540  * @param ip The IP address of the server
541  * @param port The port of the server
542  * @param acc The account to be verified
543  * @param new_username The new username of the account
544  * @param new_passwd_hash The new password hash of the account
545  * @param new_accounttype The new account type of the account
546  *
547  * @return HttpResult The result of the HTTP request
548  */
549 ClientHttpHandler::HttpResult ClientHttpHandler::handler_add_account(
550     const std::string &ip,
551     const int port,
552     const Account &acc,
553     const std::string &new_username,
554     const std::string &new_passwd_hash,
555     const Account::AccountType new_accounttype) {
```

```
556   httplib::Client client(ip, port);
557   client.set_read_timeout(5);
558
559   httplib::MultipartFormDataItems items = {
560     {"username", acc.getUsername()},
561     {"passwd_hash", acc.getPasswdHash()},
562     {"target_username", new_username},
563     {"target_passwd_hash", new_passwd_hash},
564     {"target_account_type", std::to_string((int)new_accounttype)}};
565   httplib::Result res = client.Post("/add_account", items);
566   if (!res) {
567     MyLogger::log("carinfo-manager-logger",
568             MyLogger::LOG_LEVEL::ERROR,
569             "[HTTP Add Account] Failed to connect to " + ip + ":" + std
                ::to_string(port) +
570               ". \n- Error: " + std::to_string((int)res.error()));
571     return HttpResult(0, std::to_string((int)(res.error())));
572   }
573   httplib::Response resp = res.value();
574   if (resp.status == 200)
575     MyLogger::log("carinfo-manager-logger",
576             MyLogger::LOG_LEVEL::DEBUG,
577             "[HTTP Add Account] Add account to " + ip + ":" + std::
                to_string(port) +
578               ". \n- Status Code: " + std::to_string(resp.status) +
579               "\n- Response Body: " + resp.body);
580   else
581     MyLogger::log("carinfo-manager-logger",
582             MyLogger::LOG_LEVEL::ERROR,
583             "[HTTP Add Account] Failed to add account to " + ip + ":" +
584               std::to_string(port) + ". \n- Status Code: " +
585               std::to_string(resp.status) + "\n- Response Body: " +
                  resp.body);
586   return HttpResult(resp.status, resp.body);
587 }
588
589 /**
590  * @brief Remove an account by sending a request to the server
591  *
592  * @param ip The IP address of the server
593  * @param port The port of the server
```

```
 * @param acc The account to be verified
 * @param target_username The username of the account to be removed
 *
 * @return HttpResult The result of the HTTP request
 */
ClientHttpHandler::HttpResult ClientHttpHandler::handler_remove_account(
  const std::string &ip, const int port, const Account &acc, const std::string &target_username) {
  httplib::Client client(ip, port);
  client.set_read_timeout(5);

  httplib::MultipartFormDataItems items = {{"username", acc.getUsername()},
                      {"passwd_hash", acc.getPasswdHash()},
                      {"target_username", target_username}};
  httplib::Result res = client.Post("/remove_account", items);
  if (!res) {
    MyLogger::log("carinfo-manager-logger",
            MyLogger::LOG_LEVEL::ERROR,
            "[HTTP Remove Account] Failed to connect to " + ip + ":" +
              std::to_string(port) +
              ". \n- Error: " + std::to_string((int)res.error()));
    return HttpResult(0, std::to_string((int)(res.error())));
  }
  httplib::Response resp = res.value();
  if (resp.status == 200)
    MyLogger::log("carinfo-manager-logger",
            MyLogger::LOG_LEVEL::DEBUG,
            "[HTTP Remove Account] Remove account from " + ip + ":" +
              std::to_string(port) + ". \n- Status Code: " +
              std::to_string(resp.status) + "\n- Response Body: " +
                resp.body);
  else
    MyLogger::log("carinfo-manager-logger",
            MyLogger::LOG_LEVEL::ERROR,
            "[HTTP Remove Account] Failed to remove account from " + ip
                + ":" +
              std::to_string(port) + ". \n- Status Code: " +
              std::to_string(resp.status) + "\n- Response Body: " +
                resp.body);
```

```
629   return HttpResult(resp.status, resp.body);
630 }
631
632 /**
633  * @brief Update an account by sending a request to the server
634  *
635  * @param ip The IP address of the server
636  * @param port The port of the server
637  * @param acc The account to be verified
638  * @param target_username The username of the account to be updated
639  * @param new_username The new username of the account
640  * @param new_passwd_hash The new password hash of the account
641  * @param new_accounttype The new account type of the account
642  *
643  * @return HttpResult The result of the HTTP request
644  */
645 ClientHttpHandler::HttpResult ClientHttpHandler::handler_update_account
      (
646   const std::string &ip,
647   const int port,
648   const Account &acc,
649   const std::string &target_username,
650   const std::string &new_username,
651   const std::string &new_passwd_hash,
652   const Account::AccountType new_accounttype) {
653   httplib::Client client(ip, port);
654   client.set_read_timeout(5);
655
656   httplib::MultipartFormDataItems items = {
657     {"username", acc.getUsername()},
658     {"passwd_hash", acc.getPasswdHash()},
659     {"target_username", target_username},
660     {"new_username", new_username},
661     {"new_passw_hash", new_passwd_hash},
662     {"new_account_type", std::to_string((int)new_accounttype)}};
663   httplib::Result res = client.Post("/update_account", items);
664   if (!res) {
665     MyLogger::log("carinfo-manager-logger",
666             MyLogger::LOG_LEVEL::ERROR,
667             "[HTTP Update Account] Failed to connect to " + ip + ":" +
668               std::to_string(port) +
```

```
669                 ". \n- Error: " + std::to_string((int)res.error()));
670     return HttpResult(0, std::to_string((int)(res.error())));
671   }
672   httplib::Response resp = res.value();
673   if (resp.status == 200)
674     MyLogger::log("carinfo-manager-logger",
675             MyLogger::LOG_LEVEL::DEBUG,
676             "[HTTP Update Account] Update account from " + ip + ":" +
677               std::to_string(port) + ". \n- Status Code: " +
678               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
679   else
680     MyLogger::log("carinfo-manager-logger",
681             MyLogger::LOG_LEVEL::ERROR,
682             "[HTTP Update Account] Failed to update account from " + ip
                  + ":" +
683               std::to_string(port) + ". \n- Status Code: " +
684               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
685   return HttpResult(resp.status, resp.body);
686 }
687
688 /**
689  * @brief Change the password of an account by sending a request to the
        server
690  *
691  * @param ip The IP address of the server
692  * @param port The port of the server
693  * @param acc The account to be verified
694  * @param new_passwd_hash The new password hash of the account
695  *
696  * @return HttpResult The result of the HTTP request
697  */
698 ClientHttpHandler::HttpResult ClientHttpHandler::
     handler_change_password(
699   const std::string &ip, const int port, const Account &acc, const std
        ::string &new_passwd_hash) {
700   httplib::Client client(ip, port);
701   client.set_read_timeout(5);
702
703   httplib::MultipartFormDataItems items = {{"username", acc.getUsername
```

```
    ()},
704                         {"old_passwd_hash", acc.getPasswdHash()},
705                         {"new_passwd_hash", new_passwd_hash}};
706   httplib::Result res = client.Post("/change_password", items);
707   if (!res) {
708     MyLogger::log("carinfo-manager-logger",
709             MyLogger::LOG_LEVEL::ERROR,
710             "[HTTP Change Password] Failed to connect to " + ip + ":" +
711               std::to_string(port) +
712               ". \n- Error: " + std::to_string((int)res.error()));
713     return HttpResult(0, std::to_string((int)(res.error())));
714   }
715   httplib::Response resp = res.value();
716   if (resp.status == 200)
717     MyLogger::log("carinfo-manager-logger",
718             MyLogger::LOG_LEVEL::DEBUG,
719             "[HTTP Change Password] Change password from " + ip + ":" +
720               std::to_string(port) + ". \n- Status Code: " +
721               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
722   else
723     MyLogger::log("carinfo-manager-logger",
724             MyLogger::LOG_LEVEL::ERROR,
725             "[HTTP Change Password] Failed to change password from " +
                 ip + ":" +
726               std::to_string(port) + ". \n- Status Code: " +
727               std::to_string(resp.status) + "\n- Response Body: " +
                    resp.body);
728   return HttpResult(resp.status, resp.body);
729   return HttpResult(resp.status, resp.body);
730 }
```

src/httphandler-server.cpp

```
1  /**
2   * @file src/httphandler-server.cpp
3   * @brief Implementation of class ServerHttpHandler for the server of
        carinfo-manager project
4   *
5   * @details
6   * This file contains the implementation of the ServerHttpHandler class
        , which is responsible for handling various HTTP requests
```

```
7   * in the carinfo-manager server. It includes functions for handling
        test connection requests, login requests, changing passwords,
8   * retrieving car information, retrieving car images, and adding cars
        to the system.
9   * The ServerHttpHandler class interacts with the AccountPool and
        CarPool classes to perform the necessary operations for each
        request.
10  * It also uses the httplib library for handling HTTP requests and
        responses.
11  *
12  * @author donghy23@mails.tsinghua.edu.cn
13  * @version 1.0
14  */
15
16  #include "carinfo-manager/httphandler-server.hpp"
17  #include <sstream>
18  #include "carinfo-manager/base64.hpp"
19  #include "carinfo-manager/log.hpp"
20  #include "json/json.hpp"
21
22  using json = nlohmann::json;
23
24  /**
25   * @brief Construct a new ServerHttpHandler object
26   *
27   * @param accountpool The AccountPool object for managing user accounts
28   * @param carpool The CarPool object for managing car information
29   * @param imgDir The directory path for storing car images
30   */
31  ServerHttpHandler::ServerHttpHandler(AccountPool &accountpool, CarPool
        &carpool, std::string imgDir)
32    : accountpool(accountpool), carpool(carpool), imgDir(imgDir) {}
33
34  /**
35   * @brief Parse the POST body of an HTTP request
36   *
37   * This function parses the POST body of an HTTP request and returns
        the content as a JSON object.
38   *
39   * @param req The HTTP request object
40   * @return json The parsed JSON object containing the POST body content
```

```
41  */
42 json ServerHttpHandler::parse_post_body(const httplib::Request &req)
       const {
43   json j;
44   for (const auto &file : req.files)
45     j[file.first] = file.second.content;
46   return j;
47 }
48
49 /**
50  * Handles the test connection request.
51  *
52  * This function is responsible for handling the test connection
       request and
53  * returning a response with a status code of 200 and a content of "
       Connection is OK".
54  *
55  * @param req The HTTP request object.
56  * @param res The HTTP response object.
57  */
58 void ServerHttpHandler::handler_test_connection(const httplib::Request
       &req,
59                                                 httplib::Response &res) const {
60   std::string ip = req.remote_addr;
61   int port = req.remote_port;
62   res.set_content("Connection is OK", "text/plain");
63   res.status = 200;
64   MyLogger::log(
65     "carinfo-manager-logger",
66     MyLogger::LOG_LEVEL::INFO,
67     "[HTTP Test Connection] from " + ip + ":" + std::to_string(port) +
         ". Status: 200 (OK)");
68 }
69
70 /**
71  * Handles the HTTP login request from the client.
72  *
73  * @param req The HTTP request object containing the login parameters.
74  * @param res The HTTP response object to send back to the client.
75  */
76 void ServerHttpHandler::handler_login(const httplib::Request &req,
```

```
    httplib::Response &res) const {
77  std::string ip = req.remote_addr;
78  int port = req.remote_port;
79  json params = parse_post_body(req);
80  if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end()) {
81    res.set_content("Bad Request", "text/plain");
82    res.status = 400;
83    MyLogger::log(
84      "carinfo-manager-logger",
85      MyLogger::LOG_LEVEL::WARN,
86      "[HTTP Login] from " + ip + ":" + std::to_string(port) + ".
          Status: 400 (Bad Request)");
87    return;
88  }
89  std::string username = std::string(params["username"]);
90  std::string passwd = std::string(params["passwd_hash"]);
91
92  auto result = accountpool.verifyAccount(username, passwd);
93  if (result == AccountPool::AccountVerifyResult::SUCCESS) {
94    Account acc = accountpool.getAccount(username);
95    if (acc == Account::NULL_ACCOUNT) {
96      res.set_content("Internal Server Error", "text/plain");
97      res.status = 500;
98      MyLogger::log("carinfo-manager-logger",
99              MyLogger::LOG_LEVEL::WARN,
100             "[HTTP Login] from " + ip + ":" + std::to_string(port) +
101               ".\n- Username: " + username + "\n- PasswdHash: " +
                    passwd +
102               "\n- Status: 500. (User Account Not Found)");
103   }
104   else {
105     nlohmann::json j;
106     j["username"] = acc.getUsername();
107     j["passwd_hash"] = acc.getPasswdHash();
108     j["account_type"] = int(acc.getAccountType());
109     res.set_content(j.dump(4), "application/json");
110     res.status = 200;
111     MyLogger::log("carinfo-manager-logger",
112             MyLogger::LOG_LEVEL::INFO,
113             "[HTTP Login] from " + ip + ":" + std::to_string(port) +
```

```
114                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd +
115                 "\n- Status: 200 (OK)");
116     }
117   }
118   else if (result == AccountPool::AccountVerifyResult::
        ACCOUNT_NOT_FOUND ||
119       result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
120     res.set_content("Account Not Found or Wrong Password", "text/plain"
          );
121     res.status = 401;
122     MyLogger::log("carinfo-manager-logger",
123             MyLogger::LOG_LEVEL::WARN,
124             "[HTTP Login] from " + ip + ":" + std::to_string(port) +
125               ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd +
126               "\n- Status: 401 (Account Not Found or Wrong Password)");
127   }
128   else {
129     res.set_content("Internal Server Error", "text/plain");
130     res.status = 500;
131     MyLogger::log("carinfo-manager-logger",
132             MyLogger::LOG_LEVEL::WARN,
133             "[HTTP Login] from " + ip + ":" + std::to_string(port) +
134               ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd +
135               "\n- Status: 500 (Internal Server Error)");
136   }
137 }
138
139 /**
140  * Handles the HTTP request for changing password.
141  *
142  * @param req The HTTP request object.
143  * @param res The HTTP response object.
144  */
145 void ServerHttpHandler::handler_change_password(const httplib::Request
       &req,
146                         httplib::Response &res) {
147   std::string ip = req.remote_addr;
148   int port = req.remote_port;
```

```
149   json params = parse_post_body(req);
150   if (params.find("username") == params.end() || params.find("
          old_passwd_hash") == params.end() ||
151     params.find("new_passwd_hash") == params.end()) {
152     res.set_content("Bad Request", "text/plain");
153     res.status = 400;
154     MyLogger::log("carinfo-manager-logger",
155             MyLogger::LOG_LEVEL::WARN,
156             "[HTTP Change Password] from " + ip + ":" + std::to_string(
                  port) +
157               ". Status: 400 (Bad Request)");
158     return;
159   }
160   std::string username = std::string(params["username"]);
161   std::string old_passwd_hash = std::string(params["old_passwd_hash"]);
162   std::string new_passwd_hash = std::string(params["new_passwd_hash"]);
163
164   try {
165     auto result = accountpool.verifyAccount(username, old_passwd_hash);
166     if (result == AccountPool::AccountVerifyResult::SUCCESS) {
167       Account old_acc = accountpool.getAccount(username);
168       Account new_acc = Account(username, new_passwd_hash, old_acc.
             getAccountType());
169       int status_code = accountpool.updateAccount(old_acc, new_acc);
170       if (status_code == 0) {
171         res.set_content("Password Changed", "text/plain");
172         res.status = 200;
173         MyLogger::log("carinfo-manager-logger",
174                 MyLogger::LOG_LEVEL::INFO,
175                 "[HTTP Change Password] from " + ip + ":" + std::
                      to_string(port) +
176                   ".\n- Username: " + username +
177                   "\n- OldPasswdHash: " + old_passwd_hash +
178                   "\n- NewPasswdHash: " + new_passwd_hash + "\n- Status
                        : 200 (OK)");
179       }
180       else {
181         std::string msg =
182           "Internal Server Error, status code: " + std::to_string(
                  status_code);
183         res.set_content(msg, "text/plain");
```

```
184         res.status = 500;
185         MyLogger::log("carinfo-manager-logger",
186                 MyLogger::LOG_LEVEL::WARN,
187                 "[HTTP Change Password] from " + ip + ":" + std::
                        to_string(port) +
188                   ".\n- Username: " + username + "\n- OldPasswdHash: "
                          +
189                   old_passwd_hash + "\n- NewPasswdHash: " +
                          new_passwd_hash +
190                   "\n- Status: 500 (Internal Server Error) \n- Status
                          Code: " +
191                   std::to_string(status_code));
192       }
193     }
194     else if (result == AccountPool::AccountVerifyResult::
            ACCOUNT_NOT_FOUND ||
195           result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
196       res.set_content("Forbidden", "text/plain");
197       res.status = 403;
198       MyLogger::log(
199         "carinfo-manager-logger",
200         MyLogger::LOG_LEVEL::WARN,
201         "[HTTP Change Password] from " + ip + ":" + std::to_string(port
                ) +
202           ".\n- Username: " + username + "\n- OldPasswdHash: " +
                  old_passwd_hash +
203           "\n- NewPasswdHash: " + new_passwd_hash + "\n- Status: 403 (
                  Forbidden)");
204     }
205     else {
206       res.set_content("Internal Server Error", "text/plain");
207       res.status = 500;
208       MyLogger::log("carinfo-manager-logger",
209               MyLogger::LOG_LEVEL::WARN,
210               "[HTTP Change Password] from " + ip + ":" + std::
                      to_string(port) +
211                 ".\n- Username: " + username + "\n- OldPasswdHash: " +
212                 old_passwd_hash + "\n- NewPasswdHash: " +
                        new_passwd_hash +
213                 "\n- Status: 500 (Internal Server Error)");
214     }
```

```
215   }
216   catch (std::exception &e) {
217     res.set_content("Internal Server Error", "text/plain");
218     res.status = 500;
219     MyLogger::log("carinfo-manager-logger",
220             MyLogger::LOG_LEVEL::WARN,
221             "[HTTP Change Password] from " + ip + ":" + std::to_string(
                port) +
222               ".\n- Username: " + username + "\n- OldPasswdHash: " +
                  old_passwd_hash +
223               "\n- NewPasswdHash: " + new_passwd_hash +
224               "\n- Status: 500 (Internal Server Error) \n- Exception: "
                  + e.what());
225   }
226 }
227
228 /**
229  * Handles the HTTP request for retrieving car information.
230  *
231  * @param req The HTTP request object containing the request parameters
        .
232  * @param res The HTTP response object to be sent back to the client.
233  */
234 void ServerHttpHandler::handler_get_carinfo(const httplib::Request &req
      ,
235                       httplib::Response &res) const {
236   std::string ip = req.remote_addr;
237   int port = req.remote_port;
238   json params = parse_post_body(req);
239   if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end() ||
240     params.find("car_id") == params.end() || params.find("car_owner")
          == params.end() ||
241     params.find("car_color") == params.end() || params.find("car_type")
          == params.end()) {
242     res.set_content("Bad Request", "text/plain");
243     res.status = 400;
244     MyLogger::log("carinfo-manager-logger",
245             MyLogger::LOG_LEVEL::WARN,
246             "[HTTP Get Car Info] from " + ip + ":" + std::to_string(
                port) +
```

```
247                 ". Status: 400 (Bad Request)");
248       return;
249     }
250     std::string username = std::string(params["username"]);
251     std::string passwd_hash = std::string(params["passwd_hash"]);
252     std::string car_id = std::string(params["car_id"]);
253     std::string car_owner = std::string(params["car_owner"]);
254     std::string car_color = std::string(params["car_color"]);
255     std::string car_type = std::string(params["car_type"]);
256     try {
257       auto result = accountpool.verifyAccount(username, passwd_hash);
258       if (result == AccountPool::AccountVerifyResult::SUCCESS) {
259         CarPool cars = carpool.getCar(car_id, car_color, car_owner,
                car_type);
260         std::ostringstream os;
261         int status_code = cars.save(os);
262         if (status_code != 0) {
263           std::string msg =
264             "Internal Server Error, status code: " + std::to_string(
                    status_code);
265           res.set_content(msg, "text/plain");
266           res.status = 500;
267           MyLogger::log("carinfo-manager-logger",
268                   MyLogger::LOG_LEVEL::WARN,
269                   "[HTTP Get Car Info] from " + ip + ":" + std::to_string
                        (port) +
270                    ".\n- Username: " + username + "\n- PasswdHash: " +
                         passwd_hash +
271                    "\n- Status: 500 (Internal Server Error) \n- Status
                         Code: " +
272                    std::to_string(status_code));
273           return;
274         }
275         res.set_content(os.str(), "application/json");
276         res.status = 200;
277         MyLogger::log("carinfo-manager-logger",
278                 MyLogger::LOG_LEVEL::INFO,
279                 "[HTTP Get Car Info] from " + ip + ":" + std::to_string(
                      port) +
280                  ".\n- Username: " + username + "\n- PasswdHash: " +
                       passwd_hash +
```

```
281                 "\n- Status: 200 (OK)");
282     }
283     else if (result == AccountPool::AccountVerifyResult::
            ACCOUNT_NOT_FOUND ||
284             result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
285       res.set_content("Forbidden", "text/plain");
286       res.status = 403;
287       MyLogger::log("carinfo-manager-logger",
288               MyLogger::LOG_LEVEL::WARN,
289               "[HTTP Get Car Info] from " + ip + ":" + std::to_string(
                    port) +
290                 ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
291                 "\n- Status: 403 (Forbidden)");
292     }
293     else {
294       res.set_content("Internal Server Error", "text/plain");
295       res.status = 500;
296       MyLogger::log("carinfo-manager-logger",
297               MyLogger::LOG_LEVEL::WARN,
298               "[HTTP Get Car Info] from " + ip + ":" + std::to_string(
                    port) +
299                 ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
300                 "\n- Status: 500 (Internal Server Error)");
301     }
302   }
303   catch (std::exception &e) {
304     res.set_content("Internal Server Error", "text/plain");
305     res.status = 500;
306     MyLogger::log("carinfo-manager-logger",
307             MyLogger::LOG_LEVEL::WARN,
308             "[HTTP Get Car Info] from " + ip + ":" + std::to_string(
                  port) +
309               ".\n- Username: " + username + "\n- PasswdHash: " +
                    passwd_hash +
310               "\n- Status: 500 (Internal Server Error) \n- Exception: "
                      + e.what());
311   }
312 }
313
```

```
314 /**
315  * Handles the HTTP request for retrieving a car image.
316  *
317  * This function is responsible for handling the POST request to
        retrieve a car image. It verifies the user's account
318  * credentials, checks if the specified car image file exists, and
        returns the image content in base64 format if the
319  * file is found. If the file is not found or if there is an error
        during the account verification process, appropriate
320  * HTTP response status codes and content are set.ww
321  *
322  * @param req The HTTP request object containing the request parameters
        .
323  * @param res The HTTP response object to be populated with the
        response data.
324  */
325 void ServerHttpHandler::handler_get_carimg(const httplib::Request &req,
326                         httplib::Response &res) const {
327   std::string ip = req.remote_addr;
328   int port = req.remote_port;
329   json params = parse_post_body(req);
330   if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end() ||
331     params.find("car_img_path") == params.end()) {
332     res.set_content("Bad Request", "text/plain");
333     res.status = 400;
334     MyLogger::log("carinfo-manager-logger",
335             MyLogger::LOG_LEVEL::WARN,
336             "[HTTP Get Car Image] from " + ip + ":" + std::to_string(
                port) +
337               ". Status: 400 (Bad Request)");
338     return;
339   }
340   std::string username = std::string(params["username"]);
341   std::string passwd_hash = std::string(params["passwd_hash"]);
342   std::string car_img_path = std::string(params["car_img_path"]);
343
344   try {
345     auto result = accountpool.verifyAccount(username, passwd_hash);
346     if (result == AccountPool::AccountVerifyResult::SUCCESS) {
347       std::ifstream is(car_img_path, std::ios::binary);
```

```
348       if (is.is_open()) {
349         std::string img((std::istreambuf_iterator<char>(is)),
350                 std::istreambuf_iterator<char>());
351         if (car_img_path.substr(car_img_path.size() - 4) == ".png")
352           res.set_content(img.c_str(), img.size(), "image/png");
353         else if (car_img_path.substr(car_img_path.size() - 4) == ".jpg"
               )
354           res.set_content(img.c_str(), img.size(), "image/jpeg");
355         else
356           res.set_content(img.c_str(), img.size(), "image/jpeg");
357         res.status = 200;
358         is.close();
359         MyLogger::log("carinfo-manager-logger",
360                 MyLogger::LOG_LEVEL::INFO,
361                 "[HTTP Get Car Image] from " + ip + ":" + std::
                    to_string(port) +
362                   ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
363                   "\n- CarImgPath: " + car_img_path + "\n- Status: 200
                      (OK)");
364       }
365       else {
366         res.set_content("File Not Found", "text/plain");
367         res.status = 404;
368         MyLogger::log("carinfo-manager-logger",
369                 MyLogger::LOG_LEVEL::WARN,
370                 "[HTTP Get Car Image] from " + ip + ":" + std::
                    to_string(port) +
371                   ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
372                   "\n- CarImgPath: " + car_img_path +
373                   "\n- Status: 404 (File Not Found)");
374       }
375     }
376     else if (result == AccountPool::AccountVerifyResult::
        ACCOUNT_NOT_FOUND ||
377         result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
378       res.set_content("Forbidden", "text/plain");
379       res.status = 403;
380       MyLogger::log("carinfo-manager-logger",
381               MyLogger::LOG_LEVEL::WARN,
```

```
382                 "[HTTP Get Car Image] from " + ip + ":" + std::to_string(
                        port) +
383                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
384                   "\n- CarImgPath: " + car_img_path + "\n- Status: 403 (
                          Forbidden)");
385       }
386       else {
387         res.set_content("Internal Server Error", "text/plain");
388         res.status = 500;
389         MyLogger::log("carinfo-manager-logger",
390                 MyLogger::LOG_LEVEL::WARN,
391                 "[HTTP Get Car Image] from " + ip + ":" + std::to_string(
                        port) +
392                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
393                   "\n- CarImgPath: " + car_img_path +
394                   "\n- Status: 500 (Internal Server Error)");
395       }
396     }
397     catch (std::exception &e) {
398       res.set_content("Internal Server Error", "text/plain");
399       res.status = 500;
400       MyLogger::log("carinfo-manager-logger",
401               MyLogger::LOG_LEVEL::WARN,
402               "[HTTP Get Car Image] from " + ip + ":" + std::to_string(
                      port) +
403                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
404                 "\n- CarImgPath: " + car_img_path +
405                 "\n- Status: 500 (Internal Server Error) \n- Exception: "
                        + e.what());
406     }
407 }
408
409 /**
410  * Handles the HTTP request for adding a car.
411  *
412  * This function is responsible for processing the HTTP request to add
        a car to the system.
413  * It retrieves the necessary parameters from the request, verifies the
```

```
     user account,
414  * saves the car image to the appropriate location, creates a new Car
       object, and adds it to the car pool.
415  * The function sets the appropriate response content and status code
       based on the result of the operation.
416  *
417  * @param req The HTTP request object containing the necessary
       parameters.
418  * @param res The HTTP response object to be modified based on the
       result of the operation.
419  */
420 void ServerHttpHandler::handler_add_car(const httplib::Request &req,
       httplib::Response &res) {
421   std::string ip = req.remote_addr;
422   int port = req.remote_port;
423   json params = parse_post_body(req);
424   if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end() ||
425     params.find("car_id") == params.end() || params.find("car_type") ==
          params.end() ||
426     params.find("car_owner") == params.end() || params.find("car_color"
         ) == params.end() ||
427     params.find("car_year") == params.end() || params.find("car_img")
         == params.end() ||
428     params.find("car_img_type") == params.end()) {
429     res.set_content("Bad Request", "text/plain");
430     res.status = 400;
431     MyLogger::log("carinfo-manager-logger",
432             MyLogger::LOG_LEVEL::WARN,
433             "[HTTP Add Car] from " + ip + ":" + std::to_string(port) +
434               ". Status: 400 (Bad Request)");
435     return;
436   }
437
438   std::string username = std::string(params["username"]);
439   std::string passwd_hash = std::string(params["passwd_hash"]);
440   std::string car_id = std::string(params["car_id"]);
441   std::string car_type = std::string(params["car_type"]);
442   std::string car_owner = std::string(params["car_owner"]);
443   std::string car_color = std::string(params["car_color"]);
444   int car_year = std::stoi(std::string(params["car_year"]));
```

```
445   std::string car_img = std::string(params["car_img"]);
446   std::string car_img_type = std::string(params["car_img_type"]);
447
448   try {
449     auto result = accountpool.verifyAccount(username, passwd_hash);
450     if (result == AccountPool::AccountVerifyResult::SUCCESS &&
451       accountpool.getAccountType(username) == Account::AccountType::
              ADMIN) {
452       std::string car_id_base64 = Base64::encode(
453         reinterpret_cast<const unsigned char *>(car_id.c_str()), car_id
                .size());
454       for (size_t i = 0; i < car_id_base64.size(); i++) {
455         if (car_id_base64[i] == '/')
456           car_id_base64[i] = '_';
457         if (car_id_base64[i] == '+')
458           car_id_base64[i] = '-';
459       }
460       std::string car_img_path = imgDir + car_id_base64 + car_img_type;
461       std::ofstream os(car_img_path, std::ios::binary);
462       os.write(car_img.c_str(), car_img.size());
463       os.close();
464       Car new_car = Car(car_id, car_type, car_owner, car_color,
              car_year, car_img_path);
465       int status_code = carpool.addCar(new_car);
466       if (status_code == 0) {
467         res.set_content("Car Added", "text/plain");
468         res.status = 200;
469         MyLogger::log("carinfo-manager-logger",
470                 MyLogger::LOG_LEVEL::INFO,
471                 "[HTTP Add Car] from " + ip + ":" + std::to_string(port
                      ) +
472                   ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
473                   "\n- Car Id: " + car_id + "\n- Car Type: " + car_type
                        +
474                   "\n- Car Owner: " + car_owner + "\n- Car Color: " +
                        car_color +
475                   "\n- Car Year: " + std::to_string(car_year) +
476                   "\n- Car Img Path: " + car_img_path + "\n- Status:
                        200 (OK)");
477       }
```

```
478         else {
479           std::string msg =
480             "Internal Server Error, status code: " + std::to_string(
                    status_code);
481           res.set_content(msg, "text/plain");
482           res.status = 500;
483           MyLogger::log("carinfo-manager-logger",
484                   MyLogger::LOG_LEVEL::WARN,
485                   "[HTTP Add Car] from " + ip + ":" + std::to_string(port
                        ) +
486                     ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
487                     "\n- Car Id: " + car_id + "\n- Car Type: " + car_type
                          +
488                     "\n- Car Owner: " + car_owner + "\n- Car Color: " +
                          car_color +
489                     "\n- Car Year: " + std::to_string(car_year) +
490                     "\n- Car Img Path: " + car_img_path +
491                     "\n- Status: 500 (Internal Server Error) \n- Status
                          Code: " +
492                    std::to_string(status_code));
493         }
494       }
495       else if (result == AccountPool::AccountVerifyResult::
              ACCOUNT_NOT_FOUND ||
496                result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
497         res.set_content("Forbidden", "text/plain");
498         res.status = 403;
499         MyLogger::log("carinfo-manager-logger",
500                 MyLogger::LOG_LEVEL::WARN,
501                 "[HTTP Add Car] from " + ip + ":" + std::to_string(port)
                      +
502                   ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
503                   "\n- Car Id: " + car_id + "\n- Car Type: " + car_type +
504                   "\n- Car Owner: " + car_owner + "\n- Car Color: " +
                        car_color +
505                   "\n- Car Year: " + std::to_string(car_year) +
506                   "\n- Status: 403 (Forbidden)");
507       }
508       else {
```

```
509       res.set_content("Internal Server Error", "text/plain");
510       res.status = 500;
511       MyLogger::log("carinfo-manager-logger",
512               MyLogger::LOG_LEVEL::WARN,
513               "[HTTP Add Car] from " + ip + ":" + std::to_string(port) +
514                 ".\n- Username: " + username + "\n- PasswdHash: " + passwd_hash +
515                 "\n- Car Id: " + car_id + "\n- Car Type: " + car_type +
516                 "\n- Car Owner: " + car_owner + "\n- Car Color: " + car_color +
517                 "\n- Car Year: " + std::to_string(car_year) +
518                 "\n- Status: 500 (Internal Server Error)");
519     }
520   }
521   catch (std::exception &e) {
522     res.set_content("Internal Server Error", "text/plain");
523     res.status = 500;
524     MyLogger::log("carinfo-manager-logger",
525             MyLogger::LOG_LEVEL::WARN,
526             "[HTTP Add Car] from " + ip + ":" + std::to_string(port) +
527               ".\n- Username: " + username + "\n- PasswdHash: " + passwd_hash +
528               "\n- Status: 500 (Internal Server Error) \n- Exception: " + e.what());
529   }
530 }
531
532 /**
533  * Handles the removal of a car based on the provided parameters.
534  *
535  * @param req The HTTP request object containing the parameters.
536  * @param res The HTTP response object to be modified.
537  */
538 void ServerHttpHandler::handler_remove_car(const httplib::Request &req,
      httplib::Response &res) {
539   std::string ip = req.remote_addr;
540   int port = req.remote_port;
541   json params = parse_post_body(req);
542   if (params.find("username") == params.end() || params.find("
      passwd_hash") == params.end() ||
```

```
543     params.find("car_id") == params.end()) {
544     res.set_content("Bad Request", "text/plain");
545     res.status = 400;
546     MyLogger::log("carinfo-manager-logger",
547             MyLogger::LOG_LEVEL::WARN,
548             "[HTTP Remove Car] from " + ip + ":" + std::to_string(port)
                    +
549               ". Status: 400 (Bad Request)");
550     return;
551   }
552   std::string username = std::string(params["username"]);
553   std::string passwd_hash = std::string(params["passwd_hash"]);
554   std::string car_id = std::string(params["car_id"]);
555
556   try {
557     auto result = accountpool.verifyAccount(username, passwd_hash);
558     if (result == AccountPool::AccountVerifyResult::SUCCESS &&
559       accountpool.getAccountType(username) == Account::AccountType::
             ADMIN) {
560       CarPool cars = carpool.getCarbyId(car_id);
561       if (cars.size() == 0) {
562         res.set_content("Car Not Found", "text/plain");
563         res.status = 404;
564         MyLogger::log("carinfo-manager-logger",
565                 MyLogger::LOG_LEVEL::WARN,
566                 "[HTTP Remove Car] from " + ip + ":" + std::to_string(
                        port) +
567                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
568                   "\n- Car Id: " + car_id + "\n- Status: 404 (Car Not
                          Found)");
569       }
570       else {
571         int status_code = carpool.removeCar(car_id);
572         if (status_code == 0) {
573           res.set_content("Car Removed", "text/plain");
574           res.status = 200;
575           MyLogger::log("carinfo-manager-logger",
576                   MyLogger::LOG_LEVEL::INFO,
577                   "[HTTP Remove Car] from " + ip + ":" + std::to_string
                          (port) +
```

```
578                     ".\n- Username: " + username +
579                     "\n- PasswdHash: " + passwd_hash + "\n- Car Id: " +
                            car_id +
580                     "\n- Status: 200 (OK)");
581         }
582         else {
583           std::string msg =
584             "Internal Server Error, status code: " + std::to_string(
                    status_code);
585           res.set_content(msg, "text/plain");
586           res.status = 500;
587           MyLogger::log("carinfo-manager-logger",
588                   MyLogger::LOG_LEVEL::WARN,
589                   "[HTTP Remove Car] from " + ip + ":" + std::to_string
                        (port) +
590                     ".\n- Username: " + username +
591                     "\n- PasswdHash: " + passwd_hash + "\n- Car Id: " +
                            car_id +
592                     "\n- Status: 500 (Internal Server Error) \n- Status
                            Code: " +
593                     std::to_string(status_code));
594         }
595       }
596     }
597     else if (result == AccountPool::AccountVerifyResult::
           ACCOUNT_NOT_FOUND ||
598         result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
599       res.set_content("Forbidden", "text/plain");
600       res.status = 403;
601       MyLogger::log("carinfo-manager-logger",
602               MyLogger::LOG_LEVEL::WARN,
603               "[HTTP Remove Car] from " + ip + ":" + std::to_string(
                      port) +
604                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
605                 "\n- Car Id: " + car_id + "\n- Status: 403 (Forbidden)"
                        );
606     }
607     else {
608       res.set_content("Internal Server Error", "text/plain");
609       res.status = 500;
```

```
610         MyLogger::log("carinfo-manager-logger",
611                 MyLogger::LOG_LEVEL::WARN,
612                 "[HTTP Remove Car] from " + ip + ":" + std::to_string(
                        port) +
613                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
614                   "\n- Car Id: " + car_id + "\n- Status: 500 (Internal
                          Server Error)");
615       }
616     }
617     catch (std::exception &e) {
618       res.set_content("Internal Server Error", "text/plain");
619       res.status = 500;
620       MyLogger::log("carinfo-manager-logger",
621               MyLogger::LOG_LEVEL::WARN,
622               "[HTTP Remove Car] from " + ip + ":" + std::to_string(port)
                      +
623                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
624                 "\n- Car Id: " + car_id +
625                 "\n- Status: 500 (Internal Server Error) \n- Exception: "
                        + e.what());
626     }
627 }
628
629 /**
630  * Handles the update car request.
631  *
632  * @param req The HTTP request object.
633  * @param res The HTTP response object.
634  */
635 void ServerHttpHandler::handler_update_car(const httplib::Request &req,
        httplib::Response &res) {
636     std::string ip = req.remote_addr;
637     int port = req.remote_port;
638     json params = parse_post_body(req);
639     if (params.find("username") == params.end() || params.find("
          passwd_hash") == params.end() ||
640       params.find("original_car_id") == params.end() ||
641       params.find("new_car_id") == params.end() || params.find("
          new_car_type") == params.end() ||
```

```
642     params.find("new_car_owner") == params.end() ||
643     params.find("new_car_color") == params.end() ||
644     params.find("new_car_year") == params.end() || params.find("
            new_car_img") == params.end() ||
645     params.find("new_car_img_type") == params.end()) {
646     res.set_content("Bad Request", "text/plain");
647     res.status = 400;
648     MyLogger::log("carinfo-manager-logger",
649             MyLogger::LOG_LEVEL::WARN,
650             "[HTTP Update Car] from " + ip + ":" + std::to_string(port)
                    +
651               ". Status: 400 (Bad Request)");
652     return;
653   }
654   std::string username = std::string(params["username"]);
655   std::string passwd_hash = std::string(params["passwd_hash"]);
656   std::string original_car_id = std::string(params["original_car_id"]);
657   std::string new_car_id = std::string(params["new_car_id"]);
658   std::string new_car_type = std::string(params["new_car_type"]);
659   std::string new_car_owner = std::string(params["new_car_owner"]);
660   std::string new_car_color = std::string(params["new_car_color"]);
661   int new_car_year = std::stoi(std::string(params["new_car_year"]));
662   std::string new_car_img = std::string(params["new_car_img"]);
663   std::string new_car_img_type = std::string(params["new_car_img_type"
          ]);
664
665   try {
666     auto result = accountpool.verifyAccount(username, passwd_hash);
667     if (result == AccountPool::AccountVerifyResult::SUCCESS &&
668       accountpool.getAccountType(username) == Account::AccountType::
              ADMIN) {
669       CarPool original_cars = carpool.getCarbyId(original_car_id);
670       if (original_cars.size() == 0) {
671         res.set_content("Car Not Found", "text/plain");
672         res.status = 404;
673         MyLogger::log("carinfo-manager-logger",
674                 MyLogger::LOG_LEVEL::WARN,
675                 "[HTTP Update Car] from " + ip + ":" + std::to_string(
                        port) +
676                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
```

```
677                     "\n- Original Car Id: " + original_car_id + "\n- New
                            Car Id: " +
678                     new_car_id + "\n- New Car Type: " + new_car_type +
679                     "\n- New Car Owner: " + new_car_owner +
680                     "\n- New Car Color: " + new_car_color + "\n- New Car
                            Year: " +
681                     std::to_string(new_car_year) + "\n- Status: 404 (Car
                            Not Found)");
682         }
683         else {
684           std::string new_car_id_base64 = Base64::encode(
685             reinterpret_cast<const unsigned char *>(new_car_id.c_str()),
                    new_car_id.size());
686           for (size_t i = 0; i < new_car_id_base64.size(); i++) {
687             if (new_car_id_base64[i] == '/')
688               new_car_id_base64[i] = '_';
689             if (new_car_id_base64[i] == '+')
690               new_car_id_base64[i] = '-';
691           }
692           std::string new_car_img_path = imgDir + new_car_id_base64 +
                  new_car_img_type;
693           std::ofstream os(new_car_img_path, std::ios::binary);
694           os.write(new_car_img.c_str(), new_car_img.size());
695           os.close();
696           Car new_car = Car(new_car_id,
697                     new_car_type,
698                     new_car_owner,
699                     new_car_color,
700                     new_car_year,
701                     new_car_img_path);
702           int status_code = carpool.updateCar(original_car_id, new_car);
703           if (status_code == 0) {
704             res.set_content("Car Updated", "text/plain");
705             res.status = 200;
706             MyLogger::log(
707               "carinfo-manager-logger",
708               MyLogger::LOG_LEVEL::INFO,
709               "[HTTP Update Car] from " + ip + ":" + std::to_string(port)
                      +
710                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
```

```
711                 "\n- Original Car Id: " + original_car_id +
712                 "\n- New Car Id: " + new_car_id + "\n- New Car Type: " +
                        new_car_type +
713                 "\n- New Car Owner: " + new_car_owner + "\n- New Car
                        Color: " +
714                 new_car_color + "\n- New Car Year: " + std::to_string(
                        new_car_year) +
715                 "\n- Status: 200 (OK)");
716           }
717           else {
718             std::string msg =
719               "Internal Server Error, status code: " + std::to_string(
                      status_code);
720             res.set_content(msg, "text/plain");
721             res.status = 500;
722             MyLogger::log(
723               "carinfo-manager-logger",
724               MyLogger::LOG_LEVEL::WARN,
725               "[HTTP Update Car] from " + ip + ":" + std::to_string(port)
                      +
726               ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
727               "\n- Original Car Id: " + original_car_id +
728               "\n- New Car Id: " + new_car_id + "\n- New Car Type: " +
                        new_car_type +
729               "\n- New Car Owner: " + new_car_owner + "\n- New Car
                        Color: " +
730               new_car_color + "\n- New Car Year: " + std::to_string(
                        new_car_year) +
731               "\n- Status: 500 (Internal Server Error) \n- Status Code:
                      " +
732               std::to_string(status_code));
733           }
734         }
735       }
736       else if (result == AccountPool::AccountVerifyResult::
             ACCOUNT_NOT_FOUND ||
737                result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
738         res.set_content("Forbidden", "text/plain");
739         res.status = 403;
740         MyLogger::log(
```

```
741         "carinfo-manager-logger",
742         MyLogger::LOG_LEVEL::WARN,
743         "[HTTP Update Car] from " + ip + ":" + std::to_string(port) +
744           ".\n- Username: " + username + "\n- PasswdHash: " +
                passwd_hash +
745           "\n- Original Car Id: " + original_car_id + "\n- New Car Id:
                " + new_car_id +
746           "\n- New Car Type: " + new_car_type + "\n- New Car Owner: " +
                new_car_owner +
747           "\n- New Car Color: " + new_car_color + "\n- New Car Year: "
                +
748           std::to_string(new_car_year) + "\n- Status: 403 (Forbidden)")
                ;
749     }
750     else {
751       res.set_content("Internal Server Error", "text/plain");
752       res.status = 500;
753       MyLogger::log(
754         "carinfo-manager-logger",
755         MyLogger::LOG_LEVEL::WARN,
756         "[HTTP Update Car] from " + ip + ":" + std::to_string(port) +
757           ".\n- Username: " + username + "\n- PasswdHash: " +
                passwd_hash +
758           "\n- Original Car Id: " + original_car_id + "\n- New Car Id:
                " + new_car_id +
759           "\n- New Car Type: " + new_car_type + "\n- New Car Owner: " +
                new_car_owner +
760           "\n- New Car Color: " + new_car_color + "\n- New Car Year: "
                +
761           std::to_string(new_car_year) + "\n- Status: 500 (Internal
                Server Error)");
762     }
763   }
764   catch (std::exception &e) {
765     res.set_content("Internal Server Error", "text/plain");
766     res.status = 500;
767     MyLogger::log("carinfo-manager-logger",
768             MyLogger::LOG_LEVEL::WARN,
769             "[HTTP Update Car] from " + ip + ":" + std::to_string(port)
                  +
770               ".\n- Username: " + username + "\n- PasswdHash: " +
```

```
                passwd_hash +
771             "\n- Original Car Id: " + original_car_id +
772             "\n- New Car Id: " + new_car_id + "\n- New Car Type: " +
                    new_car_type +
773             "\n- New Car Owner: " + new_car_owner + "\n- New Car
                    Color: " +
774             new_car_color + "\n- New Car Year: " + std::to_string(
                    new_car_year) +
775             "\n- Status: 500 (Internal Server Error) \n- Exception: "
                     + e.what());
776   }
777 }
778
779 /**
780  * Handles the request for retrieving account information.
781  *
782  * @param req The HTTP request object.
783  * @param res The HTTP response object.
784  */
785 void ServerHttpHandler::handler_get_accountinfo(const httplib::Request
       &req,
786                          httplib::Response &res) const {
787   std::string ip = req.remote_addr;
788   int port = req.remote_port;
789   json params = parse_post_body(req);
790   if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end() ||
791     params.find("target_username") == params.end()) {
792     res.set_content("Bad Request", "text/plain");
793     res.status = 400;
794     MyLogger::log("carinfo-manager-logger",
795             MyLogger::LOG_LEVEL::WARN,
796             "[HTTP Get Account Info] from " + ip + ":" + std::to_string
                    (port) +
797               ". Status: 400 (Bad Request)");
798     return;
799   }
800   if (std::string(params["target_username"]).empty()) {
801     res.set_content("Bad Request", "text/plain");
802     res.status = 400;
803     MyLogger::log("carinfo-manager-logger",
```

```
804             MyLogger::LOG_LEVEL::WARN,
805             "[HTTP Get Account Info] from " + ip + ":" + std::to_string
                    (port) +
806               ". Status: 400 (No Target Username)");
807     return;
808   }
809   std::string username = std::string(params["username"]);
810   std::string passwd_hash = std::string(params["passwd_hash"]);
811   std::string target_username = std::string(params["target_username"]);
812
813   try {
814     auto result = accountpool.verifyAccount(username, passwd_hash);
815     if (result == AccountPool::AccountVerifyResult::SUCCESS &&
816       accountpool.getAccountType(username) == Account::AccountType::
              ADMIN) {
817       AccountPool target_accs = accountpool.getAccountLike(
              target_username);
818       if (target_accs.empty()) {
819         res.set_content("Account Not Found", "text/plain");
820         res.status = 404;
821         MyLogger::log("carinfo-manager-logger",
822                 MyLogger::LOG_LEVEL::WARN,
823                 "[HTTP Get Account Info] from " + ip + ":" + std::
                        to_string(port) +
824                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
825                   "\n- Target Username: " + target_username +
826                   "\n- Status: 404 (Account Not Found)");
827       }
828       else {
829         std::stringstream ss;
830         int status_code = target_accs.save(ss);
831         if (status_code == 0) {
832           res.set_content(ss.str(), "application/json");
833           res.status = 200;
834           MyLogger::log(
835             "carinfo-manager-logger",
836             MyLogger::LOG_LEVEL::INFO,
837             "[HTTP Get Account Info] from " + ip + ":" + std::to_string
                    (port) +
838               ".\n- Username: " + username + "\n- PasswdHash: " +
```

```
                passwd_hash +
839             "\n- Target Username: " + target_username + "\n- Status:
                    200 (OK)");
840       }
841       else {
842         res.set_content("Internal Server Error", "text/plain");
843         res.status = 500;
844         MyLogger::log("carinfo-manager-logger",
845                 MyLogger::LOG_LEVEL::WARN,
846                 "[HTTP Get Account Info] from " + ip + ":" +
847                   std::to_string(port) + ".\n- Username: " + username
                        +
848                   "\n- PasswdHash: " + passwd_hash +
849                   "\n- Target Username: " + target_username +
850                   "\n- Status: 500 (Internal Server Error) \n- Status
                        Code: " +
851                   std::to_string(status_code));
852       }
853     }
854   }
855   else if (result == AccountPool::AccountVerifyResult::
        ACCOUNT_NOT_FOUND ||
856       result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
857     res.set_content("Forbidden", "text/plain");
858     res.status = 403;
859     MyLogger::log("carinfo-manager-logger",
860             MyLogger::LOG_LEVEL::WARN,
861             "[HTTP Get Account Info] from " + ip + ":" + std::
                    to_string(port) +
862               ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
863               "\n- Target Username: " + target_username +
864               "\n- Status: 403 (Forbidden)");
865   }
866   else {
867     res.set_content("Internal Server Error", "text/plain");
868     res.status = 500;
869     MyLogger::log("carinfo-manager-logger",
870             MyLogger::LOG_LEVEL::WARN,
871             "[HTTP Get Account Info] from " + ip + ":" + std::
                    to_string(port) +
```

```
872                 ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
873                 "\n- Target Username: " + target_username +
874                 "\n- Status: 500 (Internal Server Error)");
875     }
876   }
877   catch (std::exception &e) {
878     res.set_content("Internal Server Error", "text/plain");
879     res.status = 500;
880     MyLogger::log("carinfo-manager-logger",
881             MyLogger::LOG_LEVEL::WARN,
882             "[HTTP Get Account Info] from " + ip + ":" + std::to_string
                    (port) +
883               ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
884               "\n- Target Username: " + target_username +
885               "\n- Status: 500 (Internal Server Error) \n- Exception: "
                      + e.what());
886   }
887 }
888
889 /**
890  * Handles the request for retrieving all account information.
891  *
892  * @param req The HTTP request object.
893  * @param res The HTTP response object.
894  */
895 void ServerHttpHandler::handler_get_all_account(const httplib::Request
       &req,
896                                                 httplib::Response &res) const {
897   std::string ip = req.remote_addr;
898   int port = req.remote_port;
899   json params = parse_post_body(req);
900   if (params.find("username") == params.end() || params.find("
        passwd_hash") == params.end()) {
901     res.set_content("Bad Request", "text/plain");
902     res.status = 400;
903     MyLogger::log("carinfo-manager-logger",
904             MyLogger::LOG_LEVEL::WARN,
905             "[HTTP Get All Account] from " + ip + ":" + std::to_string(
                    port) +
```

```
906                 ". Status: 400 (Bad Request)");
907       return;
908     }
909     std::string username = std::string(params["username"]);
910     std::string passwd_hash = std::string(params["passwd_hash"]);
911
912     try {
913       auto result = accountpool.verifyAccount(username, passwd_hash);
914       if (result == AccountPool::AccountVerifyResult::SUCCESS &&
915         accountpool.getAccountType(username) == Account::AccountType::
              ADMIN) {
916         std::stringstream ss;
917         int status_code = accountpool.save(ss);
918         if (status_code == 0) {
919           res.set_content(ss.str(), "application/json");
920           res.status = 200;
921           MyLogger::log("carinfo-manager-logger",
922                   MyLogger::LOG_LEVEL::INFO,
923                   "[HTTP Get All Account] from " + ip + ":" + std::
                        to_string(port) +
924                     ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
925                     "\n- Status: 200 (OK)");
926         }
927         else {
928           res.set_content(
929             "Internal Server Error, status code: " + std::to_string(
                    status_code),
930             "text/plain");
931           res.status = 500;
932           MyLogger::log("carinfo-manager-logger",
933                   MyLogger::LOG_LEVEL::WARN,
934                   "[HTTP Get All Account] from " + ip + ":" + std::
                        to_string(port) +
935                     ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
936                     "\n- Status: 500 (Internal Server Error)");
937         }
938       }
939       else {
940         res.set_content("Forbidden", "text/plain");
```

```
941         res.status = 403;
942         MyLogger::log("carinfo-manager-logger",
943                 MyLogger::LOG_LEVEL::WARN,
944                 "[HTTP Get All Account] from " + ip + ":" + std::
                        to_string(port) +
945                   ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
946                   "\n- Status: 403 (Forbidden)");
947       }
948     }
949     catch (std::exception &e) {
950       res.set_content("Internal Server Error", "text/plain");
951       res.status = 500;
952       MyLogger::log("carinfo-manager-logger",
953               MyLogger::LOG_LEVEL::WARN,
954               "[HTTP Get All Account] from " + ip + ":" + std::to_string(
                        port) +
955                 ".\n- Username: " + username + "\n- PasswdHash: " +
                          passwd_hash +
956                 "\n- Status: 500 (Internal Server Error) \n- Exception: "
                          + e.what());
957     }
958 }
959
960 /**
961  * Handles the request for adding an account.
962  *
963  * @param req The HTTP request object.
964  * @param res The HTTP response object.
965  */
966 void ServerHttpHandler::handler_add_account(const httplib::Request &req
        , httplib::Response &res) {
967     std::string ip = req.remote_addr;
968     int port = req.remote_port;
969     json params = parse_post_body(req);
970     if (params.find("username") == params.end() || params.find("
          passwd_hash") == params.end() ||
971       params.find("target_username") == params.end() ||
972       params.find("target_passwd_hash") == params.end() ||
973       params.find("target_account_type") == params.end()) {
974       res.set_content("Bad Request", "text/plain");
```

```
975     res.status = 400;
976     MyLogger::log("carinfo-manager-logger",
977             MyLogger::LOG_LEVEL::WARN,
978             "[HTTP Add Account] from " + ip + ":" + std::to_string(port
                ) +
979               ". Status: 400 (Bad Request)");
980     return;
981   }
982   std::string username = std::string(params["username"]);
983   std::string passwd_hash = std::string(params["passwd_hash"]);
984   std::string target_username = std::string(params["target_username"]);
985   std::string target_passwd_hash = std::string(params["
        target_passwd_hash"]);
986   int target_account_type = std::stoi(std::string(params["
        target_account_type"]));
987
988   try {
989     auto result = accountpool.verifyAccount(username, passwd_hash);
990     if (result == AccountPool::AccountVerifyResult::SUCCESS &&
991       accountpool.getAccountType(username) == Account::AccountType::
            ADMIN) {
992       Account new_acc = Account(target_username,
993                     target_passwd_hash,
994                     static_cast<Account::AccountType>(
                        target_account_type));
995       int status_code = accountpool.addAccount(new_acc);
996       if (status_code == 0) {
997         res.set_content("Account Added", "text/plain");
998         res.status = 200;
999         MyLogger::log("carinfo-manager-logger",
1000                 MyLogger::LOG_LEVEL::INFO,
1001                 "[HTTP Add Account] from " + ip + ":" + std::to_string(
                        port) +
1002                   ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
1003                   "\n- Target Username: " + target_username +
1004                   "\n- Target PasswdHash: " + target_passwd_hash +
1005                   "\n- Target Account Type: " +
1006                   std::to_string(target_account_type) + "\n- Status:
                        200 (OK)");
1007       }
```

```
1008        else {
1009          std::string msg =
1010            "Internal Server Error, status code: " + std::to_string(
                  status_code);
1011          res.set_content(msg, "text/plain");
1012          res.status = 500;
1013          MyLogger::log(
1014            "carinfo-manager-logger",
1015            MyLogger::LOG_LEVEL::WARN,
1016            "[HTTP Add Account] from " + ip + ":" + std::to_string(port)
                  +
1017              ".\n- Username: " + username + "\n- PasswdHash: " +
                    passwd_hash +
1018              "\n- Target Username: " + target_username +
1019              "\n- Target PasswdHash: " + target_passwd_hash +
1020              "\n- Target Account Type: " + std::to_string(
                    target_account_type) +
1021              "\n- Status: 500 (Internal Server Error) \n- Status Code: "
                      +
1022              std::to_string(status_code));
1023        }
1024      }
1025      else if (result == AccountPool::AccountVerifyResult::
             ACCOUNT_NOT_FOUND ||
1026          result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
1027        res.set_content("Forbidden", "text/plain");
1028        res.status = 403;
1029        MyLogger::log("carinfo-manager-logger",
1030                MyLogger::LOG_LEVEL::WARN,
1031                "[HTTP Add Account] from " + ip + ":" + std::to_string(
                      port) +
1032                  ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
1033                  "\n- Target Username: " + target_username +
1034                  "\n- Target PasswdHash: " + target_passwd_hash +
1035                  "\n- Target Account Type: " + std::to_string(
                        target_account_type) +
1036                  "\n- Status: 403 (Forbidden)");
1037      }
1038      else {
1039        res.set_content("Internal Server Error", "text/plain");
```

```
1040        res.status = 500;
1041        MyLogger::log("carinfo-manager-logger",
1042                MyLogger::LOG_LEVEL::WARN,
1043                "[HTTP Add Account] from " + ip + ":" + std::to_string(
                        port) +
1044                ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
1045                "\n- Target Username: " + target_username +
1046                "\n- Target PasswdHash: " + target_passwd_hash +
1047                "\n- Target Account Type: " + std::to_string(
                        target_account_type) +
1048                "\n- Status: 500 (Internal Server Error)");
1049      }
1050    }
1051    catch (std::exception &e) {
1052      res.set_content("Internal Server Error", "text/plain");
1053      res.status = 500;
1054      MyLogger::log("carinfo-manager-logger",
1055              MyLogger::LOG_LEVEL::WARN,
1056              "[HTTP Add Account] from " + ip + ":" + std::to_string(port
                        ) +
1057              ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
1058              "\n- Target Username: " + target_username +
1059              "\n- Target PasswdHash: " + target_passwd_hash +
1060              "\n- Target Account Type: " + std::to_string(
                        target_account_type) +
1061              "\n- Status: 500 (Internal Server Error) \n- Exception: "
                      + e.what());
1062    }
1063  }
1064
1065  /**
1066   * Handles the request for removing an account.
1067   *
1068   * @param req The HTTP request object.
1069   * @param res The HTTP response object.
1070   */
1071  void ServerHttpHandler::handler_remove_account(const httplib::Request &
        req,
1072                                                 httplib::Response &res) {
```

```
1073    std::string ip = req.remote_addr;
1074    int port = req.remote_port;
1075    json params = parse_post_body(req);
1076    if (params.find("username") == params.end() || params.find("
           passwd_hash") == params.end() ||
1077      params.find("target_username") == params.end()) {
1078      res.set_content("Bad Request", "text/plain");
1079      res.status = 400;
1080      MyLogger::log("carinfo-manager-logger",
1081              MyLogger::LOG_LEVEL::WARN,
1082              "[HTTP Remove Account] from " + ip + ":" + std::to_string(
                   port) +
1083                ". Status: 400 (Bad Request)");
1084      return;
1085    }
1086    std::string username = std::string(params["username"]);
1087    std::string passwd_hash = std::string(params["passwd_hash"]);
1088    std::string target_username = std::string(params["target_username"]);
1089
1090    try {
1091      auto result = accountpool.verifyAccount(username, passwd_hash);
1092      if (result == AccountPool::AccountVerifyResult::SUCCESS &&
1093        accountpool.getAccountType(username) == Account::AccountType::
             ADMIN) {
1094        Account target_acc = accountpool.getAccount(target_username);
1095        if (target_acc == Account::NULL_ACCOUNT) {
1096          res.set_content("Account Not Found", "text/plain");
1097          res.status = 404;
1098          MyLogger::log("carinfo-manager-logger",
1099                  MyLogger::LOG_LEVEL::WARN,
1100                  "[HTTP Remove Account] from " + ip + ":" + std::
                       to_string(port) +
1101                    ".\n- Username: " + username + "\n- PasswdHash: " +
                         passwd_hash +
1102                    "\n- Target Username: " + target_username +
1103                    "\n- Status: 404 (Account Not Found)");
1104        }
1105        else {
1106          int status_code = accountpool.removeAccount(target_username);
1107          if (status_code == 0) {
1108            res.set_content("Account Removed", "text/plain");
```

```
1109            res.status = 200;
1110            MyLogger::log(
1111              "carinfo-manager-logger",
1112              MyLogger::LOG_LEVEL::INFO,
1113              "[HTTP Remove Account] from " + ip + ":" + std::to_string(
                    port) +
1114                ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
1115                "\n- Target Username: " + target_username + "\n- Status:
                      200 (OK)");
1116          }
1117          else {
1118            std::string msg =
1119              "Internal Server Error, status code: " + std::to_string(
                    status_code);
1120            res.set_content(msg, "text/plain");
1121            res.status = 500;
1122            MyLogger::log("carinfo-manager-logger",
1123                    MyLogger::LOG_LEVEL::WARN,
1124                    "[HTTP Remove Account] from " + ip + ":" + std::
                          to_string(port) +
1125                      ".\n- Username: " + username + "\n- PasswdHash: " +
1126                      passwd_hash + "\n- Target Username: " +
                            target_username +
1127                      "\n- Status: 500 (Internal Server Error) \n- Status
                            Code: " +
1128                      std::to_string(status_code));
1129          }
1130        }
1131      }
1132      else if (result == AccountPool::AccountVerifyResult::
           ACCOUNT_NOT_FOUND ||
1133            result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
1134        res.set_content("Forbidden", "text/plain");
1135        res.status = 403;
1136        MyLogger::log("carinfo-manager-logger",
1137                MyLogger::LOG_LEVEL::WARN,
1138                "[HTTP Remove Account] from " + ip + ":" + std::to_string
                      (port) +
1139                  ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
```

```
1140                 "\n- Target Username: " + target_username +
1141                 "\n- Status: 403 (Forbidden)");
1142     }
1143     else {
1144       res.set_content("Internal Server Error", "text/plain");
1145       res.status = 500;
1146       MyLogger::log("carinfo-manager-logger",
1147               MyLogger::LOG_LEVEL::WARN,
1148               "[HTTP Remove Account] from " + ip + ":" + std::to_string
                    (port) +
1149                 ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
1150                 "\n- Target Username: " + target_username +
1151                 "\n- Status: 500 (Internal Server Error)");
1152     }
1153   }
1154   catch (std::exception &e) {
1155     res.set_content("Internal Server Error", "text/plain");
1156     res.status = 500;
1157     MyLogger::log("carinfo-manager-logger",
1158             MyLogger::LOG_LEVEL::WARN,
1159             "[HTTP Remove Account] from " + ip + ":" + std::to_string(
                    port) +
1160               ".\n- Username: " + username + "\n- PasswdHash: " +
                      passwd_hash +
1161               "\n- Target Username: " + target_username +
1162               "\n- Status: 500 (Internal Server Error) \n- Exception: "
                      + e.what());
1163   }
1164 }
1165
1166 /**
1167  * Handles the request for updating an account.
1168  *
1169  * @param req The HTTP request object.
1170  * @param res The HTTP response object.
1171  */
1172 void ServerHttpHandler::handler_update_account(const httplib::Request &
      req,
1173                            httplib::Response &res) {
1174   std::string ip = req.remote_addr;
```

```
1175    int port = req.remote_port;
1176    json params = parse_post_body(req);
1177    if (params.find("username") == params.end() || params.find("
            passwd_hash") == params.end() ||
1178      params.find("target_username") == params.end() ||
1179      params.find("new_username") == params.end() ||
1180      params.find("new_passwd_hash") == params.end() ||
1181      params.find("new_account_type") == params.end()) {
1182      res.set_content("Bad Request", "text/plain");
1183      res.status = 400;
1184      MyLogger::log("carinfo-manager-logger",
1185              MyLogger::LOG_LEVEL::WARN,
1186              "[HTTP Update Account] from " + ip + ":" + std::to_string(
                        port) +
1187                ". Status: 400 (Bad Request)");
1188      return;
1189    }
1190    std::string username = std::string(params["username"]);
1191    std::string passwd_hash = std::string(params["passwd_hash"]);
1192    std::string target_username = std::string(params["target_username"]);
1193    std::string new_username = std::string(params["new_username"]);
1194    std::string new_passwd_hash = std::string(params["new_passwd_hash"]);
1195    int new_account_type = std::stoi(std::string(params["new_account_type
            "]));
1196
1197    try {
1198      auto result = accountpool.verifyAccount(username, passwd_hash);
1199      if (result == AccountPool::AccountVerifyResult::SUCCESS &&
1200        accountpool.getAccountType(username) == Account::AccountType::
                ADMIN) {
1201        Account target_acc = accountpool.getAccount(target_username);
1202        if (target_acc == Account::NULL_ACCOUNT) {
1203          res.set_content("Account Not Found", "text/plain");
1204          res.status = 404;
1205          MyLogger::log("carinfo-manager-logger",
1206                  MyLogger::LOG_LEVEL::WARN,
1207                  "[HTTP Update Account] from " + ip + ":" + std::
                            to_string(port) +
1208                    ".\n- Username: " + username + "\n- PasswdHash: " +
                              passwd_hash +
1209                    "\n- Target Username: " + target_username + "\n- New
```

```
                    Username: " +
1210                new_username + "\n- New PasswdHash: " +
                    new_passwd_hash +
1211                "\n- New Account Type: " + std::to_string(
                    new_account_type) +
1212                "\n- Status: 404 (Account Not Found)");
1213        }
1214        else {
1215          Account new_acc = Account(new_username,
1216                        new_passwd_hash,
1217                        static_cast<Account::AccountType>(
                            new_account_type));
1218          int status_code = accountpool.updateAccount(target_acc, new_acc
                );
1219          if (status_code == 0) {
1220            res.set_content("Account Updated", "text/plain");
1221            res.status = 200;
1222            MyLogger::log("carinfo-manager-logger",
1223                    MyLogger::LOG_LEVEL::INFO,
1224                    "[HTTP Update Account] from " + ip + ":" + std::
                        to_string(port) +
1225                      ".\n- Username: " + username + "\n- PasswdHash: " +
1226                      passwd_hash + "\n- Target Username: " +
                            target_username +
1227                      "\n- New Username: " + new_username +
1228                      "\n- New PasswdHash: " + new_passwd_hash +
1229                      "\n- New Account Type: " + std::to_string(
                            new_account_type) +
1230                      "\n- Status: 200 (OK)");
1231          }
1232          else {
1233            std::string msg =
1234              "Internal Server Error, status code: " + std::to_string(
                      status_code);
1235            res.set_content(msg, "text/plain");
1236            res.status = 500;
1237            MyLogger::log("carinfo-manager-logger",
1238                    MyLogger::LOG_LEVEL::WARN,
1239                    "[HTTP Update Account] from " + ip + ":" + std::
                        to_string(port) +
1240                      ".\n- Username: " + username + "\n- PasswdHash: " +
```

```
1241                    passwd_hash + "\n- Target Username: " +
                            target_username +
1242                    "\n- New Username: " + new_username +
1243                    "\n- New PasswdHash: " + new_passwd_hash +
1244                    "\n- New Account Type: " + std::to_string(
                            new_account_type) +
1245                    "\n- Status: 500 (Internal Server Error) \n- Status
                            Code: " +
1246                    std::to_string(status_code));
1247        }
1248      }
1249    }
1250    else if (result == AccountPool::AccountVerifyResult::
         ACCOUNT_NOT_FOUND ||
1251             result == AccountPool::AccountVerifyResult::WRONG_PASSWORD) {
1252      res.set_content("Forbidden", "text/plain");
1253      res.status = 403;
1254      MyLogger::log("carinfo-manager-logger",
1255              MyLogger::LOG_LEVEL::WARN,
1256              "[HTTP Update Account] from " + ip + ":" + std::to_string
                        (port) +
1257                ".\n- Username: " + username + "\n- PasswdHash: " +
                            passwd_hash +
1258                "\n- Target Username: " + target_username + "\n- New
                            Username: " +
1259                new_username + "\n- New PasswdHash: " + new_passwd_hash
                            +
1260                "\n- New Account Type: " + std::to_string(
                            new_account_type) +
1261                "\n- Status: 403 (Forbidden)");
1262    }
1263    else {
1264      res.set_content("Internal Server Error", "text/plain");
1265      res.status = 500;
1266      MyLogger::log("carinfo-manager-logger",
1267              MyLogger::LOG_LEVEL::WARN,
1268              "[HTTP Update Account] from " + ip + ":" + std::to_string
                        (port) +
1269                ".\n- Username: " + username + "\n- PasswdHash: " +
                            passwd_hash +
1270                "\n- Target Username: " + target_username + "\n- New
```

```
                    Username: " +
1271                new_username + "\n- New PasswdHash: " + new_passwd_hash
                        +
1272                "\n- New Account Type: " + std::to_string(
                        new_account_type) +
1273                "\n- Status: 500 (Internal Server Error)");
1274    }
1275  }
1276  catch (std::exception &e) {
1277    res.set_content("Internal Server Error", "text/plain");
1278    res.status = 500;
1279    MyLogger::log("carinfo-manager-logger",
1280            MyLogger::LOG_LEVEL::WARN,
1281            "[HTTP Update Account] from " + ip + ":" + std::to_string(
                        port) +
1282              ".\n- Username: " + username + "\n- PasswdHash: " +
                        passwd_hash +
1283              "\n- Target Username: " + target_username + "\n- New
                        Username: " +
1284              new_username + "\n- New PasswdHash: " + new_passwd_hash +
1285              "\n- New Account Type: " + std::to_string(
                        new_account_type) +
1286              "\n- Status: 500 (Internal Server Error) \n- Exception: "
                        + e.what());
1287  }
1288 }
```

src/log.cpp

```
1  /**
2   * @file src/log.cpp
3   * @brief Implementation of class MyLogger
4   *
5   * @details
6   * This file contains the implementation of the MyLogger class.
7   * The MyLogger class provides a static method to register loggers and
        log messages.
8   *   - registerLogger registers a logger with a logger name, log level,
        log type, and log file path.
9   *   - log logs a message with a logger name, log level, and message.
10  * The LOG_LEVEL class provides constants for different log levels.
11  * The LOG_TYPE class provides constants for different log types.
```

```
12  *
13  * @author donghy23@mails.tsinghua.edu.cn
14  * @version 1.0
15  */
16
17 #include "carinfo_manager/log.hpp"
18 #include "spdlog/sinks/rotating_file_sink.h"
19 #include "spdlog/sinks/stdout_sinks.h"
20 #include "spdlog/spdlog.h"
21 #ifdef ERROR
22 #undef ERROR
23 #endif
24
25 /**
26  * @brief Register a logger with a logger name, log level, log type,
        and log file path
27  *
28  * @param name The name of the logger
29  * @param level The log level of the logger
30  * @param type The log type of the logger
31  * @param path The log file path (needed when the log type contains
        LOG_TYPE::FILE)
32  *
33  * @return void
34  */
35 void MyLogger::registerLogger(const std::string &name,
36                               const uint8_t level,
37                               const uint8_t type,
38                               const std::string &path) {
39   std::vector<spdlog::sink_ptr> sinks;
40   if (type & LOG_TYPE::CONSOLE) {
41     auto console_sink = std::make_shared<spdlog::sinks::stdout_sink_mt
        >();
42     console_sink->set_level(spdlog::level::info);
43     sinks.push_back(console_sink);
44   }
45   if (type & LOG_TYPE::FILE && !path.empty()) {
46     auto file_sink = std::make_shared<spdlog::sinks::
        rotating_file_sink_mt>(path, 1024 * 1024 * 5, 3);
47     file_sink->set_level(spdlog::level::debug);
48     sinks.push_back(file_sink);
```

```
49    }
50
51    auto logger = std::make_shared<spdlog::logger>(name, sinks.begin(),
          sinks.end());
52    spdlog::flush_on(spdlog::level::debug);
53
54    switch (level) {
55      case LOG_LEVEL::DEBUG:
56        logger->set_level(spdlog::level::debug);
57        break;
58      case LOG_LEVEL::INFO:
59        logger->set_level(spdlog::level::info);
60        break;
61      case LOG_LEVEL::WARN:
62        logger->set_level(spdlog::level::warn);
63        break;
64      case LOG_LEVEL::ERROR:
65        logger->set_level(spdlog::level::err);
66        break;
67      case LOG_LEVEL::CRITICAL:
68        logger->set_level(spdlog::level::critical);
69        break;
70      default:
71        break;
72    }
73    spdlog::register_logger(logger);
74  }
75
76  /**
77   * @brief Log a message with a logger name, log level, and message
78   *
79   * @param name The name of the logger
80   * @param level The log level of the message
81   * @param msg The message to log
82   *
83   * @return void
84   */
85  void MyLogger::log(const std::string &name, const uint8_t level, const
        std::string &msg) {
86    auto logger = spdlog::get(name);
87    if (logger == nullptr)
```

```
    return;
  switch (level) {
    case LOG_LEVEL::DEBUG:
      logger->debug(msg);
      break;
    case LOG_LEVEL::INFO:
      logger->info(msg);
      break;
    case LOG_LEVEL::WARN:
      logger->warn(msg);
      break;
    case LOG_LEVEL::ERROR:
      logger->error(msg);
      break;
    case LOG_LEVEL::CRITICAL:
      logger->critical(msg);
      break;
    default:
      break;
  }
}
```

src/server-main.cpp

```
/**
 * @file src/server-main.cpp
 * @brief Implementation of a server for the carinfo-manager project using httplib library
 *
 * @details
 * This file contains the main entry of the server program.
 * The main function starts the logging system, loads config, data, and starts the server.
 *
 * @author donghy23@mails.tsinghua.edu.cn
 * @version 1.0
 */

#include <filesystem>
#include <fstream>
#include <iostream>
#include <set>
```

```
17 #include "carinfo-manager/accountpool.hpp"
18 #include "carinfo-manager/carpool.hpp"
19 #include "carinfo-manager/hash.hpp"
20 #include "carinfo-manager/httphandler-server.hpp"
21 #include "carinfo-manager/log.hpp"
22 #include "cpp-httplib/httplib.h"
23 #include "json/json.hpp"
24
25 using namespace std;
26 using json = nlohmann::json;
27
28 int main(int argc, char *argv[]) {
29   if (argc != 2) {
30     cout << "Usage: " << argv[0] << " <config_json_file_path>" << endl;
31     return 1;
32   }
33
34   // start logging
35   MyLogger::registerLogger("carinfo-manager-logger",
36                MyLogger::LOG_LEVEL::DEBUG,
37                MyLogger::LOG_TYPE::CONSOLE | MyLogger::LOG_TYPE::FILE,
38                "log/carinfo-manager-server.log");
39
40   // load config
41   ifstream config_file(argv[1]);
42   if (!config_file.is_open()) {
43     MyLogger::log(
44       "carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR, "Cannot
          open config file");
45     return 1;
46   }
47   json config_json_obj = json::parse(config_file);
48   config_file.close();
49   if (!config_json_obj.is_object()) {
50     MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
         "Invalid config file");
51     return 1;
52   }
53   if (config_json_obj.find("dataDir") == config_json_obj.end() ||
54     !config_json_obj["dataDir"].is_string() ||
55     config_json_obj.find("ip") == config_json_obj.end() || !
```

```
      config_json_obj["ip"].is_string() ||
    config_json_obj.find("port") == config_json_obj.end() ||
    !config_json_obj["port"].is_number_unsigned()) {
    MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
        "Invalid config file");
    return 1;
  }
  string dataDir = string(config_json_obj["dataDir"]);
  string ip = string(config_json_obj["ip"]);
  int port = int(config_json_obj["port"]);

  // print config
  MyLogger::log(
    "carinfo-manager-logger",
    MyLogger::LOG_LEVEL::INFO,
    "Using config:\n- dataDir: " + dataDir + "\n- ip: " + ip + "\n-
       port: " + to_string(port));

  // load data
  AccountPool accountpool;
  CarPool carpool;
  ifstream account_file(dataDir + "account.json");
  if (accountpool.load(account_file) != 0) {
    MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
        "Cannot load account data");
    return 1;
  }
  account_file.close();
  ifstream car_file(dataDir + "car.json");
  if (carpool.load(car_file) != 0) {
    MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::ERROR,
        "Cannot load car data");
    return 1;
  }
  car_file.close();

  // modify img files
  set<string> imgFiles;
  for (auto &car : carpool.list())
    imgFiles.insert(car.getImagePath());
  for (auto &file : filesystem::directory_iterator(dataDir + "img/")) {
```

```
92      if (file.is_regular_file()) {
93        string fullPath = file.path().string();
94        if (imgFiles.find(fullPath) == imgFiles.end())
95          filesystem::remove(fullPath);
96      }
97    }
98
99    // config server
100   httplib::Server svr;
101   ServerHttpHandler handler(accountpool, carpool, dataDir + "img/");
102   svr.Get("/test_connection", [&](const httplib::Request &req, httplib
          ::Response &res) {
103     handler.handler_test_connection(req, res);
104   });
105   svr.Post("/test_connection", [&](const httplib::Request &req, httplib
          ::Response &res) {
106     handler.handler_test_connection(req, res);
107   });
108   svr.Post("/login", [&](const httplib::Request &req, httplib::Response
          &res) {
109     handler.handler_login(req, res);
110   });
111   svr.Post("/change_password", [&](const httplib::Request &req, httplib
          ::Response &res) {
112     handler.handler_change_password(req, res);
113     ofstream account_file(dataDir + "account.json");
114     accountpool.save(account_file);
115     account_file.close();
116   });
117   svr.Post("/get_carinfo", [&](const httplib::Request &req, httplib::
          Response &res) {
118     handler.handler_get_carinfo(req, res);
119   });
120   svr.Post("/get_carimg", [&](const httplib::Request &req, httplib::
          Response &res) {
121     handler.handler_get_carimg(req, res);
122   });
123   svr.Post("/add_car", [&](const httplib::Request &req, httplib::
          Response &res) {
124     handler.handler_add_car(req, res);
125     ofstream car_file(dataDir + "car.json");
```

```
126     carpool.save(car_file);
127     car_file.close();
128   });
129   svr.Post("/remove_car", [&](const httplib::Request &req, httplib::
          Response &res) {
130     handler.handler_remove_car(req, res);
131     ofstream car_file(dataDir + "car.json");
132     carpool.save(car_file);
133     car_file.close();
134   });
135   svr.Post("/update_car", [&](const httplib::Request &req, httplib::
          Response &res) {
136     handler.handler_update_car(req, res);
137     ofstream car_file(dataDir + "car.json");
138     carpool.save(car_file);
139     car_file.close();
140   });
141   svr.Post("/get_accountinfo", [&](const httplib::Request &req, httplib
          ::Response &res) {
142     handler.handler_get_accountinfo(req, res);
143   });
144   svr.Post("/get_all_account", [&](const httplib::Request &req, httplib
          ::Response &res) {
145     handler.handler_get_all_account(req, res);
146   });
147   svr.Post("/add_account", [&](const httplib::Request &req, httplib::
          Response &res) {
148     handler.handler_add_account(req, res);
149     ofstream account_file(dataDir + "account.json");
150     accountpool.save(account_file);
151     account_file.close();
152   });
153   svr.Post("/remove_account", [&](const httplib::Request &req, httplib
          ::Response &res) {
154     handler.handler_remove_account(req, res);
155     ofstream account_file(dataDir + "account.json");
156     accountpool.save(account_file);
157     account_file.close();
158   });
159   svr.Post("/update_account", [&](const httplib::Request &req, httplib
          ::Response &res) {
```

```
160     handler.handler_update_account(req, res);
161     ofstream account_file(dataDir + "account.json");
162     accountpool.save(account_file);
163     account_file.close();
164   });
165
166   // start server
167   MyLogger::log("carinfo-manager-logger", MyLogger::LOG_LEVEL::INFO, "
        Server started");
168   svr.listen(ip.c_str(), port);
169   return 0;
170 }
```

C　评分表

项　目	评　价
设计方案的合理性与创新性	___ / 3
设计与调试结果	___ / 4
设计说明书的质量	___ / 1
程序基本要求涵盖情况	___ / 4
程序代码编写素养情况	___ / 2
课程设计周表现情况	___ / 1
综合成绩	___ / 15